

Contents

SmolLM2 Q8_0 GGUF를 Zybo Z7-20에서 돌리기 위한 단계별 프로젝트 가이드 v9	5
0. 핵심 결론	5
0.1 8~9일 프로젝트 핵심 전략	5
0.2 v4 변경점: 직접 양자화보다 Q8_0 GGUF를 먼저 쓴다	6
0.3 v5 변경점: 개념 설명과 사용자 행동 절차를 문서 내부에 통합	7
0.4 v6 변경점: 문서 구조 정리, 혼합 정밀도, KV cache, Linux console 우선	7
0.5 v7 변경점: Q8_0 scale을 FPGA fixed-point로 적용하는 RTL 경로로 수정	7
0.6 v8 변경점: 사용자 행동 지침을 육하원칙 기반 실행 카드로 재작성	8
0.7 v9 변경점: 프롬프트 산출물과 실행 카드 의존성 전수조사	8
1. 왜 SmolLM2인가	9
2. 모델 선택: 135M Q8 vs 360M Q4/Q8	9
2.1 추천 결론	9
2.2 대략적인 메모리 예산	9
2.3 추천 개발 순서	10
2.4 v4 기본 모델 파일	10
3. SmolLM2가 어떻게 돌아가는가	10
3.0 벡터, 행렬, 텐서, 레이어를 먼저 정리한다	10
3.1 전체 흐름	11
3.2 Transformer block 하나	11
3.3 Attention의 의미	12
3.4 MLP의 의미	12
3.5 LM head가 생각보다 무겁다	12
3.6 한 토큰 생성 과정을 끝까지 따라가기	12
4. Zybo Z7-20에서 역할 분담	14
4.1 PS, CPU가 맡을 것	14
4.2 PL, FPGA가 맡을 것	14
4.3 처음부터 PL에 넣지 말 것	14
5. Linux를 쓸까, bare-metal을 쓸까	15
5.1 결론	15
5.2 Linux의 장점	15
5.3 Linux의 단점	15
5.4 bare-metal의 장점과 단점	15
6. dual-core를 어떻게 쓸까	15
6.1 1차 구현	15
6.2 2차 구현	15
7. 전체 시스템 아키텍처	16
8. IP 설계 방향	16
8.1 만들 IP 이름	16
8.2 IP가 해야 할 일	16
8.3 권장 lane 수	17
8.4 데이터 타입 1차 권장: Q8_0 weight라도 RTL 출력은 fixed-point scaled output으로 간다	17
8.5 Q4는 언제 할까	18
9. GEMV IP 상태머신	18
9.0 GEMV란 무엇인가	18
9.0.1 Q8_0 scale-applied GEMV FSM	19
9.0.2 왜 FPGA float 대신 fixed-point인가	20
9.1 AXI-Lite register map 예시	20
10. Vivado에서 해야 할 일	21
10.1 프로젝트 생성	21
10.2 Zynq PS 추가	21
10.3 기본 clock/reset	21
10.4 1차 smoke test용 AXI-Lite IP	21
10.5 GEMV BRAM 테스트 IP	21
10.6 AXI DMA 추가 단계	21
11. PetaLinux/Linux 구성	22
11.1 Linux를 쓰는 이유	22
11.2 메모리 부족 걱정	22
11.3 SD 카드 구성	22
11.4 HDMI + USB keyboard가 이미 된다면 Linux console chat을 1차 UI로 간다	22
11.5 UART serial은 버리지 말고 비상 디버그용으로 남긴다	23

12. PC/PyCharm에서 해야 할 GGUF 다운로드와 FPGA layout 변환 작업	23
12.0 GGUF란 무엇인가	23
12.1 PC에서 할 일, v4 GGUF-first	24
12.2 생략되는 직접 양자화 파트	24
12.3 Q8_0 GGUF가 그대로 FPGA용이 아닌 이유	24
12.4 Q8_0 block reference와 v7 scale 정책	25
12.5 360M 확장 판단	25
12.6 최종 산출물	25
12.7 Q8_0인데 왜 int16, int32, fixed-point, float32가 함께 나오나	26
13. 개발 마일스톤	26
Milestone 0 - PC reference	26
Milestone 1 - Quantized Python reference	26
Milestone 2 - C reference runtime	26
Milestone 3 - Vivado AXI-Lite smoke test	27
Milestone 4 - 작은 GEMV IP	27
Milestone 5 - AXI DMA weight stream	27
Milestone 6 - SmolLM2 135M의 Linear 하나 가속	27
Milestone 7 - 모든 Linear를 PL로 교체	27
Milestone 8 - UI 확장	27
Milestone 9 - 360M Q4 확장	28
13.1 바닥부터 천장까지 누락 보강 체크리스트	28
13.1.1 전체 파이프라인 산출물	28
13.1.2 S급 필수 누락 보강 항목	28
13.1.3 CPU-only 기준선	29
13.1.4 tensor 이름 매핑표	29
13.1.5 추천 모델/quant 포맷, v4 기준	29
13.1.6 golden vector 규칙: FPGA 검증용 정답지	30
13.1.7 FPGA용 weight/scale layout	31
13.1.8 PS-PL ABI와 register map	32
13.1.9 AXI-Stream weight/scale protocol	33
13.1.10 Linux device tree와 DMA/cache	33
13.1.11 SmolLM2 layer scheduler	34
13.1.12 KV cache 구조: Linux runtime이 관리한다	34
13.1.13 Linux console, HDMI/USB, UART UI 경로	35
13.1.14 실패 조건별 fallback	35
13.1.15 최종 시연 체크리스트	35
14. Codex 전역 프로젝트 지침	36
15. Codex 단계별 프롬프트 통합본 v9: Q8_0 GGUF-first + dependency-audited execution	36
Prompt 00 - 프로젝트 폴더와 PC Python 환경 만들기	36
Prompt 01 - Q8_0 GGUF 모델 다운로드 스크립트	37
Prompt 02 - GGUF 파일 검사 및 metadata 덤프	38
Prompt 03 - Tokenizer smoke test	38
Prompt 04 - GGUF tensor name mapping 생성	39
Prompt 05 - GGUF Q8_0 block decoder reference	39
Prompt 06 - Q8_0 GGUF -> FPGA lane layout 변환기	40
Prompt 07 - pre-quantized model 기준 golden vector 생성기	41
Prompt 08 - C reference GEMV, GGUF Q8_0 layout 기준	42
Prompt 09 - Verilog GEMV, Q8_0 fixed-point scale 적용 기준	42
Prompt 10 - AXI-Stream weight/scale protocol 명세 고정	44
Prompt 11 - PS-PL ABI header와 register map	44
Prompt 12 - 1-lane GEMV core smoke test	45
Prompt 13 - 16-lane GEMV 확장	45
Prompt 14 - AXI-Lite wrapper	45
Prompt 15 - AXI DMA + AXI4-Stream FIFO 통합용 RTL wrapper	46
Prompt 16 - Vivado block design TCL 초안	46
Prompt 17 - Linux device tree: UIO, DMA, reserved-memory	47
Prompt 18 - Linux /dev/mem 또는 UIO control app	47
Prompt 19 - DMA buffer와 cache coherency 처리	47
Prompt 20 - Linux DMA weight stream app	48
Prompt 21 - CPU-only minimal runtime	48
Prompt 22 - runtime에서 GEMV 호출부 추상화	48
Prompt 23 - RMSNorm/RoPE/Attention/MLP CPU 구현 검증	49

Prompt 24 - KV cache 구현	49
Prompt 25 - Linux console chat 프로그램	49
Prompt 26 - HDMI/USB console 확인과 UI polish	49
Prompt 27 - Vivado utilization/timing report 파서	50
Prompt 28 - 성능 측정 로거	50
Prompt 29 - 실패 조건별 fallback 스크립트	50
Prompt 30 - 최종 demo checklist 생성	50
Prompt 31 - 전체 regression runner	50
Prompt 32 - 직접 양자화 fallback appendix	51
Prompt 33 - 360M Q4_K_M 확장 설계	51
Prompt 34 - 문서 업데이트 자동화	51
Prompt 35 - 최종 통합 검토	51
15.1 각 Codex 프롬프트 사이에서 사용자가 해야 할 행동 - 육하원칙 실행 카드	52
15.1.0 먼저 고정할 실행 위치 이름	52
15.1.0.1 프롬프트 사이 검증에 Codex를 쓰는 공통 방법	52
15.1.1 Prompt 00 전후 - 프로젝트 폴더와 Python 환경	53
15.1.2 Prompt 01 전후 - Q8_0 GGUF 다운로드	53
15.1.3 Prompt 02 전후 - GGUF metadata와 tensor 목록 검사	54
15.1.4 Prompt 03 전후 - Tokenizer smoke test	54
15.1.5 Prompt 04 전후 - GGUF tensor name mapping	55
15.1.6 Prompt 05 전후 - Q8_0 decoder reference	55
15.1.7 Prompt 06 전후 - FPGA lane layout 변환	56
15.1.8 Prompt 07 전후 - Golden vector 생성	56
15.1.9 Prompt 08 전후 - C reference GEMV	57
15.1.10 Prompt 09 전후 - Verilog GEMV core	58
15.1.11 Prompt 10 전후 - AXI-Stream weight/scale protocol	58
15.1.12 Prompt 11 전후 - PS-PL ABI와 register map	59
15.1.13 Prompt 12 전후 - 1-lane smoke test	59
15.1.14 Prompt 13 전후 - 16-lane 확장	60
15.1.15 Prompt 14 전후 - AXI-Lite wrapper	60
15.1.16 Prompt 15 전후 - DMA/FIFO wrapper	61
15.1.17 Prompt 16 전후 - Vivado block design	61
15.1.18 Prompt 17 전후 - Device tree	61
15.1.19 Prompt 18 전후 - Linux register access	62
15.1.20 Prompt 19 전후 - DMA buffer/cache coherency	62
15.1.21 Prompt 20 전후 - Linux DMA weight stream app	62
15.1.22 Prompt 21 전후 - CPU-only minimal runtime	63
15.1.23 Prompt 22 전후 - GEMV backend 추상화	63
15.1.24 Prompt 23 전후 - CPU 부가 연산 검증	63
15.1.25 Prompt 24 전후 - KV cache	64
15.1.26 Prompt 25 전후 - Linux console chat	64
15.1.27 Prompt 26 전후 - HDMI/USB console 확인과 UI polish	64
15.1.28 Prompt 27 전후 - Vivado report parser	65
15.1.29 Prompt 28 전후 - 성능 측정 logger	65
15.1.30 Prompt 29 전후 - fallback script	65
15.1.31 Prompt 30 전후 - 최종 demo checklist	65
15.1.32 Prompt 31 전후 - 전체 regression runner	65
15.1.33 Prompt 32 전후 - 직접 양자화 fallback	66
15.1.34 Prompt 33 전후 - 360M Q4_K_M 확장	66
15.1.35 Prompt 34 전후 - 문서 업데이트 자동화	66
15.1.36 Prompt 35 전후 - 최종 통합 검토	66
15.1.37 중요한 저장점	66
16. 프로젝트 디렉터리 구조	67
17. 수치 검증 전략	68
17.1 작은 test vector	68
17.2 실제 크기 test vector	68
17.3 성공 기준	69
18. 속도 감각	69
18.1 135M 근사	69
18.2 360M 근사	69
18.3 lane 수별 감각	69
19. 자주 터지는 문제와 예방책	70

20. 최종 UI 계획	70
20.1 1차: Linux console chat, HDMI/USB keyboard 우선	70
20.2 2차: SSH 또는 network console	70
20.3 UART serial은 boot/debug fallback	70
21. 8~9일 초단기 애자일 일정	70
21.1 전체 원칙	71
21.2 Day 1 - Q8_0 GGUF 다운로드와 모델 파일 검사	71
21.3 Day 2 - Q8_0 decoder, FPGA layout, golden vector	71
21.4 Day 3 - Zybo Linux 부팅, HDMI/USB console, SD 파일 읽기	71
21.5 Day 4 - CPU-only C runtime 최소화	72
21.6 Day 5 - Vivado AXI-Lite + 작은 Q8_0 fixed-scale GEMV IP	72
21.7 Day 6 - Linux에서 GEMV IP 호출	72
21.8 Day 7 - AXI DMA 또는 FIFO 기반 Q8_0 packet stream 1차 연결	73
21.9 Day 8 - SmolLM2 runtime에 PL GEMV 붙이기	73
21.10 Day 9 - 안정화와 발표용 확장	73
21.11 8일 프로젝트에서 포기해야 할 것	73
22. 변화: AXI master를 직접 만들고 싶다면	74
23. Linux 설치 절차: Zybo Z7-20 기준	74
23.1 권장 결론	74
23.2 준비물	75
23.3 Vivado에서 XSA 만들기	75
23.4 PetaLinux 프로젝트 생성 흐름	75
23.5 SD 카드 구성	76
23.6 첫 부팅 체크	76
23.7 PL register 접근 방법	77
23.8 Linux가 난이도를 올리나, 내리나	77
23.9 1GB DDR이 Linux 때문에 부족할까	77
24. Transformer 용어 사전: QKV, Embedding, Attention, RMSNorm	78
24.1 Token	78
24.2 Embedding	78
24.3 Positional Encoding과 RoPE	78
24.4 Q, K, V	78
24.5 Attention	79
24.6 Multi-Head Attention	79
24.7 KV Cache	79
24.8 RMSNorm	79
24.9 MLP, SwiGLU, SiLU	79
24.10 LM Head와 logits	80
24.11 왜 GEMV가 핵심인가	80
24.12 GEMV	80
24.13 GGUF	80
25. 리소스 사용률 예상과 성능 확장 루트	80
25.1 예상 사용률	80
25.2 성능 확장 우선순위	81
25.3 135M과 360M 확장 판단	81
26. 버퍼, DMA, Vivado IP 활용 전략	81
26.1 ping-pong buffer와 ring buffer	81
26.2 데이터 종류별 권장 버퍼	82
26.3 최대한 활용할 Vivado IP	82
26.4 v1 권장 Block Design	82
26.5 v2 권장 Block Design	82
26.6 weight layout은 PC에서 미리 바뀌라	82
26.7 valid/ready 규칙	83
26.8 ILA로 반드시 볼 신호	83
26.9 v9 파일 의존성 전수조사 요약	84
27. 참고자료	84

SmolLM2 Q8_0 GGUF를 Zybo Z7-20에서 돌리기 위한 단계별 프로젝트 가이드 v9

대상 보드: Digilent Zybo Z7-20 / Zynq-7020

목표 모델: SmolLM2-135M-Instruct Q8_0 GGUF 우선, SmolLM2-360M Q4_K_M/Q8_0 확장

목표 방식: PS Linux + PL GEMV 가속기 + SD 카드 모델 저장 + DDR 작업 메모리

작성 목적: 이 문서만 보고 프로젝트를 단계적으로 진행하고, Codex에 단계별 코드를 요청할 수 있게 만드는 실행 가이드

패치/검증 버전: v9 full dependency audit patch export 20260626

중요: 이 파일은 v8 문서를 기준으로 전수조사한 뒤, 프롬프트 산출물과 사용자 실행 명령 사이의 누락/불일치를 고친 v9 패치본이다. 특히 실행 카드에서 요구하는 스크립트와 binary가 어느 Prompt에서 생성되는지 명확히 연결했다.

0. 핵심 결론

이 프로젝트의 빠른 성공 루트는 다음과 같다.

1. 처음 완성 목표는 이미 양자화된 SmolLM2-135M-Instruct Q8_0 GGUF 모델로 잡는다.
2. 360M은 처음부터 가지 말고, 135M으로 PS-Linux-SD-DDR-PL-GEMV 파이프라인이 완성된 뒤 확장한다.
3. FPGA에는 Transformer 전체를 만들지 않는다. PL은 행렬-벡터 곱, 즉 GEMV 전용 가속기로 둔다.
4. CPU, 즉 Zynq PS의 ARM Cortex-A9는 RMSNorm, RoPE, SiLU/SwiGLU, softmax, sampling, tokenizer, 파일 입출력, 스케줄링을 맡긴다.
5. Linux는 처음에는 복잡해 보이지만, SD 카드 파일, USB 키보드, HDMI 콘솔/프레임버퍼, 멀티스레드, 모델 파일 관리 때문에 최종 완성 단계에서는 bare-metal보다 유리하다.
6. Vivado에서는 처음부터 AXI master를 만들지 않는다. AXI-Lite control + AXI DMA + AXI-Stream GEMV IP를 목표로 한다. AXI master는 시간 남으면 하는 번외편이다.
7. Codex에게는 절대 "SmolLM2 전체 포팅해줘"라고 시키지 않는다. 작은 모듈, 작은 테스트, 기준 Python 코드를 붙여서 단계별로 시킨다.

프로젝트의 한 줄 요약:

PS Linux는 SmolLM2 런타임을 실행하고,
PL FPGA는 가장 무거운 Linear/GEMV만 반복 가속한다.

0.1 8~9일 프로젝트 핵심 전략

이 프로젝트는 몇 주짜리 연구 프로젝트가 아니라 8~9일 안에 결과물을 만들어야 하는 초단기 MVP 프로젝트다. 따라서 문서의 모든 선택은 다음 원칙을 우선한다.

최우선 목표:

Zybo Z7-20에서 SmolLM2 계열 모델이 Linux console chat 형태로 실제로 한 토크씩 생성된다. UART serial은 부팅/복구용 fa

해야 하는 것:

PC에서 Q8_0 GGUF 다운로드/검사/FPGA용 weight layout 변환
Linux에서 SD 카드 모델 파일 읽기
PS에서 Transformer runtime 실행
PL에서 GEMV/Linear 일부 또는 전체 가속
Linux console 또는 SSH에서 대화 검증. UART serial은 fallback

하지 않는 것:

처음부터 HDMI/USB UI 완성
처음부터 360M Q8 고집
처음부터 AXI master 직접 구현
처음부터 RMSNorm/RoPE/Softmax를 PL로 이관
처음부터 완벽한 고성능 DMA pipeline 구현

8~9일 프로젝트에서 성공 기준은 "최적 성능"이 아니라 끝까지 돌아가는 수직 슬라이스다. 따라서 처음에는 느려도 괜찮다. CPU-only reference, 작은 GEMV, Linux console chat, PL 가속을 순서대로 붙인다.

권장 MVP는 다음과 같다.

Day 1~2:

PC에서 SmoLLM2-135M-Instruct Q8_0 GGUF 다운로드, 검사, reference 준비

Day 3~4:

Zybo Linux 부팅, SD 파일 읽기, HDMI/USB Linux console 확인, UART fallback 확보

Day 5~6:

Vivado에서 AXI-Lite + 작은 GEMV IP 검증

Day 7:

Linux user-space에서 GEMV IP 호출

Day 8:

SmoLLM2 runtime의 Linear 일부를 PL GEMV로 교체

Day 9, 있으면:

안정화, 발표용 demo, HDMI/USB 또는 성능 개선 일부

8일 안에 모든 Linear를 완벽히 PL로 옮기지 못하더라도, **CPU runtime + PL GEMV 호출 경로가 실제로 동작하면** 프로젝트는 살릴 수 있다. 반대로 고성능 DMA/360M/HDMI부터 건드리면 완성 실패 위험이 커진다.

0.2 v4 변경점: 직접 양자화보다 Q8_0 GGUF를 먼저 쓴다

v4부터 기본 루트는 **BF16 원본을 받아 직접 INT8 양자화**하는 방식이 아니다. 8~9일 프로젝트에서는 이미 공개된 **SmoLLM2-135M-Instruct Q8_0 GGUF**를 먼저 받아서 쓴다.

기존 v3 기본 루트:

- BF16 safetensors 다운로드
- 직접 INT8 양자화
- scale 정책 설계
- FPGA layout 변환

v4 기본 루트:

- Q8_0 GGUF 다운로드
- GGUF metadata/tensor 검사
- Q8_0 block decode reference 작성
- FPGA lane layout 변환
- golden vector 생성

이 변경으로 생략되는 것:

- BF16 safetensors를 직접 INT8로 양자화하는 실험
- per-row/per-group scale 정책 탐색
- 양자화 전후 오차 분석의 상당 부분
- 자체 quant format 설계 1차 버전

대신 새로 필요한 것:

- GGUF metadata와 tensor 목록 검사
- Q8_0 block 구조 이해
- GGUF tensor name -> 내부 tensor name 매핑
- GGUF Q8_0 -> FPGA lane16 stream layout 변환
- GGUF에서 뽑은 weight로 golden vector 생성

핵심 판단:

프로젝트 완성 우선:

SmoLLM2-135M-Instruct Q8_0 GGUF

시간이 남을 때 품질 확장:

SmoLLM2-360M-Instruct Q4_K_M GGUF

처음부터 피할 것:

- 360M Q8_0 기본 목표
- BF16 원본 직접 양자화부터 시작

직접 양자화는 여전히 의미가 있지만, 8~9일짜리 본편이 아니라 **변외/확장 작업**으로 둔다.

0.3 v5 변경점: 개념 설명과 사용자 행동 절차를 문서 내부에 통합

v5에서는 기존 v4의 Q8_0 GGUF-first 경로를 유지하면서, 문서만 보고 따라갈 때 막히기 쉬운 부분을 추가했다.

이번 패치의 삽입/병합 정책은 다음과 같다.

추가 내용	문서 위치	처리 방식
벡터, 행렬, 텐서, 레이어의 관계	3장 앞부분	기존 SmolLM2 동작 설명에 3.0으로 병합
한 토큰이 생성되는 전체 흐름	3장 후반	3.6으로 추가
GEMV의 뜻과 이 프로젝트에서의 의미	9장 앞부분	GEMV IP 설명 전에 9.0으로 추가
GGUF의 뜻과 왜 변환이 필요한지	12장 앞부분	PC/PyCharm 작업 설명 전에 12.0으로 추가
GEMV/GGUF 용어 사전 항목	24장 후반	기존 Transformer 용어 사전에 24.12, 24.13으로 추가
각 Codex 프롬프트 사이에서 사용자가 해야 할 일	15장 뒤	15.1 장으로 추가

핵심 원칙은 다음과 같다.

Codex는 작성자다.

사용자는 빌드/실행/검증/승인 담당자다.

각 Prompt가 끝날 때마다 사용자는 결과를 실행하고, golden/reference와 비교하고, 증거를 남긴 뒤 다음 Prompt로 넘어간다.

0.4 v6 변경점: 문서 구조 정리, 혼합 정밀도, KV cache, Linux console 우선

이 v6는 v5에서 흩어져 있던 설명을 읽기 쉬운 흐름으로 정리한다. 추가/정리한 핵심은 다음과 같다.

- Golden vector, ABI, KV cache의 의미를 본문 안에 더 명확히 설명
- PC/PyCharm, RTL simulation, Vivado, 보드 Linux의 경계를 한눈에 보이게 정리
- 팀원이 준비한 Linux가 HDMI framebuffer와 USB HID를 이미 지원하는 상황을 반영
- 1차 UI를 UART serial chat이 아니라 Linux console chat으로 변경
- UART serial은 부팅 로그와 비상 디버그용 fallback으로 재정의
- Q8_0 모델에서 int8, int16, int32, float32가 왜 함께 나오는지 정리
- KV cache는 Linux C runtime이 DDR/RAM에서 관리하고, MVP context는 128~256으로 시작하도록 명확화
- Codex는 코드 작성자, 사용자는 실행/검증/판정자라는 역할 분담을 강화

이 문서는 여전히 8~9일짜리 MVP를 기준으로 한다. 따라서 처음부터 모든 것을 정수화하거나, 모든 UI를 직접 framebuffer/HID API로 만들거나, 최대 context 8192를 무조건 할당하는 식의 고위험 선택은 기본 경로에서 제외한다.

0.5 v7 변경점: Q8_0 scale을 FPGA fixed-point로 적용하는 RTL 경로로 수정

v7의 가장 큰 변경은 Prompt 09의 RTL 목표를 raw int32 accumulator만 출력하는 방식에서 Q8_0 scale을 FPGA 내부에서 fixed-point로 적용하는 방식으로 바꾼 것이다. 다만 Verilog에서 floating-point 연산기를 직접 쓰는 방향은 기본 경로에서 제외한다.

기존 v6 기본 경로:

```
FPGA: int16 activation x int8 weight -> raw int32 accumulator 출력
CPU : Q8_0 scale 적용 후 float32 후처리
```

v7 기본 경로:

```
PC : GGUF Q8_0 scale을 fixed-point scale_q로 변환
FPGA: int16 x int8 -> block_acc_i32 -> scale_q 적용 -> row_output_i32 출력
CPU : FPGA가 낸 scaled output을 float32 runtime으로 이어 받아 후속 연산 처리
```

이 변경으로 영향을 받는 부분은 Prompt 09 하나가 아니다. 다음 섹션도 같이 정리했다.

- 8.4 데이터 타입 정책
- 9.0 GEMV IP 상태머신
- 9.1 register map
- 12.3~12.7 GGUF 변환과 mixed precision 설명
- 13.1.6 golden vector

- 13.1.7 FPGA weight/scale layout
- 13.1.8 PS-PL ABI/register map
- 13.1.9 AXI-Stream protocol
- Prompt 06~13
- Prompt 15.1 사용자 행동표
- 17 수치 검증 전략
- 21 Day 5~8 일정
- 26 DMA/buffer/IP 전략

v7의 원칙은 다음과 같다.

- 기본 동작 mode=0은 scaled row output이다.
- 디버그 mode=1은 block_acc_i32 debug output이다.
- scale은 PC에서 scale_q fixed-point로 미리 변환한다.
- FPGA 내부 row_acc는 48-bit 이상으로 잡고, 최종 output_i32로 saturation/rounding한다.
- block_acc debug mode는 반드시 남겨 scale/weight alignment 문제를 추적할 수 있게 한다.

0.6 v8 변경점: 사용자 행동 지침을 육하원칙 기반 실행 카드로 재작성

v7의 15.1 사용자 행동표는 명령어와 통과 기준은 들어 있었지만, 초보자가 읽었을 때 **어디서, 왜, 무엇을, 어떤 순서로, 결과를 어떻게 판단해야 하는지**가 부족했다. v8에서는 해당 장을 단순 명령어 목록이 아니라 실행 카드 형태로 재작성한다.

v8의 사용자 행동 지침은 각 Prompt마다 다음 순서를 따른다.

왜 하는가:

이 단계가 전체 프로젝트에서 필요한 이유

누가 하는가:

사용자, 팀원 A, 팀원 B, Codex 중 책임 주체

언제 하는가:

Codex 프롬프트 실행 전/후, 또는 특정 산출물 생성 후

어디서 하는가:

PC/PyCharm shell, Vivado PC, Zybo Linux console, SSH 중 실제 실행 위치

무엇을 입력으로 쓰는가:

필요한 파일과 폴더

어떻게 실행하는가:

cd 위치부터 시작하는 실행 명령

무엇을 확인하는가:

화면 출력, 파일 존재, PASS 로그, 크기, diff, report

실패하면 무엇을 해야 하는가:

다음 단계로 넘어가지 말아야 할 조건과 Codex에 다시 줄 로그

이 변경은 기능 설계를 바꾸는 패치가 아니다. v7의 Q8_0 fixed-scale RTL 방향은 유지하고, 사용자가 실제로 따라할 수 있게 절차 설명을 정리한다.

0.7 v9 변경점: 프롬프트 산출물과 실행 카드 의존성 전수조사

v9에서는 문서 전체를 다시 훑어서, 사용자 행동 카드에서 실행하라고 한 파일이 실제로 어느 Prompt에서 생성되는지를 맞췄다. v8에는 scripts/run_gemv_sim.tcl처럼 실행 카드에는 나오지만 Prompt 산출물 요구에는 빠져 있던 항목이 있었다. 이런 항목은 사용자가 실행 단계에서 바로 막히므로 v9에서 전수 보정한다.

v9의 수정 원칙은 다음과 같다.

1. 실행 카드가 어떤 파일을 실행한다면, 그 파일은 반드시 앞선 Prompt의 산출물에 들어간다.
2. 파일이 없을 때는 다음 단계로 넘어가지 않고, Codex에게 누락 산출물을 생성하게 한다.

3. 각 Prompt 사이 검증도 Codex를 활용한다. 단, Codex의 말이 아니라 사용자의 실제 실행 로그가 최종 기준이다.
4. scripts/*.tcl, runtime_c/CMakeLists.txt, linux_app/Makefile처럼 실행에 필요한 보조 파일을 명시한다.
5. 중복 제목과 낡은 경로를 정리한다.

이번 전수조사에서 핵심적으로 고친 항목은 다음과 같다.

- Prompt 04: validate_tensor_map.py 생성 요구 추가
- Prompt 07: generate_golden_from_gguf.py 위치와 실행 카드 경로 일치
- Prompt 08: runtime_c/CMakeLists.txt, test runner 생성 요구 추가
- Prompt 09: scripts/run_gemv_sim.tcl, docs/rtl_sim_howto.md 생성 요구 추가
- Prompt 10: weight stream protocol test TCL 생성 요구 추가
- Prompt 12: scripts/run_gemv_sim_1lane.tcl 생성 요구 추가
- Prompt 13: scripts/run_gemv_sim_multilane.tcl 생성 요구 추가
- Prompt 14/15: wrapper 단독 test TCL 생성 요구 추가
- Prompt 16: scripts/create_block_design.tcl wrapper와 vivado_ip/scripts/create_bd_gemv_q8_0.tcl 생성 요구 정렬
- Prompt 18/20: Linux app Makefile과 실행 binary 이름 정렬
- 15.1: 각 단계마다 "파일 존재 확인 -> 없으면 Codex 수정 -> 있으면 실행" 흐름 추가

1. 왜 SmolLM2인가

SmolLM2는 Hugging Face가 공개한 소형 언어모델 계열로, 135M, 360M, 1.7B 크기가 있다. 모델 카드는 이 계열이 온디바이스 실행을 목표로 하는 compact language model family라고 설명한다. [R1]

SmolLM2-135M 설정은 LLaMA 계열 LlamaForCausalLM이며, 주요 파라미터는 hidden size 576, intermediate size 1536, layer 30개, attention head 9개, KV head 3개, vocab size 49152, torch dtype bfloat16이다. [R2]

SmolLM2-360M 설정은 hidden size 960, intermediate size 2560, layer 32개, attention head 15개, KV head 5개, vocab size 49152, torch dtype bfloat16이다. [R3]

즉 원본 모델은 8비트 정수 모델이 아니다. 원본은 BF16 계열이다. 이 프로젝트의 기본 경로는 직접 BF16을 양자화하는 것이 아니라, 이미 Q8_0으로 양자화된 GGUF를 받아서 PC에서 FPGA용 layout과 scale_q로 변환하는 것이다. 직접 양자화는 fallback/appendix로만 둔다.

2. 모델 선택: 135M Q8 vs 360M Q4/Q8

2.1 추천 결론

선택지	추천도	이유
SmolLM2-135M-Instruct Q8	매우 높음	처음 완성용. 메모리 여유, 구현 단순성, 품질 균형이 좋다.
SmolLM2-135M-Instruct Q4	중간	메모리는 줄지만 135M에서는 Q8이 더 안전하다. Q4 디코더 구현 부담이 생긴다.
SmolLM2-360M-Instruct Q4	높음, 2차 목표	135M 완성 후 확장용. 메모리는 가능권, 품질은 더 좋을 가능성이 크다.
SmolLM2-360M-Instruct Q8	낮음~중간	DDR에는 들어가지만 Linux, KV cache, 버퍼, 대역폭까지 생각하면 여유가 없다.

처음부터 360M Q8을 잡으면, 모델 크기보다 **디버깅 복잡도**가 문제다. 행렬 크기가 커지고 layer 수도 많아서, 버그 하나 찾는 데 시간이 훨씬 늘어난다.

2.2 대략적인 메모리 예산

정확한 파일 크기는 양자화 포맷에 따라 달라진다. 아래는 설계 예산용 근사치다.

모델	Q8 weight 예상	Q4 weight 예상	비고
135M	약 135~170 MB	약 75~100 MB	Linux 포함해도 1 GB DDR에서 여유 있음
360M	약 360~430 MB	약 200~260 MB	Q4 권장, Q8은 가능하지만 여유 감소

KV cache는 context 길이에 따라 달라지지만, weight에 비하면 작다. 예를 들어 135M, context 512, K/V int16 기준이면 대략 수십 MB 미만이다. 360M도 context 512 수준에서는 DDR을 터뜨리는 크기는 아니다. 문제는 weight와 Linux, 입출력 버퍼, DMA buffer가 동시에 올라간다는 점이다.

2.3 추천 개발 순서

1단계: 135M Q8 CPU-only C reference
2단계: 135M Q8 + PL GEMV 가속
3단계: 135M Q8 완전체 채팅
4단계: 360M Q4로 모델 교체
5단계: 시간이 남으면 360M Q8 또는 더 좋은 quant 방식 실험

2.4 v4 기본 모델 파일

v4 기본 모델 파일은 다음처럼 잡는다.

기본 repo:

lmstudio-community/SmolLM2-135M-Instruct-GGUF

기본 파일:

SmolLM2-135M-Instruct-Q8_0.gguf

대체 repo 후보:

bartowski/SmolLM2-135M-Instruct-GGUF

360M 확장 후보:

HuggingFaceTB/SmolLM2-360M-Instruct-GGUF

bartowski/SmolLM2-360M-Instruct-GGUF

unsloth/SmolLM2-360M-Instruct-GGUF

주의:

GGUF Q8_0을 받는다고 해서 FPGA가 바로 읽을 수 있는 것은 아니다.

GGUF는 llama.cpp/ggml 런타임용 컨테이너다.

따라서 PC에서 Q8_0 tensor를 읽고, GEMV IP가 먹기 좋은 lane layout으로 재배열해야 한다.

3. SmolLM2가 어떻게 돌아가는가

3.0 벡터, 행렬, 텐서, 레이어를 먼저 정리한다

Transformer 설명에서 벡터, 행렬, 텐서가 계속 튀어나오는 이유는 모델 내부 표현이 전부 숫자 배열이기 때문이다. DNN/CNN을 조금 알고 있다면 다음처럼 잡으면 된다.

벡터(vector): 숫자 1줄

행렬(matrix): 숫자 2차원 표

텐서(tensor): 벡터/행렬까지 포함하는 n차원 숫자 덩어리

레이어(layer): 행렬 하나가 아니라, 여러 행렬 연산과 비선형/정규화/잔차 연결을 묶은 계산 블록

따라서 각 레이어가 매트릭스이고 그게 쌓여 텐서가 된다고 보면 조금 틀리다. 더 정확한 그림은 다음과 같다.

모델 전체

- ├─ embedding table
- ├─ layer 0
 - ├─ q_proj weight matrix
 - ├─ k_proj weight matrix
 - ├─ v_proj weight matrix
 - ├─ o_proj weight matrix
 - ├─ gate_proj weight matrix
 - ├─ up_proj weight matrix
 - ├─ down_proj weight matrix
 - ├─ input RMSNorm weight vector
 - └─ post-attention RMSNorm weight vector
- ├─ layer 1
 - └─ 같은 구조 반복
- ├─ ...
- ├─ final RMSNorm weight vector
- └─ lm_head/output weight matrix

즉 레이어 안에 행렬이 여러 개 들어있고, 그 행렬들도 넓은 의미로는 tensor다. GGUF 파일에서 보이는 `tensor name`은 이런 weight matrix/vector 하나하나의 이름이라고 보면 된다.

벡터가 중요한 이유는 토큰 하나가 모델 내부에서 hidden vector 하나로 표현되기 때문이다. 예를 들어 텍스트가 tokenizer를 지나면 숫자 ID가 된다.

"hello" -> token id 15339

하지만 신경망은 ID 하나만으로 의미 계산을 할 수 없다. 그래서 embedding table에서 해당 ID에 대응되는 숫자 벡터를 꺼낸다.

```
token id 15339
↓ embedding lookup
[0.12, -0.03, 0.55, ...] // hidden size 길이의 벡터
```

SmolLM2-135M 계열에서는 hidden size가 576이므로, 감각적으로는 토큰 하나가 길이 576짜리 벡터 하나가 된다.

여러 토큰을 한 번에 보면 전체 hidden state는 다음과 같은 2차원 배열이 된다.

[토큰 수, hidden size]

하지만 자동 회귀 생성에서는 새 토큰을 하나씩 만들기 때문에, 현재 단계에서는 주로 마지막 토큰의 hidden vector 하나를 다음 레이어들에 통과시킨다. 이 때문에 행렬 곱셈도 행렬 $\times$ 벡터, 즉 GEMV 형태가 핵심이 된다.

SmolLM2는 GPT류 decoder-only Transformer다. 입력 문장을 토큰 ID 배열로 바꾸고, 각 토큰을 벡터로 바꾼 뒤, 여러 Transformer block을 통과시켜 다음 토큰의 점수를 계산한다.

3.1 전체 흐름

```
문자열 입력
↓
tokenizer
↓
토큰 ID 배열
↓
embedding lookup
↓
Transformer block x N
↓
final RMSNorm
↓
LM head, 즉 vocab 방향 Linear
↓
logits[49152]
↓
softmax/top-k/top-p/sampling
↓
다음 토큰 선택
```

135M은 30 layer, 360M은 32 layer다. [R2][R3]

3.2 Transformer block 하나

LLaMA 계열 block은 대략 다음 구조다.

```
입력 x
↓
RMSNorm
↓
Self-Attention
↓
Residual:  $x = x + \text{attention\_out}$ 
↓
RMSNorm
↓
MLP, 보통 gate/up/down 구조
↓
Residual:  $x = x + \text{mlp\_out}$ 
```

하드웨어 관점에서 가장 중요한 것은 대부분의 시간이 Linear/GEMV에서 쓰인다는 점이다.

3.3 Attention의 의미

Attention은 현재 토큰이 이전 토큰들 중 무엇을 얼마나 참고할지 계산한다.

$Q = q_proj(x)$

$K = k_proj(x)$

$V = v_proj(x)$

$score[t] = dot(Q, K_cache[t])$

$weight[t] = softmax(score[t])$

$attention_out = sum(weight[t] * V_cache[t])$

현재 토큰의 K/V는 cache에 저장되고, 다음 토큰이 들어왔을 때 재사용된다. 그래서 token-by-token 생성에서는 K/V cache가 중요하다.

3.4 MLP의 의미

MLP는 토큰 간 정보를 섞는 것이 아니라, 현재 토큰 벡터 내부를 강하게 변환한다.

SmolLM2-135M 기준:

hidden 576

intermediate 1536

SmolLM2-360M 기준:

hidden 960

intermediate 2560

구조는 보통 다음과 같은 형태다.

$gate = Linear_gate(x)$

$up = Linear_up(x)$

$hidden = silu(gate) * up$

$out = Linear_down(hidden)$

FPGA에서 처음부터 SiLU까지 구현하려고 하면 귀찮다. 그래서 첫 버전에서는 gate/up/down의 Linear만 PL로 보내고, SiLU와 elementwise multiply는 CPU에서 처리한다.

3.5 LM head가 생각보다 무겁다

마지막에 hidden vector를 vocab size 49152개의 점수로 바꾼다.

135M 기준:

576 -> 49152

약 28.3M MAC

360M 기준:

960 -> 49152

약 47.2M MAC

따라서 LM head도 CPU에 맡기면 느려진다. 최종적으로는 LM head도 PL GEMV로 보내는 것이 좋다.

3.6 한 토큰 생성 과정을 끝까지 따라가기

SmolLM2가 한 토큰을 생성하는 전체 흐름을 아주 거칠게 쓰면 다음과 같다.

사용자 문장

↓

tokenizer

↓

token id 배열

↓

embedding lookup

↓
 hidden vector
 ↓
 Transformer layer 0
 ↓
 Transformer layer 1
 ↓
 ...
 ↓
 Transformer layer 29
 ↓
 final RMSNorm
 ↓
 lm_head GEMV
 ↓
 vocab 전체 logits
 ↓
 다음 token 선택
 ↓
 그 token을 문장 뒤에 붙이고 반복
 레이어 하나 안에서는 다음 계산이 반복된다.

입력 hidden vector x
 ↓
 $\text{RMSNorm}(x)$
 ↓
 $q = W_q x$
 $k = W_k x$
 $v = W_v x$
 ↓
 $\text{RoPE}(q, k)$
 ↓
 KV cache에 k/v 저장
 ↓
 q 와 과거 k 들을 dot product
 ↓
 softmax로 attention weight 생성
 ↓
 과거 v 들을 attention weight로 섞음
 ↓
 $o = W_o \text{ attention_output}$
 ↓
 $x = x + o$ // residual
 ↓
 $\text{RMSNorm}(x)$
 ↓
 $\text{gate} = W_{\text{gate}} x$
 $\text{up} = W_{\text{up}} x$
 ↓
 SwiGLU 또는 SiLU 계열 비선형
 ↓
 $\text{mlp} = W_{\text{down}} \text{ hidden}$
 ↓
 $x = x + \text{mlp}$ // residual

이 과정을 30개 layer에 대해 반복한 뒤, 마지막 hidden vector를 lm_head 행렬에 넣는다.

$\text{logits} = \text{lm_head} \times \text{final_hidden}$

logits는 vocab 전체 토큰에 대한 점수다. 예를 들어 vocab size가 49152라면 출력 점수도 49152개다.

token 0 점수
 token 1 점수
 ...

token 49151 점수

그중 가장 높은 토큰을 고르면 greedy decoding이고, temperature/top-k/top-p를 쓰면 약간 랜덤하게 고르는 sampling이 된다.

하드웨어 관점에서 중요한 사실은 다음과 같다.

q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj, lm_head는 전부 행렬 x 벡터 계산이다.

그래서 이 프로젝트의 FPGA IP는 Transformer 전체를 통째로 구현하는 것이 아니라, Transformer 내부에서 반복해서 나타나는 GEMV 계산기를 만드는 것이다.

4. Zybo Z7-20에서 역할 분담

Zybo Z7은 Zynq-7000 계열 보드이며, Zynq는 ARM Cortex-A9 기반 PS와 7-series PL을 한 칩에 합친 구조다. AMD Zynq-7000 TRM은 Zynq-7000이 dual/single ARM Cortex-A9 MPCore CPU와 PL을 포함하는 SoC라고 설명한다. [R6]

Zybo Z7 reference manual에는 1 GB DDR3L, 32-bit bus, 1066 MHz DDR가 명시되어 있다. [R5]

4.1 PS, CPU가 맡을 것

- Linux 실행
- SD 카드 파일 읽기
- 모델 weight를 DDR에 배치
- tokenizer
- chat template
- 전체 layer scheduling
- RMSNorm
- RoPE
- SiLU/SwiGLU
- softmax, top-k, top-p, sampling
- KV cache 주소 관리
- Linux console chat, SSH chat, UART fallback
- HDMI console / USB keyboard / framebuffer UI 확장

4.2 PL, FPGA가 맡을 것

- GEMV/Linear 계산
- q_proj, k_proj, v_proj
- o_proj
- MLP gate/up/down
- LM head
- quantized weight stream decode
- int8/int4 weight와 int16 activation의 MAC

4.3 처음부터 PL에 넣지 말 것

- SD card controller
- filesystem
- tokenizer
- softmax
- RoPE
- RMSNorm
- USB keyboard
- HDMI text UI
- AXI master

이것들은 시간을 잡아먹는다. 프로젝트 완성을 우선하면 CPU/Linux에 맡겨라.

5. Linux를 쓸까, bare-metal을 쓸까

5.1 결론

최종 목표가 SmolLM2 채팅이면 Linux 추천.
초기 IP 검증은 bare-metal도 좋다.

즉 실제 추천은 다음과 같다.

Vivado IP smoke test: bare-metal 가능

모델 파일/SD/채팅/USB/HDMI 포함 최종 시스템: Linux 권장

5.2 Linux의 장점

- SD 카드 파일을 그냥 파일로 읽을 수 있음
- ext4, FAT32, mmap, fread 사용 가능
- pthread로 dual-core 활용 가능
- USB keyboard, HDMI/framebuffer, network, ssh 확장 쉬움
- 모델 파일 교체가 쉬움
- 디버깅 로그 남기기 쉬움

AMD PetaLinux 문서는 SD 카드 boot용으로 FAT32 boot partition과 ext4 root filesystem partition을 쓰는 준비 절차를 설명한다. [R7]

5.3 Linux의 단점

- boot image, device tree, kernel 설정이 필요함
- DMA를 user-space에서 쓰려면 dma-proxy, u-dma-buf, custom driver 중 하나가 필요함
- cache coherency를 신경써야 함
- 메모리 일부를 OS가 먹음

하지만 1 GB DDR이면 headless minimal Linux + 135M Q8 또는 360M Q4는 현실적이다. GUI desktop을 올리지 말고 headless console 위주로 가면 된다.

5.4 bare-metal의 장점과 단점

bare-metal은 AXI-Lite register, AXI DMA standalone example을 돌리기에는 간단하다. AMD/Xilinx AXI DMA standalone driver는 simple DMA와 scatter/gather DMA를 지원한다. [R9]

하지만 SmolLM2 전체 프로젝트에서는 bare-metal이 점점 귀찮아진다.

- SD 카드 파일 시스템 처리 필요
- tokenizer와 모델 로더를 직접 관리해야 함
- USB keyboard/HDMI UI 구현이 어려움
- dual-core 활용이 더 번거로움

따라서 문서의 본 루트는 Linux 최종, bare-metal은 초기 검증 옵션으로 잡는다.

6. dual-core를 어떻게 쓸까

Zynq-7020의 PS는 dual-core Cortex-A9다. 이득은 있다. 다만 처음부터 복잡한 멀티스레드로 가지 마라.

6.1 1차 구현

core 0 하나만 사용

- 추론 루프
- PL IP 호출
- Linux console chat

6.2 2차 구현

Linux에서 pthread를 사용한다.

thread 0:

메인 추론 루프, PL GEMV 호출

thread 1:

sampling/top-k, 다음 weight chunk 준비, 로그, UI 입력 처리

단, 실제 속도 이득은 제한적이다. 대부분의 시간은 weight stream + GEMV가 먹는다. dual-core는 UI 반응성, 데이터 준비, 디버깅에 더 유용하다.

7. 전체 시스템 아키텍처

최종 목표 구조는 다음과 같다.

microSD

- boot files
- Linux rootfs
- quantized SmoLLM2 weight files

↓

PS Linux

- model loader
- tokenizer
- runtime scheduler
- CPU ops: RMSNorm/RoPE/SiLU/softmax/sampling
- DMA proxy or u-dma-buf

↓ AXI-Lite control

PL GEMV IP

↑ AXI-Stream weight/input tiles from AXI DMA

↓ output buffer / AXI-Stream / BRAM

DDR

- weights
- activations
- KV cache
- DMA buffers

처음에는 완성형 구조를 바로 만들지 않고, 아래처럼 단계적으로 확장한다.

Phase A: AXI-Lite register만 있는 dummy IP

Phase B: 작은 BRAM GEMV IP

Phase C: 135M용 C reference runtime

Phase D: AXI DMA + stream GEMV IP

Phase E: SmoLLM2 135M Q8 전체 채팅

Phase F: 360M Q4 확장

8. IP 설계 방향

8.1 만들 IP 이름

smoLLM2_gemv_accel_v1

8.2 IP가 해야 할 일

이 IP는 Transformer 전체가 아니라, 하나의 Linear/GEMV tile을 계산한다.

입력:

input vector $x[N]$

quantized weight tile $W[LANES][N]$

scale 정보

출력:

$y[LANES] = W_tile * x$

8.3 권장 lane 수

처음에는 1 lane. 이후 4 lane, 8 lane으로 확장한다.

1 lane:

구현 쉬움, 디버깅 쉬움

4 lane:

현실적인 첫 병렬화

8 lane:

Zybo Z7-20에서 성능 목표용

16 lane:

가능성은 있지만 DDR stream, timing, routing 부담이 커짐

8.4 데이터 타입 1차 권장: Q8_0 weight라도 RTL 출력은 fixed-point scaled output으로 간다

Q8_0 GGUF에서 8비트라는 말은 주로 **weight 저장 형식**을 뜻한다. 전체 추론 과정이 처음부터 끝까지 int8로만 돈다는 뜻은 아니다. 특히 Q8_0은 단순 int8 배열이 아니라, 보통 32개 weight block마다 scale을 가진다.

Q8_0 block:

scale 1개

int8 weight 32개

실제 weight 의미:

$\text{real_weight}[i] \sim \text{int8_weight}[i] \times \text{scale}$

v6에서는 FPGA가 raw int32 accumulator를 내고 CPU가 scale을 적용하는 방식을 안전한 1차안으로 잡았다.

v7에서는 출력 대역폭과 CPU 후처리를 줄이기 위해 기본 경로를 다음처럼 수정한다.

PC 변환 단계:

GGUF Q8_0 scale(fp16/f32 의미)을 fixed-point scale_q로 변환

예: $\text{scale_q} = \text{round}(\text{scale} \times 2^{\text{SCALE_SHIFT}})$

FPGA GEMV:

$\text{block_acc_i32} = \text{sum}(\text{input_i16} \times \text{weight_i8})$ for one Q8_0 block

$\text{scaled_block} = (\text{block_acc_i32} \times \text{scale_q}) \gg \text{SCALE_SHIFT}$

$\text{row_acc} += \text{scaled_block}$

$\text{output_i32} = \text{saturate_or_round}(\text{row_acc})$

CPU runtime:

output_i32를 float32 hidden vector로 복원/해석

RMSNorm/RoPE/Attention/SiLU/Softmax/sampling 처리

v7의 타입 정책은 다음으로 고정한다.

값	권장 타입	위치	이유
weight 저장	Q8_0 block, int8 + scale	GGUF/SD/DDR	모델 크기와 DDR 대역폭
scale 변환	scale_q, signed int24 또는 int32	PC layout 변환 결과	FPGA에서 float IP 없이
SCALE_SHIFT	14~20, 기본 20	manifest/register/RTL parameter	fixed-point scale 소수
GEMV 입력 activation	signed int16	CPU -> FPGA	int8보다 오차가 작고 R
Q8 block MAC	signed int16 x signed int8	FPGA	핵심 가속 대상
block accumulator	signed int32	FPGA 내부	32개 항 누산용
scale 곱 temp	signed int48/int64	FPGA 내부	$\text{block_acc_i32} \times \text{scale_q}$
row accumulator	signed int48 이상	FPGA 내부	block별 scaled 결과 누
GEMV 기본 출력	signed int32 scaled output	FPGA -> CPU	row당 1개 출력으로 ou
debug 출력	block_acc_i32 stream	FPGA -> CPU/testbench	scale/packing 버그 추
CPU 부가연산	float32	Linux runtime	RMSNorm/RoPE/soft
KV cache v1	float32	Linux runtime	정확도와 디버깅 우선

초기 데이터 흐름은 다음으로 고정한다.

Q8_0 GGUF weight
= int8 weight + block scale

PC layout converter
int8 weight를 lane layout으로 재배치
block scale을 scale_q fixed-point로 변환
weight stream + scale stream + manifest 생성

CPU float32 hidden vector
-> GEMV 호출 직전 int16 activation으로 변환

FPGA GEMV mode=0, 기본
int16 input x int8 weight -> block_acc_i32
block_acc_i32 x scale_q >> SCALE_SHIFT
row_acc 누산
scaled output_i32 출력

FPGA GEMV mode=1, debug
scale 적용 전 block_acc_i32를 block 단위로 출력

CPU 후처리
scaled output_i32를 float32 hidden vector로 해석/복원
RMSNorm/RoPE/Attention/SiLU/Softmax 처리

즉 이 프로젝트의 v1 목표는 완전 int8 추론기가 아니다. 목표는 Q8_0 weight를 쓰고, FPGA에서 가장 무거운 GEMV와 Q8_0 scale 적용까지 처리하되, Transformer의 복잡한 부가 연산은 CPU float32로 유지하는 혼합 정밀도 추론기다.

중요한 제한도 있다.

하지 말 것:
Verilog에서 real/float를 C처럼 합성하려고 하기
Floating-Point Operator IP를 v1 기본 경로로 쓰기

할 것:
scale은 PC에서 fixed-point scale_q로 변환
FPGA에서는 정수 곱셈과 shift로 scale 적용
debug mode로 block_acc_i32를 반드시 확인 가능하게 유지

8.5 Q4는 언제 할까

Q4는 메모리와 대역폭을 줄이지만 디코더가 필요하다.

Q8:
weight byte 하나 읽으면 바로 MAC 가능

Q4:
byte 하나에서 4-bit weight 두 개를 꺼냄
sign extension 필요
group scale 필요
packing/unpacking 버그 가능성 증가

따라서 135M Q8로 완성한 뒤, 360M Q4로 넘어간다.

9. GEMV IP 상태머신

9.0 GEMV란 무엇인가

GEMV는 General Matrix-Vector multiplication의 약자다. 말 그대로 행렬과 벡터를 곱하는 연산이다.

행렬 W x 벡터 x = 출력 벡터 y

예를 들어 W 가 3행 4열이고 x 가 길이 4짜리 벡터라면 출력 y 는 길이 3짜리 벡터가 된다.

$$y[0] = W[0][0]x[0] + W[0][1]x[1] + W[0][2]x[2] + W[0][3]x[3]$$

$$y[1] = W[1][0]x[0] + W[1][1]x[1] + W[1][2]x[2] + W[1][3]x[3]$$

$$y[2] = W[2][0]x[0] + W[2][1]x[1] + W[2][2]x[2] + W[2][3]x[3]$$

CNN에서 입력 window x weight를 곱해서 accumulator에 더한다는 감각과 같다. 차이는 CNN에서는 3x3x채널 window를 쓰고, Transformer에서는 hidden vector를 쓴다는 점이다.

CNN conv:

3x3x5 window -> 45개 값 -> MAC 반복 -> 출력 채널

Transformer GEMV:

hidden vector 576개 값 -> MAC 반복 -> q/k/v/mlp/logit 출력 벡터

v7에서 GEMV IP는 단순 raw accumulator 출력기가 아니라 **Q8_0 scale-applied GEMV**다. 즉 Q8_0 block마다 scale_q를 받아 정수 fixed-point로 scale을 적용하고, row당 최종 output_i32를 낸다.

기본 mode=0:

input_i16 x weight_i8 -> block_acc_i32
 block_acc_i32 x scale_q >> SCALE_SHIFT
 row_acc 누산
 row_output_i32 출력

디버그 mode=1:

scale 적용 전 block_acc_i32를 block 단위로 출력

Transformer를 전부 FPGA에 넣는 것이 아니라, 가장 많이 반복되는 행렬 곱셈과 Q8_0 scale 적용을 PL에 맡기고, RMSNorm/RoPE/softmax/SiLU/sampling 같은 제어와 비선형 연산은 CPU/Linux runtime에 둔다.

9.0.1 Q8_0 scale-applied GEMV FSM

GEMV IP의 내부 FSM은 다음 구조로 잡는다.

IDLE

start 기다림

LOAD_CONFIG

in_features, out_features, lanes, mode, scale_shift 읽음

LOAD_INPUT

input vector를 내부 BRAM에 로드하거나 이미 로드됐다고 표시

ROW_GROUP_START

현재 row group의 active lane 계산
 row_acc[LANES] = 0

BLOCK_HEADER

현재 Q8_0 block에 대응하는 scale_q[LANES] 수신
 mode=1 debug이면 scale_q를 받아도 bypass 가능

BLOCK_MAC

32개 column에 대해 input_i16 x weight_i8 MAC
 block_acc_i32[LANES] 생성

SCALE_APPLY, mode=0

scaled = (block_acc_i32 x scale_q) >>> SCALE_SHIFT
 row_acc += scaled

DEBUG_WRITE, mode=1

block_acc_i32를 output debug stream/BRAM에 기록

NEXT_BLOCK

다음 32-column block으로 이동

WRITE_OUT, mode=0

row_acc를 saturation/rounding해 output_i32로 기록

padded lane은 출력하지 않음

DONE

done = 1

이 FSM에서 가장 중요한 것은 **scale_q와 weight block의 alignment**다. scale이 한 block 밀리면 수치가 그럴듯하게 나오면서 전부 틀린다. 따라서 RTL/testbench/ILA는 다음 신호를 반드시 볼 수 있어야 한다.

```
row_group_idx
block_idx
lane_idx
scale_q[lane]
weight_i8[lane]
block_acc_i32[lane]
scaled_block[lane]
row_acc[lane]
mode
error_code
```

9.0.2 왜 FPGA float 대신 fixed-point인가

Verilog의 real 같은 타입은 일반적인 RTL 합성 경로에서 C의 float처럼 쓰는 대상이 아니다. Vivado에는 floating-point operator IP를 붙이는 방법이 있지만, v1에서는 다음 이유로 기본 경로에서 제외한다.

- int32 -> float 변환, fp scale 곱, float accumulate, 다시 int/fixed 변환이 필요하다.
- latency와 handshake가 늘어나 AXI-Stream 디버깅이 어려워진다.
- Q8_0 scale 적용 하나 때문에 resource/timing 리스크가 커진다.
- Python/C/RTL golden 비교에서 float 오차 기준까지 같이 관리해야 한다.

따라서 v7 기본안은 다음이다.

PC:

```
scale_q = round(scale x 2^SCALE_SHIFT)
```

FPGA:

```
scaled = (block_acc_i32 x scale_q) >>> SCALE_SHIFT
```

9.1 AXI-Lite register map 예시

Offset	이름	설명
0x00	CONTROL	bit0 start, bit1 clear_done, bit2 soft_reset, bit3 irq_enable
0x04	STATUS	bit0 busy, bit1 done, bit2 error, bit3 debug_mode_active
0x08	INPUT_SIZE	N, in_features
0x0C	OUTPUT_SIZE	M, out_features
0x10	TILE_ROWS	build-time LANES 확인용
0x14	MODE	0=scaled row output, 1=block_acc debug output
0x18	SCALE_SHIFT	fixed-point scale shift. 기본 20
0x1C	QUANT_MODE	1=Q8_0 fixed-scale, 2=Q4 reserved
0x20	INPUT_ADDR_LOW	나중에 DMA/AXI master용 자리
0x24	WEIGHT_ADDR_LOW	weight/packet stream source 주소
0x28	SCALE_ADDR_LOW	scale_q 별도 buffer를 쓸 때의 주소. packet 내 포함이면 optional
0x2C	OUTPUT_ADDR_LOW	output buffer 주소
0x30	STREAM_WORD_COUNT	DMA/stream word 수
0x34	OUTPUT_WORD_COUNT	mode별 예상 output word 수
0x38	DEBUG_ROW	현재 row group index
0x3C	DEBUG_BLOCK	현재 Q8_0 block index
0x40	ERROR_CODE	bad TLAST, stream length mismatch, overflow 등
0x44	ABI_VERSION	C/RTL/register map 호환성 확인

1차 IP에서는 ADDR register는 쓰지 않아도 된다. 나중에 AXI master 또는 DMA descriptor 방식으로 갈 때를 위해 자리만 잡는다. 단, MODE, SCALE_SHIFT, ABI_VERSION은 v7에서 반드시 넣는다.

10. Vivado에서 해야 할 일

10.1 프로젝트 생성

1. Vivado 실행
2. New Project
3. RTL Project 선택
4. Board 선택에서 **Zybo Z7-20** 선택
5. board file이 안 보이면 Digilent board file 설치
6. Create Block Design 생성

10.2 Zynq PS 추가

1. Block Design에서 ZYNQ7 Processing System 추가
2. Run Block Automation 클릭
3. DDR, fixed IO 자동 연결 확인
4. FCLK_CLK0를 100 MHz로 설정
5. M_AXI_GP0 활성화 - PS가 PL의 AXI-Lite register 접근
6. 나중에 DMA를 쓸 경우 S_AXI_HP0 활성화 - PL/DMA가 DDR 접근

10.3 기본 clock/reset

1. Processor System Reset IP 추가
2. FCLK_CLK0를 PL clock으로 사용
3. 모든 custom IP, AXI DMA, AXI Interconnect의 clock을 동일하게 맞춤
4. reset polarity를 확인. Verilog에서 reset_p를 쓰면 active high에 맞춰 연결하거나 wrapper에서 변환

10.4 1차 smoke test용 AXI-Lite IP

1. Tools - Create and Package New IP
2. Create AXI4 Peripheral
3. Interface는 AXI4-Lite slave 하나만 생성
4. register 4~8개 생성
5. slv_reg0에 쓴 값을 slv_reg1로 읽는 정도부터 테스트
6. Block Design에 IP 추가
7. M_AXI_GP0 - AXI Interconnect - custom IP 연결
8. Address Editor에서 base address 할당
9. Validate Design
10. Generate Bitstream
11. Export Hardware, include bitstream

10.5 GEMV BRAM 테스트 IP

1차 smoke test가 되면 custom IP 안에 gemv_core_simple.v를 넣는다.

처음에는 외부 DMA 없이 내부 memory를 쓴다.

```
input_mem[0:N-1]
weight_mem[0:M*N-1]
output_mem[0:M-1]
```

PS가 AXI-Lite로 작은 테스트 데이터를 써주고, IP가 계산한 뒤 PS가 output을 읽는다. 이 단계의 목적은 속도가 아니라 FSM, start/done, 수치 검증이다.

10.6 AXI DMA 추가 단계

대량 weight를 매번 AXI-Lite로 밀어 넣으면 너무 느리다. 완성용은 AXI DMA를 쓴다.

Vivado block design:

```
Zynq PS M_AXI_GP0
-> AXI Interconnect
    -> AXI DMA S_AXI_LITE
    -> GEMV IP S_AXI_LITE
```

AXI DMA M_AXI_MM2S
-> Zynq PS S_AXI_HP0

AXI DMA M_AXIS_MM2S
-> GEMV IP S_AXIS_WEIGHT

출력은 처음엔 GEMV 내부 output buffer를 AXI-Lite/BRAM으로 읽어도 된다. 나중에 출력도 크면 AXI-Stream S2MM을 추가한다.

AXI DMA의 Linux user-space 사용은 Xilinx DMA Proxy 방식이 대표적이다. Xilinx wiki의 DMA Proxy 문서는 Xilinx DMA Engine driver를 이용하고, user-space application이 character device를 통해 DMA buffer를 mmap하고 ioctl로 전송을 시작하는 구조를 설명한다. [R8]

11. PetaLinux/Linux 구성

11.1 Linux를 쓰는 이유

SmolLM2 프로젝트에서는 Linux가 장기적으로 유리하다.

- SD 카드에 모델 파일을 넣고 쉽게 읽음
- serial terminal 채팅 구현 쉬움
- 나중에 USB keyboard/HDMI UI 확장 쉬움
- pthread로 dual-core 활용 쉬움
- C/C++ runtime 개발 및 디버깅 쉬움

11.2 메모리 부족 걱정

1 GB DDR에서 desktop GUI까지 올리면 뻑뻑해질 수 있다. 하지만 **headless minimal Linux**면 괜찮다.

권장:

- X11/desktop 환경 사용하지 않음
- SSH/UART console 중심
- rootfs는 ext4 SD 카드
- model file은 SD에서 읽어 DDR에 mmap/load
- 135M Q8 또는 360M Q4 우선

비추천:

- full desktop Linux
- Python inference를 보드에서 직접 실행
- 360M Q8 + 큰 context + GUI

11.3 SD 카드 구성

PetaLinux 문서 기준 일반적인 SD boot 구성은 다음과 같다. [R7]

partition 1:

FAT32
BOOT.BIN
image.ub 또는 kernel/dtb/u-boot 관련 파일

partition 2:

ext4
root filesystem
model files
runtime binary

64 GB SD면 충분하다. Basys3의 SPI flash XIP보다 훨씬 쉽다. SD 카드는 PS의 SDIO controller와 Linux driver가 처리하므로, PL에서 SD protocol을 직접 구현하지 않는다.

11.4 HDMI + USB keyboard가 이미 된다면 Linux console chat을 1차 UI로 간다

팀원이 구워온 SD Linux에서 framebuffer와 HID가 이미 살아 있고, HDMI 화면과 USB keyboard 입력이 된다면 UART serial chat을 1차 목표로 둘 필요가 낮다. 이 경우 가장 쉬운 UI는 별도 GUI가 아니라 **Linux console**에서

실행되는 stdin/stdout 콘솔 앱이다.

HDMI monitor + USB keyboard
-> Linux console 또는 tty
-> ./smollm2_chat
-> SmoLLM2 runtime
-> CPU + FPGA GEMV backend

중요한 점은 framebuffer API나 /dev/input/eventX를 직접 만지는 것이 아니다. 이미 Linux console이 잡혀 있다면 앱은 그냥 표준입력/표준출력을 쓰면 된다. 그러면 같은 binary를 HDMI console, SSH, UART console에서 모두 실행할 수 있다.

11.5 UART serial은 버리지 말고 비상 디버그용으로 남긴다

UART serial은 1차 채팅 UI가 아니라 부팅 로그와 복구용 보험으로 남긴다. HDMI가 안 나오거나 Linux가 중간에 멈추면 UART가 가장 빠른 단서가 된다.

기본 사용:
HDMI + USB keyboard + Linux console chat

비상 디버그:
UART serial console
boot log, dmesg, network 실패 확인

따라서 문서의 UI 우선순위는 v6부터 다음으로 잡는다.

1차: Linux console chat, stdin/stdout 기반
2차: SSH 접속 후 같은 console app 실행
3차: UART serial은 boot/debug fallback
4차: 시간 남을 때 framebuffer/HID 직접 UI

12. PC/PyCharm에서 해야 할 GGUF 다운로드와 FPGA layout 변환 작업

12.0 GGUF란 무엇인가

GGUF는 LLM 모델을 저장하는 파일 포맷이다. 확장자 관점에서 보면 다음처럼 생각하면 된다.

.jpg = 이미지 파일 포맷
.mp3 = 음원 파일 포맷
.gguf = LLM 모델 파일 포맷

GGUF 파일 안에는 보통 다음 정보가 들어있다.

모델 weight tensor
각 tensor의 이름
각 tensor의 shape
양자화 형식
tokenizer 관련 정보
모델 metadata

예를 들어 다음 파일명은 이렇게 읽으면 된다.

SmolLM2-135M-Instruct-Q8_0.gguf

SmolLM2-135M-Instruct 모델
+ Q8_0 방식으로 8비트 양자화된 weight
+ GGUF 컨테이너에 저장된 파일

GGUF와 safetensors의 차이는 다음과 같이 잡으면 된다.

항목	safetensors	GGUF
주 용도	Hugging Face/PyTorch 계열 weight 저장	llama.cpp/GGML 계열 추론용 모델 저장
흔한 dtype	FP16, BF16, FP32	Q8_0, Q4_K_M 등 양자화 포맷
장점	PyTorch에서 다루기 쉬움	양자화 모델을 파일 하나로 배포하기 좋음
이 프로젝트에서의 위치	직접 양자화 fallback	기본 경로

중요한 점은 GGUF 파일을 FPGA가 그대로 읽게 만들지 않는다는 것이다. GGUF 내부에는 header, metadata, tensor table, padding, quantized block 등이 섞여 있다. FPGA가 이것 직접 파싱하게 만들면 프로젝트 난이도가 크게 올라간다.

따라서 v5 기본 흐름은 다음과 같다.

PC/PyCharm:

- GGUF 파일을 연다.
- 필요한 tensor를 찾는다.
- Q8_0 block을 해석한다.
- FPGA 16-lane GEMV가 읽기 좋은 순서로 재배치한다.
- 단순 .bin + manifest.json으로 저장한다.

Zybo/Linux:

- 변환된 .bin 파일을 SD카드/DDR에서 읽는다.
- DMA로 FPGA에 weight stream을 보낸다.
- GEMV IP는 단순 stream만 처리한다.

즉 GGUF는 모델이 들어있는 상자이고, GEMV는 그 상자에서 꺼낸 weight로 수행하는 핵심 계산이다.

보드에서 모델 변환을 하지 않는다. PC/PyCharm에서 한다. v4의 기본 루트는 이미 양자화된 Q8_0 GGUF를 다운로드하고, 이를 FPGA용 layout으로 변환하는 것이다.

12.1 PC에서 할 일, v4 GGUF-first

1. Hugging Face에서 SmolLM2-135M-Instruct Q8_0 GGUF 다운로드
2. 같은 원본 모델의 tokenizer 파일 다운로드 또는 AutoTokenizer로 로드
3. GGUF metadata와 tensor 목록 검사
4. tensor name mapping 작성
5. Q8_0 block decoder reference 작성
6. Q8_0 tensor를 FPGA lane16 stream layout으로 변환
7. manifest.json 생성
8. golden vector 생성
9. C reference와 Verilog testbench에서 같은 golden 파일 사용

12.2 생략되는 직접 양자화 파트

v4 본편에서는 다음을 기본 루트에서 제외한다.

- BF16 safetensors 원본 weight를 직접 INT8로 양자화
- per-row/per-group scale 정책 실험
- 양자화 전후 모델 품질 비교
- 자체 INT8 파일 포맷 설계

이 작업들은 나중에 직접 quant 방식을 개선하고 싶을 때 Appendix/번외로 돌린다.

12.3 Q8_0 GGUF가 그대로 FPGA용이 아닌 이유

GGUF는 모델 weight와 metadata를 담는 추론 런타임용 파일이다. Q8_0은 이미 8비트 weight를 담고 있지만, 일반적으로 다음 이유 때문에 그대로 PL로 stream할 수 없다.

1. GGUF tensor header/metadata가 섞여 있다.
2. tensor별 offset과 quant type을 파싱해야 한다.
3. Q8_0은 block 단위 scale + int8 payload 구조다.
4. tensor 이름이 HF safetensors와 다르다.
5. GEMV IP가 원하는 lane layout과 저장 순서가 다르다.
6. v7 RTL은 scale을 fixed-point scale_q로 요구하므로 scale 변환이 필요하다.

따라서 PC에서 다음 변환을 거친다.

GGUF Q8_0 tensor

- > Q8_0 block inspect/decode
- > 내부 tensor name mapping
- > int8 weight를 lane layout으로 재배열
- > block scale을 scale_q fixed-point로 변환
- > weight packet 또는 weight.bin + scale_q.bin + manifest.json 생성

12.4 Q8_0 block reference와 v7 scale 정책

Q8_0은 가장 단순한 GGUF 계열 quant 중 하나다. 일반적으로 block 하나가 32개 weight를 담당하고, block scale과 32개의 int8 quant 값을 가진다. 구현할 때는 llama.cpp/ggml의 실제 block 정의를 기준으로 엔디안과 scale 해석을 맞춘다.

실제 계산 의미는 다음과 같다.

```
for each output row:
    row = 0
    for each 32-column Q8_0 block:
        block_acc = sum(input_i16[col] x weight_i8[col])
        row += block_acc x scale
```

여기서 scale은 block마다 다르므로, row 전체에 단일 scale을 곱하는 방식은 틀리다. v7에서는 다음 정책으로 고정한다.

```
PC:
    scale_q = round(scale x 2^SCALE_SHIFT)
    기본 SCALE_SHIFT = 20
```

```
PL mode=0, 기본:
    block_acc_i32 계산
    scaled_block = (block_acc_i32 x scale_q) >>> SCALE_SHIFT
    row_acc에 누산
    output_i32 출력
```

```
PL mode=1, debug:
    scale 적용 전 block_acc_i32를 block 단위로 출력
```

이 방식은 FPGA 내부 floating-point IP를 쓰지 않으면서도 CPU로 row x block 크기의 대량 block_acc를 되돌려 보내지 않아도 된다.

12.5 360M 확장 판단

135M Q8_0:
기본값. 구현 단순, 파일 작음, 검증 쉬움.

360M Q4_K_M:
품질 확장 후보. 대역폭은 줄지만 Q4_K decode가 복잡함.

360M Q8_0:
구현은 단순하지만 weight traffic이 커짐. 처음부터 본편 목표로 두지 않음.

8~9일 일정에서는 135M Q8_0 완주가 먼저다. 360M은 같은 runtime과 GEMV IP가 돌아간 뒤 모델 파일과 tensor shape를 바꾸는 확장으로 진행한다.

12.6 최종 산출물

PC 단계가 끝나면 다음이 있어야 한다.

```
quantized_model/original_gguf/SmolLM2-135M-Instruct-Q8_0.gguf
docs/gguf_inspect/summary.txt
docs/gguf_inspect/tensors.csv
fpga_layout/tensor_map.json
fpga_layout/q8_0_lane16/manifest.json
fpga_layout/q8_0_lane16/*weight*.bin
fpga_layout/q8_0_lane16/*scale_q*.bin 또는 packet 내부 scale_q
golden/fake_gemv/
golden/layer0_q_proj/
runtime_c/gemv_q8_0_ref.c
runtime_c/gemv_q8_0_ref.h
docs/fixed_scale_error_report.md
```

12.7 Q8_0인데 왜 int16, int32, fixed-point, float32가 함께 나오나

Q8_0 GGUF는 weight 저장을 8비트로 줄인 모델 파일이다. 하지만 추론 전체가 int8 하나로만 굴러가는 것은 아니다. 특히 이 프로젝트는 CPU와 FPGA를 나눠 쓰기 때문에 데이터 타입이 역할별로 나뉜다.

GGUF 파일:

weight = int8 + block scale

PC layout 변환:

block scale -> scale_q fixed-point

FPGA GEMV 입력:

activation = int16

FPGA GEMV 내부:

int8 x int16 -> block_acc_i32

block_acc_i32 x scale_q -> int48/int64 temp

shift 후 row_acc에 누산

FPGA GEMV 출력:

mode=0: scaled output_i32

mode=1: block_acc_i32 debug output

CPU runtime:

scaled output_i32를 float32 hidden vector로 연결

RMSNorm/RoPE/Attention/SiLU/Softmax 처리

KV cache v1:

float32로 시작해서 정확성과 디버깅 우선

이 선택은 성능보다 8~9일 안에 안정적으로 완성하기 위한 것이다. 완전히 정수화된 inference는 다음 단계 최적화 주제다. v1에서는 CPU float32를 허용해 모델 동작을 먼저 살리고, FPGA는 제일 무거운 GEMV와 Q8_0 scale 적용을 정수 fixed-point로 처리한다.

13. 개발 마일스톤

Milestone 0 - PC reference

목표:

PC에서 SmolLM2-135M-Instruct를 transformers로 실행

동일 prompt에서 정상 출력 확인

완료 조건:

- tokenizer 사용 가능
- chat template 사용 가능
- 원본 BF16/FP32 inference가 PC에서 돌아감

Milestone 1 - Quantized Python reference

목표:

직접 양자화한 int8 weight로 Python reference inference 구현

완료 조건:

- 원본과 출력 품질이 크게 무너지지 않음
- 최소 한 prompt에서 토큰 생성 가능
- 모든 tensor shape와 scale 파일 저장 완료

Milestone 2 - C reference runtime

목표:

Python 없이 C/C++로 한 토큰 생성 루프 구현

완료 조건:

- PC에서 C runtime이 Python reference와 비슷한 logits를 냄
- RMSNorm, RoPE, SiLU, softmax, sampling 구현 완료
- GEMV는 아직 CPU C 함수여도 됨

Milestone 3 - Vivado AXI-Lite smoke test

목표:

PS에서 custom IP register write/read 성공

완료 조건:

- Vitis bare-metal 또는 Linux /dev/mem에서 register 접근 성공
- start/done dummy FSM 동작 확인

Milestone 4 - 작은 GEMV IP

목표:

4x8, 16x16 같은 작은 행렬-벡터 곱을 PL에서 계산

완료 조건:

- testbench 통과
- 보드에서 PS가 input/weight 쓰고 output 읽음
- C reference와 bit-exact 일치

Milestone 5 - AXI DMA weight stream

목표:

DDR에 있는 weight tile을 AXI DMA로 GEMV IP에 stream

완료 조건:

- DMA loopback 또는 dummy stream test 성공
- GEMV IP가 stream weight를 받아 MAC 수행
- output이 C reference와 일치

Milestone 6 - SmolLM2 135M의 Linear 하나 가속

목표:

q_proj 또는 down_proj 하나를 PL GEMV로 교체

완료 조건:

- CPU-only C runtime 결과와 PL-accelerated 결과 비교
- 오차가 quant scale 허용 범위 안에 있음

Milestone 7 - 모든 Linear를 PL로 교체

목표:

q/k/v/o, gate/up/down, lm_head를 모두 PL GEMV로 호출

완료 조건:

- 135M Q8로 prompt 하나에 대해 여러 토큰 생성 가능
- Linux console 또는 SSH에서 대화 가능. UART는 fallback

Milestone 8 - UI 확장

목표:

Linux console chat을 HDMI/USB keyboard 기반으로 안정화하고, 필요하면 framebuffer UI로 확장

완료 조건:

- Linux console 또는 framebuffer에서 입력/출력 가능

Milestone 9 - 360M Q4 확장

목표:

동일 runtime과 GEMV IP로 360M Q4 모델 실행

완료 조건:

- 360M Q4 모델 로딩
- 토큰 생성 가능
- 속도와 메모리 사용량 측정

13.1 바닥부터 천장까지 누락 보강 체크리스트

이 절은 기존 문서의 큰 흐름 사이에 빠져 있던 연결부를 보강한다. 목표는 **모델 다운로드부터 최종 시연까지 끊기지 않는 수직 슬라이스**를 만드는 것이다. 8~9일 프로젝트에서는 아래 항목을 모두 완벽히 고성능으로 만드는 것이 아니라, 각 항목마다 최소 동작 버전을 확보하는 것이 중요하다.

13.1.1 전체 파이프라인 산출물

최종적으로 프로젝트 폴더에는 다음 산출물이 있어야 한다.

models/original/<model_id>/

Hugging Face에서 받은 원본 config/tokenizer/safetensors

models/quantized/<model_id>_q8/

PC에서 변환한 int8 weight, scale, manifest.json

tests/golden/

Python/C/FPGA 비교용 golden vector

reference/python/

CPU-only float/quantized reference

reference/c/

C/C++ quantized reference

hw/ip_repo/smollm2_gemv_accel/

GEMV RTL, AXI-Lite wrapper, AXI-Stream weight 입력부, testbench

hw/vivado_project/

Zynq PS + AXI DMA + AXI Stream FIFO + custom GEMV IP block design

linux/runtime/

Linux user-space runtime, UART chat, GEMV IP 제어 코드

linux/device_tree/

UIO, DMA, reserved-memory 관련 device tree overlay/source

reports/

latency, tokens/sec, DMA throughput, PL utilization, fallback 결과

이 목록 중 하나가 비면 그 지점에서 천장까지 이어지는 사다리가 끊긴다.

13.1.2 S급 필수 누락 보강 항목

아래 항목은 선택이 아니라 필수다.

항목	왜 필요한가	최소 산출물
CPU-only SmolLM2 기준선 tensor name mapping quant format 정의	FPGA 결과가 맞는지 비교할 기준 safetensors 이름과 내부 runtime 이름을 연결 C/RTL/PC 변환기가 같은 숫자 해석	PC에서 한 토큰 생성되는 Python refer tensor_map.csv 또는 tensor_map.json quant_format.md, manifest.json

항목	왜 필요한가	최소 산출물
golden vector	단계별 regression 검증	tests/golden/*.bin, *.json
FPGA weight layout 변환	multi-lane GEMV가 DDR stream을 바로 먹게 함	lane별 재배열 weight bin
PS-PL ABI	C driver와 RTL register map 불일치 방지	gemv_regs.h, gemv_regmap.md
DMA/cache coherency 처리	Linux에서 옛날 데이터 읽는 문제 방지	coherent buffer 또는 flush/invalidate
device tree	Linux user-space에서 PL IP/DMA 접근	system-user.dtsi 예시
Vivado IP 설정값	block design을 재현 가능하게 함	Vivado 체크리스트 또는 TCL
AXI-Stream 프로토콜	DMA와 GEMV의 TLAST/TKEEP 해석 통일	stream_protocol.md
layer scheduler	SmolLM2의 실제 실행 순서 고정	run_layer() 설계
KV cache layout	대화 context 유지	kv_cache.h/kv_cache.c
Linux console chat protocol	시연 가능한 입출력	stdin/stdout, SSH, UART fallback 규
fallback plan	8~9일 내 실패 시 프로젝트 생존	Day별 전환 조건
demo checklist	발표/검증용 완성 기준	demo_checklist.md

13.1.3 CPU-only 기준선

PL을 붙이기 전에 PC에서 다음이 먼저 되어야 한다.

원본 SmolLM2 safetensors + tokenizer

↓

Python에서 tokenizer encode/decode 확인

↓

Hugging Face transformers로 logits 기준값 생성

↓

직접 작성한 Python runtime으로 한 토큰 forward

↓

layer별 주요 tensor shape와 일부 값 비교

처음부터 모든 값이 bit-exact일 필요는 없지만, 최소한 다음을 확인한다.

- tokenization 결과가 transformers와 동일
- config.json의 layer/head/hidden/intermediate/vocab 값이 runtime에 반영됨
- embedding output shape가 맞음
- 각 layer의 q/k/v/o/gate/up/down projection shape가 맞음
- 마지막 lm_head logits shape가 vocab_size와 맞음

13.1.4 tensor 이름 매핑표

SmolLM2 계열의 safetensors 이름은 runtime 내부 이름과 1:1로 매핑해야 한다. 실제 이름은 다운로드한 모델에서 자동 덤프해서 확인한다.

HF tensor 이름 예시	내부 이름	용도
model.embed_tokens.weight	tok_embeddings	token id -> hidden vector
model.layers.N.self_attn.q_proj.weight	layers[N].q_proj	Q 생성
model.layers.N.self_attn.k_proj.weight	layers[N].k_proj	K 생성
model.layers.N.self_attn.v_proj.weight	layers[N].v_proj	V 생성
model.layers.N.self_attn.o_proj.weight	layers[N].o_proj	attention output projection
model.layers.N.mlp.gate_proj.weight	layers[N].gate_proj	SwiGLU gate
model.layers.N.mlp.up_proj.weight	layers[N].up_proj	MLP up
model.layers.N.mlp.down_proj.weight	layers[N].down_proj	MLP down
model.layers.N.input_layernorm.weight	layers[N].attn_norm	attention 전 RMSNorm
model.layers.N.post_attention_layernorm.weight	layers[N].ffn_norm	MLP 전 RMSNorm
model.norm.weight	final_norm	최종 RMSNorm
lm_head.weight	lm_head	hidden -> vocab logits

Codex에는 이 표를 하드코딩시키지 말고, safetensors에서 실제 tensor 이름을 덤프한 뒤 매핑 파일을 생성하게 시킨다.

13.1.5 추천 모델/quant 포맷, v4 기준

v4 기본 포맷은 pre-quantized GGUF Q8_0이다.

기본:

SmolLM2-135M-Instruct Q8_0 GGUF

PC 변환:

GGUF Q8_0 tensor -> FPGA lane16 layout

PL v1 계산:

int16 activation x int8 weight -> int32 accumulator

CPU 처리:

Q8_0 scale 적용

RMSNorm/RoPE/SiLU/softmax/sampling

직접 양자화 방식은 다음 조건에서만 본다.

- Q8_0 GGUF 파서가 일정 안에 안 맞을 때
- 직접 만든 단순 per-row INT8 format이 더 빨리 붙을 때
- 발표 데모를 살리기 위한 fallback이 필요할 때

즉 기본 루트는 GGUF-first이고, 직접 양자화는 fallback/appendix다.

13.1.6 golden vector 규칙: FPGA 검증용 정답지

Golden vector는 말 그대로 **정답 데이터 세트**다. FPGA가 맞게 계산했는지 보려면 정답지가 필요하다. v7에서는 raw accumulator뿐 아니라 fixed-point scale 적용 결과도 같이 검증해야 한다.

GEMV 하나를 검증한다면 최소한 다음이 필요하다.

입력 vector

weight stream 또는 packet

scale_q stream 또는 scale_q file

정답 block_acc_i32, debug mode용

정답 scaled_output_i32, 기본 mode용

정답 float output, CPU reference 비교용

manifest.json

예시는 다음과 같다.

golden/

```
|— fake_gemv_000/
|   |— meta.json
|   |— input_i16.bin
|   |— weight_i8_fpga_layout.bin
|   |— scale_q_i32.bin
|   |— output_block_acc_ref_i32.bin
|   |— output_scaled_ref_i32.bin
|   |— output_f32_ref.bin
|   |— manifest.json
|— layer00_q_proj/
|— layer00_mlp_up/
|— lm_head_small_slice/
```

이 파일들은 다음 순서의 기준점이다.

Python reference

-> golden vector 생성

C reference

-> 같은 golden vector와 비교

RTL simulation mode=1

-> block_acc_i32 debug output 비교

RTL simulation mode=0

-> fixed-point scaled output_i32 비교

보드 Linux + FPGA IP

-> 같은 golden vector와 비교

성공 기준은 다음과 같다.

fake GEMV:

Python vs C vs RTL vs board block_acc_i32 bit-exact

Python vs C vs RTL vs board scaled_output_i32 bit-exact

실제 q_proj/gate_proj:

block_acc debug mode가 Python/C와 일치

fixed-point scaled output이 Python/C와 일치

float Q8_0 reference와 fixed-point 결과의 오차가 허용 범위 내

전체 token:

CPU-only quantized runtime과 FPGA-accelerated runtime의 top-k 후보가 유사

Golden vector가 없으면 FPGA가 틀렸을 때 원인을 분해할 수 없다. 특히 다음 문제를 구분할 수 없게 된다.

GGUF Q8_0 decode 오류

scale_q 변환 오류

weight/scale alignment 오류

weight layout 변환 오류

endian 오류

C runtime 오류

RTL MAC 누락

AXI stream packing 오류

DMA/cache coherency 오류

따라서 golden vector 생성기는 부가 기능이 아니라 필수 디버깅 장치다.

13.1.7 FPGA용 weight/scale layout

multi-lane GEMV는 원본 row-major weight를 그대로 쓰면 stream 공급이 불편하다. PC 변환 단계에서 lane 친화 layout으로 바꾼다. v7에서는 weight뿐 아니라 Q8_0 block scale도 lane과 block에 맞춰 변환한다.

원본 row-major:

row0: col0 col1 col2 ...

row1: col0 col1 col2 ...

row2: col0 col1 col2 ...

LANES=16용 FPGA stream layout:

row group 0..15:

block0 header: scale_q row0,row1,...row15

block0 weights:

col0: row0,row1,...row15

col1: row0,row1,...row15

...

col31: row0,row1,...row15

block1 header: scale_q row0,row1,...row15

block1 weights:

col32..col63

row group 16..31:

...

manifest에는 반드시 다음 정보를 넣는다.

```
{
  "tensor": "layers.0.self_attn.q_proj.weight",
  "in_features": 576,
  "out_features": 576,
  "dtype": "q8_0_fixed_scale",
  "layout": "q8_0_block32_col_major_lane16_scale_header",
  "lanes": 16,
```

```

"q8_block_size": 32,
"scale_shift": 20,
"scale_dtype": "int32",
"row_acc_dtype": "int48_or_int64_internal",
"output_dtype": "int32_scaled",
"weight_file": "layer_00_q_proj.q8.lane16.bin",
"scale_file": "layer_00_q_proj.scale_q20.i32.bin",
"debug_output": "block_acc_i32"
}

```

scale을 별도 파일로 돌지, block header로 packet 안에 넣을지는 Prompt 10에서 고정한다. 어느 쪽이든 **scale_q와 weight block의 순서가 1:1로 맞아야 한다.**

13.1.8 PS-PL ABI와 register map

ABI는 Application Binary Interface의 약자다. 이 프로젝트에서는 어려운 운영체제 용어라기보다 **CPU 소프트웨어와 FPGA IP 사이의 약속**으로 보면 된다.

ABI에 포함되는 것은 다음이다.

- GEMV IP register map
- 각 register offset과 bit 의미
- DMA buffer layout
- weight/scale stream packing 방식
- status/error bit의 의미
- start/done/clear 동작 규칙
- mode=0 scaled output, mode=1 block_acc debug output의 차이

예를 들어 C 코드가 0x18을 SCALE_SHIFT로 믿는데 RTL이 그 주소를 다른 의미로 해석하면 fixed-point 결과는 전부 틀어진다. 그래서 다음 네 곳은 반드시 같은 값을 써야 한다.

문서 register table

C header

Verilog localparam

testbench/checker

v7 register map은 다음으로 고정한다.

Offset	이름	설명
0x00	CONTROL	bit0 start, bit1 clear_done, bit2 soft_reset, bit3 irq_enable
0x04	STATUS	bit0 busy, bit1 done, bit2 error, bit3 debug_mode_active
0x08	IN_FEATURES	입력 길이
0x0C	OUT_FEATURES	출력 길이
0x10	LANES	RTL build-time lane 수 확인용
0x14	MODE	0=scaled row output, 1=block_acc debug output
0x18	SCALE_SHIFT	fixed-point scale shift. 기본 20
0x1C	QUANT_MODE	1=Q8_0 fixed-scale, 2=Q4 reserved
0x20	INPUT_ADDR_LO	DDR input buffer 주소 하위 32-bit
0x24	WEIGHT_ADDR_LO	DDR/stream weight 또는 packet 주소
0x28	SCALE_ADDR_LO	scale_q 별도 buffer 주소. packet에 포함하면 optional
0x2C	OUTPUT_ADDR_LO	DDR output buffer 주소
0x30	STREAM_WORD_COUNT	DMA로 보낼 32-bit word 수
0x34	OUTPUT_WORD_COUNT	mode별 예상 output word 수
0x38	DEBUG_ROW	현재 row group
0x3C	DEBUG_BLOCK	현재 Q8_0 block
0x40	ERROR_CODE	stream underflow, bad TLAST, overflow 등
0x44	ABI_VERSION	C/RTL/register map 호환성 확인

Zynq-7000은 32-bit 주소 공간에서 시작해도 충분하므로 v1은 _HI register를 생략해도 된다. 단, 코드 구조는 나중에 64-bit 주소 확장이 가능하게 작성한다.

ABI 변경은 반드시 별도 commit으로 하고, **ABI_VERSION** register 또는 header 상수도 함께 올린다. Codex가 C와 RTL 중 하나만 수정하고 다른 쪽을 잇는 일이 많기 때문이다.

13.1.9 AXI-Stream weight/scale protocol

v7 stream protocol은 복잡한 descriptor queue를 만들지 않고, Q8_0 block packet을 순서대로 보낸다. 기본은 32-bit TDATA다.

TDATA width:
32-bit

packet 단위:
row_group 하나의 Q8_0 block 하나

block header:
active lane 수만큼 scale_q 전송
scale_q는 int32 little-endian word
lane padding이 있으면 scale_q=0

weight payload:
32 columns x LANES weights
한 beat에는 signed int8 weight 4개를 little-endian packing

TLAST:
해당 matrix/tile stream의 마지막 beat에서만 1

TUSER:
v1에서는 사용하지 않음

backpressure:
GEMV가 받을 수 없으면 TREADY=0

16-lane, Q8 block size 32라면 한 row_group/block packet은 개념적으로 다음 순서다.

scale_q lane0
scale_q lane1
...
scale_q lane15

col0 weights lane0..lane15
col1 weights lane0..lane15
...
col31 weights lane0..lane15

32-bit beat 기준 weight payload는 col마다 4 beats가 된다.

col0 beat0: lane0..3
col0 beat1: lane4..7
col0 beat2: lane8..11
col0 beat3: lane12..15

디버깅을 쉽게 하기 위해 v1에서는 “matrix 하나 전송 -> 계산 -> done”으로 한다. 여러 matrix를 연속으로 streaming하는 descriptor queue는 v2 이후로 미룬다.

13.1.10 Linux device tree와 DMA/cache

Linux에서 PL IP를 user-space에서 제어하려면 최소한 다음 중 하나를 선택한다.

단순 제어:
generic-uio + /dev/uioX 또는 /dev/mem

DMA buffer:
dma-proxy
u-dma-buf
reserved-memory + mmap

cache coherency는 반드시 명시한다.

CPU가 input/weight buffer 작성 후 PL/DMA가 읽기 전:
flush 또는 non-cacheable/coherent buffer 사용

PL/DMA가 output buffer 작성 후 CPU가 읽기 전:
invalidate 또는 coherent buffer 사용

문제가 생기면 v1 fallback은 다음이다.

DMA/cache가 꼬이면:
small GEMV는 AXI BRAM Controller 기반으로 먼저 시연
DMA는 별도 throughput demo로 분리

13.1.11 SmolLM2 layer scheduler

runtime은 다음 함수 경계로 나눈다.

```
void run_model(TokenBuffer *tokens, GenerateConfig *cfg);
void run_prefill(Runtime *rt, const int *tokens, int n);
int run_decode_one(Runtime *rt, int last_token);
void run_layer(Runtime *rt, int layer_id, Tensor *x, int pos);
void run_attention(Runtime *rt, int layer_id, Tensor *x, int pos);
void run_mlp(Runtime *rt, int layer_id, Tensor *x);
void call_linear(Runtime *rt, LinearId id, Tensor *x, Tensor *y);
void call_gemv_accel_or_cpu(Runtime *rt, LinearId id, Tensor *x, Tensor *y);
```

처음에는 call_gemv_accel_or_cpu()가 CPU reference를 호출하게 하고, GEMV IP가 검증된 tensor부터 PL 호출로 교체한다.

13.1.12 KV cache 구조: Linux runtime이 관리한다

KV cache는 attention에서 쓰는 과거 토큰들의 K/V 벡터 저장소다. 새 토큰이 들어올 때마다 각 layer에서 K와 V를 만들고, 이것을 저장해둔다. 다음 토큰에서는 과거 토큰의 K/V를 다시 계산하지 않고 cache에서 꺼내 쓴다.

초기 프로젝트에서는 KV cache를 FPGA가 아니라 **Linux C runtime이 DDR/RAM에서 관리**한다. 즉 malloc, calloc, mmap 등으로 잡은 CPU 메모리 안에 둔다. FPGA가 직접 KV cache 주소와 position을 관리하게 만들면 난이도가 급상승한다.

초기 v1:
CPU/Linux runtime이 KV cache 관리
FPGA는 GEMV 계산기 역할만 수행

나중 확장:
attention score 또는 V 가중합 일부를 PL로 이동 가능

135M 기준 대략적인 KV cache 크기는 다음과 같이 계산한다.

layers = 30
kv_heads = 3
head_dim = 64
K와 V = 2개

values/token = 30 x 3 x 64 x 2 = 11520

int16/fp16이면 값 하나가 2 byte이므로 다음 정도다.

context 128 -> 약 2.8 MB
context 256 -> 약 5.6 MB
context 512 -> 약 11.25 MB
context 1024 -> 약 22.5 MB
context 8192 -> 약 180 MB

float32면 위의 약 2배다. 1 GB DDR에서 135M Q8 모델과 함께 들어가긴 하지만, Linux, DMA buffer, activation buffer, 앱 메모리까지 생각하면 처음부터 8192를 잡는 것은 MVP에 불리하다.

권장 정책은 다음이다.

구조:
max_position 8192까지 확장 가능한 코드 구조

실제 MVP 할당:

max_context 128 또는 256

성공 후 확장:

512 -> 1024 -> 필요 시 8192 실험

dtype:

v1은 float32로 시작

안정화 후 int16/fp16 cache로 최적화

즉 처음부터 최대 크기 구조를 막지는 않되, 실제 할당은 작게 시작한다. 이것이 8~9일 프로젝트에서 가장 안전하다.

13.1.13 Linux console, HDMI/USB, UART UI 경로

팀원이 준비한 Linux에서 HDMI framebuffer와 USB HID가 이미 동작한다면 v6의 기본 UI는 **Linux console chat**이다. 별도 framebuffer 앱이나 HID 이벤트 파서를 만들지 않고, 표준입력/표준출력을 쓰는 콘솔 프로그램으로 시작한다.

1차 기본 UI:

HDMI monitor + USB keyboard + Linux console

./smollm2_chat 실행

2차 편의 UI:

SSH 접속 후 같은 binary 실행

비상 디버그:

UART serial console

boot log, dmesg, 네트워크/HDMI 실패 확인

후순위 확장:

framebuffer 직접 출력

/dev/input/eventX 직접 처리

따라서 기존의 “serial chat 먼저, HDMI/USB 나중” 흐름은 v6부터 “Linux console chat 먼저, UART는 fallback”으로 정리한다.

13.1.14 실패 조건별 fallback

시점	실패 상황	즉시 전환
Day 2	PC CPU-only reference가 안 맞음	transformers 호출 기반 wrapper로 데모 기준선 확보
Day 3	Linux 부팅 실패	bare-metal Hello + UART + 작은 GEMV demo로 축소
Day 5	Vivado block design 지연	AXI DMA 제외, AXI-Lite/BRAM GEMV만 시연
Day 6	DMA/cache 문제	DMA는 throughput demo로 분리, runtime은 CPU-only 유지
Day 7	PL GEMV 결과 불일치	fake GEMV bit-exact demo + CPU-only chat으로 발표
Day 8	135M이 너무 느림	max_new_tokens 축소, prompt 짧게, 일부 layer만 FPGA demo
Day 9	HDMI/USB 미완	UART/SSH demo로 고정

13.1.15 최종 시연 체크리스트

최종 발표/검증에서는 다음 로그를 남긴다.

- [] Linux boot log 또는 bare-metal boot log
- [] 모델 파일 SD/DDR 로드 성공 로그
- [] tokenizer encode/decode smoke test
- [] CPU-only one-token generation
- [] fake GEMV Python/C/RTL bit-exact 결과
- [] 실제 q_proj 또는 mlp_up GEMV PL 호출 결과
- [] Linux console chat 입력/출력. UART fallback 여부 확인
- [] tokens/sec 또는 sec/token
- [] DMA throughput, 사용한 경우
- [] Vivado utilization report: LUT/FF/DSP/BRAM
- [] fallback 사용 여부와 이유

14. Codex 전역 프로젝트 지침

아래 내용을 Codex 프로젝트 전역 지침으로 넣는다.

너는 Zybo Z7-20 / Zynq-7020에서 SmoLLM2 추론을 구현하는 프로젝트의 코딩 보조자다.

프로젝트 목표:

- SmoLLM2-135M-Instruct를 우선 대상으로 한다.
- 기본 weight는 이미 양자화된 SmoLLM2-135M-Instruct Q8_0 GGUF에서 추출한다. BF16 직접 양자화는 fallback/appendix다.
- PS Linux가 모델 런타임, tokenizer, 파일 IO, RMSNorm, RoPE, SiLU/SwiGLU, softmax, sampling을 담당한다.
- PL FPGA는 GEMV/Linear 가속만 담당한다.
- 처음부터 Transformer 전체를 Verilog로 만들지 않는다.
- 처음부터 AXI master를 만들지 않는다. 본편은 AXI-Lite control + AXI DMA + AXI-Stream IP를 사용한다.
- 실행 카드에서 사용하는 파일명과 Prompt 산출물 파일명을 반드시 일치시킨다.
- 스크립트나 binary를 문서에 적기 전에, 어느 Prompt가 그 파일을 생성하는지 함께 명시한다.

경로 원칙:

- 문서의 `PROJECT\_ROOT`는 `/home/user22/Desktop/smolLM2-zybo`다.
- Python 도구는 보통 `PROJECT\_ROOT/pycharm`에서 실행하지만, 산출물은 가능하면 `PROJECT\_ROOT/fpga\_layout`, `PROJECT\_ROOT`
- Python 스크립트 내부에서는 `Path(\_\_file\_\_).resolve()`로 `PROJECT\_ROOT`를 계산해 상대경로 혼동을 줄인다.

코딩 원칙:

- 한 번에 큰 코드를 만들지 말고, 작은 모듈 단위로 작성한다.
- 모든 수치 연산 코드는 Python reference와 C reference를 먼저 만든다.
- Verilog/SystemVerilog 모듈은 반드시 testbench를 같이 작성한다.
- 클럭 always 블록에서는 nonblocking assignment(<=)를 사용한다.
- ready/valid handshake는 valid를 소비될 때까지 유지하는 방식으로 작성한다.
- 인터페이스를 바꿔야 할 경우 먼저 이유와 변경 전/후 포트 목록을 설명한다.
- 기존 동작을 깨는 변경을 하지 않는다. 변경 후 regression test를 제안한다.
- 합성 가능한 Verilog를 작성한다. 시뮬레이션 전용 문법은 testbench에만 사용한다.
- AXI, DMA, cache coherency는 추측으로 처리하지 말고 명확히 TODO 또는 가정으로 표시한다.
- 한국어 주석을 사용하되, 신호명과 파일명은 영어 snake_case를 사용한다.

산출물 원칙:

- 코드만 던지지 말고, 사용법과 테스트 방법을 함께 쓴다.
- 각 단계는 “목표, 파일, 실행 방법, 성공 기준, 다음 단계”를 포함한다.
- 오류 가능성이 큰 부분은 주석으로 경고한다.

15. Codex 단계별 프롬프트 통합본 v9: Q8_0 GGUF-first + dependency-audited execution

이 장은 기존에 나뉘어 있던 기본 프롬프트와 추가 프롬프트를 하나로 통합한 v4 프롬프트 세트다. v4의 기본 전제는 이미 8비트 양자화된 SmoLLM2-135M-Instruct Q8_0 GGUF를 다운로드해서 사용하는 것이다. 직접 BF16 원본을 INT8로 양자화하는 파트는 본편에서 제외하고, 필요 시 fallback/appendix로만 둔다.

사용 원칙:

- Prompt 00부터 순서대로 진행한다.
- 한 프롬프트의 산출물이 다음 프롬프트의 입력이다.
- Codex가 큰 구조를 바꾸려 하면 멈추고, 변경 이유와 diff를 먼저 요구한다.
- Python reference -> C reference -> RTL/testbench 순서로 검증한다.
- 모델 전체를 한 번에 만들라고 시키지 않는다.
- 기본 모델은 SmoLLM2-135M-Instruct-Q8_0.gguf다.

Prompt 00 - 프로젝트 폴더와 PC Python 환경 만들기

너는 이 프로젝트의 코딩 에이전트다.

목표:

Zybo Z7-20에서 SmoLLM2 Q8_0 GGUF 모델을 사용해 CPU+FPGA 혼합 추론기를 만든다.

첫 목표 모델은 SmoLLM2-135M-Instruct Q8_0 GGUF이다.

작업:

1. 현재 폴더에 다음 프로젝트 구조를 생성해라.

```
project/  
  pc_tools/  
  pc_reference/  
  quantized_model/  
  fpga_layout/  
  golden/  
  runtime_c/  
  vivado_ip/  
  linux_app/  
  docs/  
  reports/
```

2. Python 가상환경 기준 requirements.txt를 작성해라.

필수 패키지:

- huggingface_hub
- numpy
- tqdm
- gguf 또는 gguf-py 계열 패키지
- transformers
- tokenizers

3. README.md에 전체 실행 순서를 짧게 적어라.

제약:

- Windows/PyCharm과 Linux 둘 다 고려해라.
- 경로는 pathlib 사용.
- 모든 스크립트는 argparse 기반 CLI를 가져야 한다.
- 실패 시 어떤 파일이 없어서 실패했는지 명확히 출력해라.

Prompt 01 - Q8_0 GGUF 모델 다운로드 스크립트

SmoLLM2 Q8_0 GGUF 모델을 Hugging Face에서 다운로드하는 Python 스크립트를 작성해줘.

파일 위치:

pc_tools/download_gguf_model.py

기본 모델:

lmstudio-community/SmolLM2-135M-Instruct-GGUF

기본 파일명:

SmolLM2-135M-Instruct-Q8_0.gguf

추가 옵션:

```
--repo-id  
--filename  
--out-dir
```

요구사항:

1. huggingface_hub.hf_hub_download을 사용한다.
2. 기본 저장 경로는 quantized_model/original_gguf/ 이다.
3. 이미 파일이 있으면 재다운로드하지 않는다.
4. 다운로드 후 다음을 출력한다.
 - repo id
 - filename
 - local path
 - file size MB
5. 360M Q4_K_M 또는 Q8_0로 바꾸기 쉬운 구조로 작성한다.

6. main() 함수를 포함한다.
7. 네트워크 오류와 파일 없음 오류를 친절히 처리한다.

주의:

이 단계에서는 양자화를 직접 하지 않는다.

이미 양자화된 GGUF 파일을 받는 것이 목표다.

Prompt 02 - GGUF 파일 검사 및 metadata 덤프

GGUF 파일을 검사하고 metadata와 tensor 목록을 덤프하는 Python 스크립트를 작성해줘.

파일 위치:

pc_tools/inspect_gguf.py

입력:

--gguf quantized_model/original_gguf/SmolLM2-135M-Instruct-Q8_0.gguf

--out reports/gguf_inspect/

출력:

1. metadata.json

2. tensors.csv

3. summary.txt

tensors.csv에는 최소 다음 컬럼을 넣어라.

- tensor_name
- shape
- dtype_or_quant_type
- offset
- nbytes

summary.txt에는 다음을 사람이 읽기 쉽게 출력해라.

- 모델 이름
- vocab size
- hidden size
- layer 수
- attention head 수
- kv head 수
- intermediate size
- context length
- quantization type 목록
- 총 tensor 수
- 총 파일 크기

제약:

- gguf Python 라이브러리를 우선 사용한다.
- 라이브러리 API 차이가 있으면 wrapper 함수로 격리한다.
- API가 실패하면 최소한 파일 존재/크기와 에러 메시지를 출력한다.

Prompt 03 - Tokenizer smoke test

SmolLM2 tokenizer를 테스트하는 Python 스크립트를 작성해줘.

파일 위치:

pc_tools/tokenizer_smoke_test.py

목표:

GGUF 파일만으로 tokenizer를 직접 처리하기 어렵다면,

같은 원본 모델 repo인 HuggingFaceTB/SmolLM2-135M-Instruct에서 tokenizer 파일을 다운로드해서 사용한다.

요구사항:

1. transformers.AutoTokenizer를 사용한다.
2. 기본 repo id는 HuggingFaceTB/SmolLM2-135M-Instruct.

3. 테스트 문장: "Hello, how are you?"

4. 출력:

- token id 목록
- decode 결과
- special token 목록
- chat template 존재 여부

5. 결과를 reports/tokenizer_smoke_test.txt에 저장한다.

주의:

모델 weight는 GGUF Q8_0을 쓰더라도 tokenizer는 원본 Hugging Face repo의 tokenizer를 사용해도 된다.

Prompt 04 - GGUF tensor name mapping 생성

GGUF 내부 tensor name을 프로젝트 내부 이름으로 매핑하는 스크립트와 검증 스크립트를 작성해줘.

생성할 파일:

- pc_tools/create_tensor_map.py
- pc_tools/validate_tensor_map.py

입력:

```
--inspect-csv reports/gguf_inspect/tensors.csv
--gguf quantized_model/original_gguf/SmolLM2-135M-Instruct-Q8_0.gguf
```

출력:

- fpga_layout/tensor_map.json
- fpga_layout/mapping_report.txt
- reports/tensor_map_validation.txt

create_tensor_map.py 요구사항:

1. GGUF에서 흔히 쓰이는 이름 후보를 반영한다.

예:

```
token_embd.weight -> tok_embeddings
output.weight -> lm_head
blk.N.attn_q.weight -> layer_N_q_proj
blk.N.attn_k.weight -> layer_N_k_proj
blk.N.attn_v.weight -> layer_N_v_proj
blk.N.attn_output.weight -> layer_N_o_proj
blk.N.ffn_gate.weight -> layer_N_gate_proj
blk.N.ffn_up.weight -> layer_N_up_proj
blk.N.ffn_down.weight -> layer_N_down_proj
blk.N.attn_norm.weight -> layer_N_input_norm
blk.N.ffn_norm.weight -> layer_N_post_attn_norm
```

2. 자동 매핑이 불확실한 항목은 unknown으로 남기고 사용자가 수정하게 한다.

3. tensor_map.json에는 원본 이름, 내부 이름, shape, 역할, 사용 여부를 넣는다.

4. mapping_report.txt를 생성한다.

5. 누락된 필수 tensor가 있으면 non-zero exit로 실패한다.

validate_tensor_map.py 요구사항:

1. fpga_layout/tensor_map.json을 읽는다.

2. GGUF metadata 또는 tensors.csv와 shape, quant type, tensor count를 비교한다.

3. layer 0과 마지막 layer의 q/k/v/o, gate/up/down이 모두 있는지 검사한다.

4. 누락/중복/shape mismatch가 있으면 non-zero exit로 실패한다.

5. 성공/실패 결과를 reports/tensor_map_validation.txt에 저장한다.

주의:

15.1.5 실행 카드에서 validate_tensor_map.py를 사용하므로, 이 파일이 없으면 Prompt 04 미완료로 본다.

Prompt 05 - GGUF Q8_0 block decoder reference

GGUF Q8_0 block decoder의 Python reference 구현을 작성해줘.

파일 위치:

pc_reference/q8_0_decode_ref.py

목표:

GGUF Q8_0 block을 int8 weight와 scale로 해석해서,
특정 tensor의 일부 row/col을 float 또는 int representation으로 복원한다.

요구사항:

1. Q8_0의 block 구조를 명확히 주석으로 설명한다.
2. block size, scale dtype, quantized int8 payload 크기를 상수로 분리한다.
3. decode_tensor_slice(tensor_name, row_start, row_count, col_start, col_count)를 제공한다.
4. 작은 slice를 decode해서 numpy array로 반환한다.
5. llama.cpp/gguf 라이브러리에서 얻은 dequant 결과와 비교할 수 있으면 비교 코드를 넣는다.
6. 비교가 불가능하면 자체 round-trip test를 넣는다.
7. 엔디안과 tensor offset 계산을 별도 함수로 분리한다.

주의:

이 코드는 FPGA 구현의 기준이 되므로, 대충 짜면 안 된다.

Prompt 06 - Q8_0 GGUF -> FPGA lane layout 변환기

GGUF Q8_0 weight tensor를 FPGA GEMV용 lane layout과 fixed-point scale layout으로 변환하는 Python 스크립트를 작성해줘.

파일 위치:

pc_tools/convert_q8_0_to_fpga_layout.py

목표:

원본 Q8_0 tensor를 FPGA 16-lane GEMV가 순차 stream으로 먹기 좋은 구조로 변환한다.
Q8_0 block scale은 FPGA에서 floating-point로 처리하지 않고, PC에서 fixed-point scale_q로 변환한다.

기본 lane 수:

--lanes 16

기본 Q8 block size:

--q8-block-size 32

기본 scale shift:

--scale-shift 20

입력:

```
--gguf quantized_model/original_gguf/SmolLM2-135M-Instruct-Q8_0.gguf
--tensor-map fpga_layout/tensor_map.json
--out fpga_layout/q8_0_lane16/
```

출력:

1. 각 weight tensor별 int8 lane-layout .bin 파일
2. 각 tensor별 scale_q 파일 또는 scale header packet 파일
3. manifest.json
4. fixed_scale_error_report.md

레이아웃:

output row를 lane 개수 단위 group으로 묶는다.
각 group에서 Q8_0 32-column block 단위로 순회한다.
각 block마다 lane별 scale_q를 weight payload와 1:1로 대응되게 저장한다.

개념 순서:

```
for row_group in rows step LANES:
    for block in cols step 32:
        store scale_q[row_group+0][block]
        store scale_q[row_group+1][block]
        ...
        store scale_q[row_group+LANES-1][block]
```

```

for col in block..block+31:
    store w[row_group+0][col]
    store w[row_group+1][col]
    ...
    store w[row_group+LANES-1][col]

```

요구사항:

1. row 수가 lane으로 나누어떨어지지 않으면 zero padding한다.
2. manifest.json에는 tensor name, original shape, padded shape, in_features, out_features, lanes, q8 block size, quantization scale, etc.를 포함한다.
3. $scale_q = round(scale * 2^{SCALE_SHIFT})$ 로 변환한다.
4. 작은 tensor 하나를 대상으로 layout 변환 후 역변환 검증을 수행한다.
5. float scale reference와 fixed-point scale_q reference의 max error/mean error를 report로 저장한다.
6. 기존 raw block_acc debug를 위해 scale 적용 전 reference도 생성 가능해야 한다.

Prompt 07 - pre-quantized model 기준 golden vector 생성기

이미 양자화된 GGUF Q8_0 모델을 기준으로 golden vector를 생성하는 Python 스크립트를 작성해줘.

생성할 파일:

- generate_golden_from_gguf.py
- pc_reference/golden_compare.py
- reports/golden_generation_report.txt

파일 위치 원칙:

- 실행 스크립트는 pycharm/generate_golden_from_gguf.py에 둔다.
- 공용 함수가 필요하면 pc_reference/ 아래에 module로 분리한다.
- 15.1.8 실행 카드에서 `python generate\_golden\_from\_gguf.py`를 사용하므로, 이 경로를 지킨다.

목표:

FPGA GEMV 검증에 사용할 입력 벡터, weight stream, scale_q stream, reference output을 생성한다.

대상:

1. 작은 fake GEMV
2. layer 0 q_proj
3. layer 0 gate_proj
4. lm_head 일부 row

출력:

```

golden/
  fake_gemv/
  layer0_q_proj/
  layer0_gate_proj/
  lm_head_slice/

```

각 폴더에 저장:

- input_i16.bin
- weight_q8_fpga_layout.bin
- scale_q_i32.bin 또는 packet 안에 포함된 scale_q
- output_block_acc_ref_i32.bin
- output_scaled_ref_i32.bin
- output_ref_float.bin
- manifest.json

요구사항:

1. input vector는 random seed 고정.
2. Q8_0 decode reference로 float reference output을 계산한다.
3. scale_q fixed-point 경로로 output_scaled_ref_i32를 계산한다.
4. mode=1 debug 검증용으로 output_block_acc_ref_i32도 저장한다.
5. FPGA layout을 그대로 사용했을 때의 stream 순서도 같이 저장한다.
6. C와 Verilog testbench가 같은 파일을 읽을 수 있게 little-endian raw binary로 저장한다.
7. fake_gemv는 손계산 가능한 작은 shape도 함께 생성한다.
8. 생성 완료 후 reports/golden_generation_report.txt에 생성된 case, 파일 크기, shape, seed, PASS/FAIL을 기록한다.

9. import 상수 오류가 나지 않도록 convert_q8_0_to_fpga_layout.py와 공유하는 DEFAULT_* 상수 이름을 맞춘다.

Prompt 08 - C reference GEMV, GGUF Q8_0 layout 기준

C reference GEMV를 GGUF Q8_0 pre-quantized weight와 fixed-point scale_q 기준으로 작성해줘.

생성할 파일:

- runtime_c/gemv_q8_0_ref.c
- runtime_c/gemv_q8_0_ref.h
- runtime_c/gemv_q8_0_test.c
- runtime_c/CMakeLists.txt
- docs/c_reference_howto.md

빌드 산출물:

- build/runtime_c/gemv_q8_0_test

요구사항:

1. input은 int16_t vector.
2. weight는 FPGA lane layout binary를 읽는다.
3. scale_q는 int32_t 또는 int24-packed가 아닌 int32_t 파일로 먼저 지원한다.
4. SCALE_SHIFT를 인자로 받는다. 기본값은 20이다.
5. mode=0: block_acc_i32에 scale_q를 적용해 scaled output_i32를 생성한다.
6. mode=1: scale 적용 전 block_acc_i32 debug output을 생성한다.
7. output_ref_float와 fixed-point scaled output_i32 사이의 오차를 리포트한다.
8. golden/fake_gemv와 golden/layer0_q_proj 데이터를 읽어서 Python reference와 비교하는 CLI를 만든다.
9. CLI 이름은 gemv_q8_0_test로 한다.
10. CLI 옵션은 최소 `--case <golden\_dir> --mode scaled|block-acc`를 지원한다.
11. 오차 허용값을 인자로 받는다.
12. endian 문제를 검사하는 sanity check를 넣는다.
13. CMakeLists.txt는 `cmake -S runtime\_c -B build/runtime\_c` 명령으로 빌드 가능해야 한다.
14. 빌드/실행 방법을 docs/c_reference_howto.md에 적는다.

주의:

이 단계에서 BF16 원본 weight는 사용하지 않는다.

이미 받은 Q8_0 GGUF에서 변환한 파일만 사용한다.

15.1.9 실행 카드가 runtime_c/CMakeLists.txt와 gemv_q8_0_test를 사용하므로, 둘 중 하나라도 없으면 Prompt 08 미완료로 본다

Prompt 09 - Verilog GEMV, Q8_0 fixed-point scale 적용 기준

Verilog GEMV FSM을 Q8_0 pre-quantized weight stream + fixed-point scale_q 기준으로 작성해줘.

생성할 파일:

- vivado_ip/rtl/gemv_q8_0_stream_core.v
- vivado_ip/tb/tb_gemv_q8_0_stream_core.v
- scripts/run_gemv_sim.tcl
- scripts/verify_gemv_sim_outputs.py
- docs/rtl_sim_howto.md

목표:

AXI-Stream 또는 단순 valid/ready stream으로 들어오는 Q8_0 block packet을 받아 16-lane GEMV를 수행한다.

기본 mode=0에서는 FPGA 내부에서 Q8_0 scale을 fixed-point로 적용해 row_output_i32를 출력한다.

디버그 mode=1에서는 scale 적용 전 block_acc_i32를 block 단위로 출력한다.

중요 설계 결정:

- Verilog에서 floating-point IP를 쓰지 않는다.
- GGUF Q8_0 scale은 PC layout 변환 단계에서 fixed-point scale_q로 변환되어 들어온다.
- SCALE_SHIFT는 parameter와 register 둘 다 지원한다. 기본값은 20이다.
- scale_q는 우선 signed int32로 받는다.
- block_acc는 signed int32다.
- temp multiply는 signed 64-bit 또는 충분한 폭으로 둔다.
- row_acc는 signed 48-bit 이상으로 둔다.
- 최종 output_i32는 rounding 또는 saturation 정책을 명시한다.

입력:

- clk
- reset_p
- start
- mode, 0=scaled output, 1=block_acc debug
- scale_shift
- in_features
- out_features
- input vector BRAM read port
- Q8_0 block packet stream tdata/tvalid/tready/tlast

출력:

- output BRAM write port 또는 output stream
- busy
- done
- error
- error_code

기본:

- lanes = 16 parameter
- q8 block size = 32 parameter
- input dtype = signed int16
- weight dtype = signed int8
- scale_q dtype = signed int32
- block_acc dtype = signed int32
- row_acc dtype = signed 48-bit 이상

packet 구조:

1. block header로 lane별 scale_q를 먼저 받는다.
2. 이후 32 columns x lanes개의 int8 weight를 받는다.
3. 32-bit tdata에는 int8 weight 4개를 little-endian으로 packing한다.

RTL 요구사항:

1. 한 Q8 block마다 lane별 block_acc_i32를 만든다.
2. mode=0에서는 $\text{scaled} = (\text{block_acc_i32} * \text{scale_q}) \gg \text{scale_shift}$ 를 계산해 row_acc에 누산한다.
3. mode=1에서는 block_acc_i32를 output에 기록한다.
4. row group 단위로 출력한다.
5. out_features가 lanes로 나누어떨어지지 않으면 padded lane은 계산하되 출력하지 않는다.
6. tlast 위치와 stream length가 예상과 다르면 error를 올린다.
7. scale_q와 weight block alignment를 검증할 수 있도록 debug_row/debug_block/debug_lane 신호를 둔다.
8. testbench에서 golden/fake_gemv를 읽어 mode=0과 mode=1을 모두 비교한다.
9. X/Z가 output에 전파되지 않게 reset과 default assignment를 명확히 한다.

scripts/run_gemv_sim.tcl 요구사항:

1. 프로젝트 root에서 실행된다고 가정한다.
2. vivado_ip/rtl/*.v와 vivado_ip/tb/*.v를 읽는다.
3. 실제 존재하는 testbench top을 자동 또는 명시적으로 사용한다.
4. xelab/xsim을 실행한다.
5. logs/gemv_sim_result.txt에 시뮬레이션 로그를 저장한다.
6. RTL/testbench/golden 파일이 없으면 성공 처리하지 말고 명확한 에러를 낸다.
7. scripts/와 logs/ 폴더가 없으면 생성한다.

문서 요구사항:

- docs/rtl_sim_howto.md에 왜 실행하는지, 어디서 실행하는지, 명령, 필요한 입력 파일, PASS 기준, 실패 시 로그 위치를 적는다.

주의:

- 15.1.10 실행 카드가 scripts/run_gemv_sim.tcl을 사용하므로, 이 파일이 없으면 Prompt 09 미완료로 본다.

Prompt 10 - AXI-Stream weight/scale protocol 명세 고정

GEMV IP의 AXI-Stream Q8_0 block packet protocol 문서를 작성하고, Python converter/C test/Verilog testbench가 같은 규칙

조건:

- TDATA width는 v1에서 32-bit로 한다.
- stream 단위는 Q8_0 block packet이다.
- packet은 scale header와 weight payload로 구성한다.
- scale header는 lane별 scale_q int32 word를 little-endian으로 보낸다.
- 한 weight beat에는 signed int8 weight 4개를 little-endian packing한다.
- 16-lane GEMV는 한 col당 weight beat 4개를 모아 lane 16개 weight로 사용한다.
- Q8 block size는 32 columns다.
- TKEEP는 v1에서 항상 4'b1111로 가정한다.
- TLAST는 matrix/tile stream의 마지막 beat에서만 1이다.
- TUSER는 v1에서 사용하지 않는다.
- stream 길이가 예상과 다르면 error flag를 올린다.

산출물:

- docs/axi_stream_q8_0_packet_protocol.md
- runtime_c/weight_stream_pack.h
- runtime_c/weight_stream_pack.c
- vivado_ip/tb/weight_stream_protocol_tb.v
- scripts/run_weight_stream_protocol_tb.tcl
- docs/weight_stream_protocol_test_howto.md

주의:

15.1.11에서 protocol test를 실행할 수 있게 run_weight_stream_protocol_tb.tcl도 생성한다.

Prompt 11 - PS-PL ABI header와 register map

Linux app과 Verilog AXI-Lite wrapper가 공유할 PS-PL ABI를 정의해줘.

산출물:

- docs/gemv_register_map.md
- runtime_c/gemv_regs.h
- vivado_ip/rtl/gemv_axi_lite_regs.v
- scripts/check_register_map_consistency.py

필수 register:

- CONTROL
- STATUS
- INPUT_ADDR
- WEIGHT_ADDR
- SCALE_ADDR, scale_q가 packet에 포함될 경우 optional임을 명시
- OUTPUT_ADDR
- IN_FEATURES
- OUT_FEATURES
- LANE_COUNT
- MODE, 0=scaled row output, 1=block_acc debug output
- SCALE_SHIFT
- QUANT_MODE, 1=Q8_0 fixed-scale
- STREAM_WORD_COUNT
- OUTPUT_WORD_COUNT
- ERROR_STATUS
- DEBUG_ROW
- DEBUG_BLOCK
- ABI_VERSION

주의:

Zynq-7000은 32-bit 주소 공간을 우선 가정한다.

64-bit 주소 확장은 optional로만 둔다.

문서, C header, Verilog localparam, testbench가 반드시 같은 offset을 써야 한다.

Prompt 12 - 1-lane GEMV core smoke test

16-lane 전에 1-lane Q8_0 fixed-scale GEMV core와 시뮬레이션 스크립트를 작성해줘.

생성할 파일:

- vivado_ip/tb/tb_gemv_q8_0_1lane.v
- scripts/run_gemv_sim_1lane.tcl
- docs/gemv_1lane_sim_howto.md

조건:

- input은 BRAM-style read port
- Q8_0 packet은 단순 valid/ready stream
- output은 BRAM-style write port 또는 testbench-visible output stream
- in_features/out_features/mode/scale_shift는 runtime register로 받음
- block_acc는 int32
- row_acc는 48-bit 이상
- testbench는 golden/fake_gemv 사용
- mode=0 scaled output과 mode=1 block_acc debug output을 모두 검증

scripts/run_gemv_sim_1lane.tcl 요구사항:

- 프로젝트 root에서 실행된다.
- 필요한 RTL/testbench/golden이 없으면 실패한다.
- logs/gemv_sim_1lane_result.txt를 남긴다.

목표:

작은 행렬에서 bit-exact로 맞춘 뒤 16-lane으로 확장한다.

15.1.13이 scripts/run_gemv_sim_1lane.tcl을 사용하므로, 이 파일이 없으면 Prompt 12 미완료로 본다.

Prompt 13 - 16-lane GEMV 확장

1-lane Q8_0 fixed-scale GEMV core를 16-lane으로 확장하고, lane별 regression 스크립트를 작성해줘.

생성할 파일:

- vivado_ip/tb/tb_gemv_q8_0_multilane.v
- scripts/run_gemv_sim_multilane.tcl
- scripts/compare_multilane_outputs.py
- docs/gemv_multilane_sim_howto.md

조건:

- lanes parameter 기본값 16
- row_group 단위 계산
- Q8_0 block packet에서 lane별 scale_q와 weight를 정확히 매칭
- padded output row 처리
- output write enable은 실제 row에 대해서만 발생
- 각 lane block_acc는 int32
- 각 lane row_acc는 48-bit 이상
- mode=0 scaled output과 mode=1 block_acc debug output을 모두 지원
- testbench는 fake_gemv와 layer0_q_proj golden을 모두 사용
- lane 수 1 -> 2 -> 4 -> 8 -> 16 순서로 regression을 돌릴 수 있게 parameter화

scripts/run_gemv_sim_multilane.tcl 요구사항:

- lane 1/2/4/8/16을 순차 실행한다.
- 각 결과를 logs/gemv_sim_lane*.txt로 저장한다.
- 하나라도 실패하면 전체 실패로 종료한다.

Prompt 14 - AXI-Lite wrapper

GEMV core를 AXI-Lite control register로 감싸는 wrapper와 단독 testbench를 작성해줘.

생성할 파일:

- vivado_ip/rtl/gemv_axi_lite_wrapper.v
- vivado_ip/tb/tb_gemv_axi_lite_wrapper.v

- scripts/run_axi_lite_regs_tb.tcl
- docs/axi_lite_wrapper_howto.md

조건:

- Vivado Create and Package IP에 넣기 쉬운 구조
- register map은 docs/gemv_register_map.md와 일치
- start bit는 write-one-to-start
- done bit는 clear-on-start
- error bit는 clear register로 지움
- debug row/block read 가능
- MODE와 SCALE_SHIFT register가 core에 연결됨

주의:

AXI-Lite wrapper가 안 맞으면 Linux register access가 전부 실패하므로, 단독 testbench를 반드시 만든다.

Prompt 15 - AXI DMA + AXI4-Stream FIFO 통합용 RTL wrapper

AXI DMA MM2S -> AXI4-Stream Data FIFO -> GEMV weight stream으로 연결되는 것을 가정한 top wrapper와 시뮬레이션 스크립트를

생성할 파일:

- vivado_ip/rtl/gemv_dma_stream_top.v
- vivado_ip/tb/tb_gemv_dma_stream_top.v
- scripts/run_dma_fifo_wrapper_tb.tcl
- docs/dma_fifo_wrapper_howto.md

조건:

- GEMV는 S_AXIS_WEIGHT를 가진다.
- output은 v1에서는 BRAM write port로 둔다.
- input vector도 v1에서는 BRAM에서 읽는다.
- DMA S2MM output은 optional로만 둔다.
- AXI master 직접 구현 금지.
- backpressure, TLAST mismatch, reset 중 stream test를 포함한다.

Prompt 16 - Vivado block design TCL 초안

Zybo Z7-20용 Vivado block design TCL 초안을 작성해줘.

포함 IP:

- ZYNQ7 Processing System
- Processor System Reset
- AXI Interconnect 또는 SmartConnect
- AXI DMA, MM2S enabled, simple mode 우선
- AXI4-Stream Data FIFO
- AXI BRAM Controller
- Block Memory Generator
- custom gemv_q8_0_stream IP

설정:

- FCLK_CLK0 = 100 MHz
- M_AXI_GPO는 AXI-Lite control용
- S_AXI_HPO는 DMA DDR 접근용
- AXI DMA scatter-gather는 처음에는 끄
- stream data width는 32-bit

산출물:

- vivado_ip/scripts/create_bd_gemv_q8_0.tcl
- scripts/create_block_design.tcl
- docs/vivado_block_design_steps.md

scripts/create_block_design.tcl 요구사항:

- 프로젝트 root에서 실행되는 wrapper TCL이다.
- vivado_ip/scripts/create_bd_gemv_q8_0.tcl을 source하거나 동일한 block design을 생성한다.

- logs/vivado_block_design.log 또는 Vivado journal 경로를 문서에 남긴다.
- board file/custom IP가 없으면 조용히 성공 처리하지 말고 에러를 낸다.

주의:

15.1.17 실행 카드가 scripts/create_block_design.tcl을 사용하므로, 이 파일이 없으면 Prompt 16 미완료로 본다.

Prompt 17 - Linux device tree: UIO, DMA, reserved-memory

PetaLinux device tree overlay 예시를 작성해줘.

목표:

Linux user-space에서 GEMV AXI-Lite register와 DMA buffer를 안정적으로 사용한다.

포함:

- gemv@43c00000 generic-uio node
- axi-dma node 확인 포인트
- reserved-memory 또는 CMA 사용 설명
- cache flush/invalidate가 필요한 이유 설명
- docs/linux_device_tree_gemv.md 생성

Prompt 18 - Linux /dev/mem 또는 UIO control app

Linux user-space에서 GEMV register를 제어하는 C 프로그램과 Makefile을 작성해줘.

생성할 파일:

- linux_app/gemv_control.c
- linux_app/Makefile
- docs/linux_register_access_howto.md

빌드 산출물:

- linux_app/gemv_control

기능:

- /dev/uio 또는 /dev/mem mmap
- register map 출력
- ABI_VERSION 읽기
- start/done polling
- debug row/block 출력
- error status 출력
- --read-version
- --dump-registers
- --base <addr>
- --uio /dev/uioX 옵션

주의:

처음에는 DMA 없이 register smoke test만 한다.

15.1.19 실행 카드가 ./linux_app/gemv_control을 사용하므로, Makefile target도 맞춘다.

Prompt 19 - DMA buffer와 cache coherency 처리

Linux에서 DMA/PL이 DDR buffer를 읽고 쓸 때 필요한 cache coherency 처리 문서를 작성하고 C helper를 만들어줘.

산출물:

- docs/linux_dma_cache_rules.md
- linux_app/dma_buffer.h
- linux_app/dma_buffer.c

설명 포함:

- CPU가 input buffer를 쓴 뒤 PL이 읽기 전 flush 필요
- PL이 output buffer를 쓴 뒤 CPU가 읽기 전 invalidate 필요
- /dev/mem cached mapping 위험

- UI0, dma-buf, dma-proxy, reserved-memory의 차이
- v1에서는 가장 단순한 안전 경로를 추천

Prompt 20 - Linux DMA weight stream app

Linux에서 Q8_0 lane layout weight 파일을 읽고 AXI DMA MM2S로 GEMV IP에 stream하는 smoke test 앱을 작성해줘.

입력:

- fpga_layout/q8_0_lane16/layer_00_q_proj.q8.bin
- golden/layer0_q_proj/input_i16.bin

동작:

1. input vector BRAM 또는 DDR buffer에 입력 복사
2. DMA로 weight stream 전송
3. GEMV start
4. done 대기
5. output 읽기
6. golden output과 비교

산출물:

- linux_app/gemv_dma_smoke.c
- linux_app/Makefile에 gemv_dma_smoke target 추가
- docs/linux_dma_weight_stream_howto.md
- logs/gemv_dma_smoke_example.txt

주의:

실행 파일 이름은 linux_app/gemv_dma_smoke로 고정한다.

Prompt 21 - CPU-only minimal runtime

SmolLM2 Q8_0 GGUF 기반 CPU-only 최소 runtime skeleton을 작성해줘.

목표:

FPGA 없이도 한 토큰 생성 경로가 돌아가야 한다.

포함:

- tokenizer 입력
- embedding lookup
- RMSNorm
- Q/K/V projection은 일단 C GEMV reference 호출
- RoPE
- KV cache
- attention score/softmax
- MLP gate/up/down
- lm_head
- top-k sampling 또는 greedy sampling

제약:

처음에는 느려도 된다.

정확한 구조와 디버깅 가능성이 우선이다.

Prompt 22 - runtime에서 GEMV 호출부 추상화

CPU-only runtime의 GEMV 호출부를 backend interface로 분리해줘.

구조:

- gemv_backend_cpu.c
- gemv_backend_fpga.c
- gemv_backend.h

요구사항:

- 같은 함수 시그니처로 CPU/FPGA backend 교체

- 환경변수 또는 CLI 옵션으로 backend 선택
- layer/tensor 이름과 latency 로그 출력
- FPGA 실패 시 CPU fallback 가능

Prompt 23 - RMSNorm/RoPE/Attention/MLP CPU 구현 검증

RMSNorm, RoPE, attention, MLP CPU 구현에 대해 작은 단위 테스트를 작성해줘.

조건:

- 각 함수별 deterministic input 사용
- Python reference와 C output 비교
- 오차 허용값 설정 가능
- reports/operator_tests.txt 생성

Prompt 24 - KV cache 구현

SmolLM2 135M/360M에 대응 가능한 KV cache 구조를 C로 작성해줘.

요구사항:

- layer, kv_head, position, head_dim 기준 indexing 명확화
- dtype은 v1에서 float 또는 int16 선택 가능하게 enum 제공
- context 길이는 CLI 옵션
- context overflow 시 sliding window 또는 reset 중 하나를 선택
- unit test 작성

Prompt 25 - Linux console chat 프로그램

Linux console에서 실행되는 stdin/stdout 기반 채팅 프로그램을 작성해줘.

상황:

- Zybo Linux는 이미 HDMI framebuffer와 USB keyboard 입력이 동작한다.
- 따라서 1차 UI는 UART serial 전용 프로그램이 아니라 일반 Linux console app이다.
- 같은 binary가 HDMI console, SSH, UART console 어디서든 실행되어야 한다.

요구사항:

- 한 줄 입력을 받아 tokenizer 적용
- SmolLM2 runtime 호출
- token을 하나씩 stdout으로 출력
- Ctrl+C 처리
- max_new_tokens, temperature, top_k 옵션
- FPGA backend timeout 시 CPU fallback
- 실행 시 backend, model path, max_context, dtype 정책을 출력
- UART serial은 boot/debug fallback 용도로만 문서화

Prompt 26 - HDMI/USB console 확인과 UI polish

HDMI monitor와 USB keyboard를 이용한 Linux console 실행 절차를 정리하고, Prompt 25의 console chat을 보드에서 실행하는 체

상황:

- v6 기본 경로는 이미 HDMI/HID가 살아있는 Linux console을 사용한다.
- 따라서 새 UI를 만드는 것이 아니라 console 환경을 안정화하고 문서화한다.

요구사항:

- /dev/fb*, /dev/dri, /dev/input 존재 확인
- 현재 tty 확인
- smolLM2_chat 실행 예시
- SSH/UART/HDMI console에서 같은 앱을 실행하는 방법
- UART serial을 boot/debug fallback으로 남기는 방법
- 직접 framebuffer/HID 앱은 후순위 optional로 분리

Prompt 27 - Vivado utilization/timing report 파서

Vivado synthesis/implementation report에서 LUT, FF, DSP, BRAM, timing slack을 추출하는 Python 스크립트를 작성해줘.

파일 위치:

pc_tools/parse_vivado_reports.py

출력:

- reports/vivado_utilization_summary.txt
- reports/vivado_utilization.csv

목표:

8-lane/16-lane/32-lane 확장 시 자원 사용률을 비교한다.

Prompt 28 - 성능 측정 로거

Linux runtime에 성능 측정 로거를 추가해줘.

측정:

- token latency
- tokens/sec
- 각 GEMV tensor별 latency
- DMA 전송 시간
- CPU operator 시간
- peak memory usage

출력:

- reports/runtime_perf.csv
- reports/runtime_perf_summary.txt

Prompt 29 - 실패 조건별 fallback 스크립트

8~9일 프로젝트 실패 조건별 fallback 계획을 README와 scripts로 정리해줘.

조건:

- Linux 부팅 실패 시 bare-metal smoke path
- DMA 실패 시 AXI-Lite/BRAM smoke demo
- FPGA GEMV 불일치 시 CPU-only chat demo
- SmolLM2가 너무 느리면 작은 fake model demo
- 360M은 항상 optional

Prompt 30 - 최종 demo checklist 생성

발표/시연용 최종 체크리스트를 작성해줘.

포함:

1. Zybo Linux boot 확인
2. SD 카드에서 GGUF/model layout 파일 확인
3. tokenizer smoke test
4. CPU-only 한 토큰 생성
5. FPGA GEMV fake_gemv 통과
6. FPGA GEMV layer0_q_proj 통과
7. runtime에서 일부 GEMV FPGA backend 사용
8. Linux console chat 동작. UART serial은 fallback
9. 성능 로그 출력
10. Vivado utilization report 출력

Prompt 31 - 전체 regression runner

PC와 Zybo에서 실행 가능한 regression runner를 작성해줘.

PC:

- GGUF inspect
- tokenizer test
- Q8_0 decode test
- layout roundtrip
- golden generation
- C reference test

Zybo:

- register smoke
- DMA smoke
- fake_gemv
- layer0_q_proj
- Linux console chat short run

출력:

reports/regression_result.txt

Prompt 32 - 직접 양자화 fallback appendix

Q8_0 GGUF 경로가 실패할 때를 대비해, BF16 safetensors -> 단순 per-row INT8 포맷으로 변환하는 fallback 도구를 appendix로

주의:

이 도구는 기본 경로가 아니다.

8~9일 프로젝트에서 GGUF 파싱이 막혔을 때 데모를 살리기 위한 fallback이다.

Prompt 33 - 360M Q4_K_M 확장 설계

135M Q8_0 경로가 완성된 뒤 360M Q4_K_M으로 확장하기 위한 설계 메모를 작성해줘.

포함:

- 360M tensor shape 차이
- Q4_K_M decode 복잡도
- PL에서 바로 decode할지 CPU에서 unpack할지 비교
- DDR bandwidth 추정
- 일정상 하지 말아야 할 것

Prompt 34 - 문서 업데이트 자동화

프로젝트 진행 중 생성된 reports/*.txt, reports/*.csv를 바탕으로 docs/status.md를 자동 생성하는 스크립트를 작성해줘.

포함:

- 현재 성공한 단계
- 실패한 테스트
- 다음 작업
- Vivado utilization
- runtime performance

Prompt 35 - 최종 통합 검토

현재 코드베이스 전체를 검토하고, SmoLLM2 Q8_0 GGUF -> FPGA layout -> C runtime -> Vivado IP -> Linux app -> Linux console

출력:

- missing_links.md
- risk_list.md
- final_patch_plan.md

주의:

새 기능을 바로 구현하지 말고, 먼저 누락/위험/수정 계획을 보고하라.

15.1 각 Codex 프롬프트 사이에서 사용자가 해야 할 행동 - 육하원칙 실행 카드

이 장은 Codex 프롬프트 자체가 아니라, Codex가 만든 결과를 사람이 어떻게 실행하고 검수할지를 설명한다. 명령어만 복붙하는 장이 아니다. 각 단계에서 사용자는 "왜 이걸 하는지", "어디서 실행하는지", "무엇을 보고 성공이라고 판단하는지"를 확인해야 한다.

15.1.0 먼저 고정할 실행 위치 이름

아래 문서에서 쓰는 위치 이름은 다음 뜻이다. 실제 경로가 다르면 본인 프로젝트 위치에 맞게 바꾼다.

PROJECT_ROOT:

/home/user22/Desktop/smollm2-zybo

프로젝트 최상위 폴더다. pycharm, quantized_model, docs 등이 있는 위치다.

PYCHARM_ROOT:

/home/user22/Desktop/smollm2-zybo/pycharm

Python 도구, golden 생성기, C reference 빌드 명령을 주로 실행하는 위치다.

BOARD_LINUX:

Zybo Z7-20에서 부팅된 Linux 콘솔이다.

HDMI + USB keyboard, UART serial, 또는 SSH 접속 중 하나일 수 있다.

VIVADO_PC:

Vivado가 설치된 PC 또는 Linux 환경이다.

RTL simulation, block design, bitstream 생성을 여기서 한다.

모든 단계에서 공통으로 지킬 원칙은 다음이다.

1. Codex가 코드를 만들면 바로 다음 Prompt로 넘어가지 않는다.
2. 먼저 git diff를 보고, 의도하지 않은 파일 삭제/대규모 재작성 여부를 확인한다.
3. 문서의 "어디서 실행" 위치로 이동한 뒤 명령을 실행한다.
4. PASS 로그, 생성 파일, 파일 크기, diff, report 중 최소 하나의 증거를 남긴다.
5. 실패하면 에러 전문을 Codex에 다시 넣고 같은 단계에서 고친다.
6. "Codex가 됐다고 말함"은 성공 기준이 아니다. 사람이 실행한 결과가 기준이다.

15.1.0.1 프롬프트 사이 검증에 Codex를 쓰는 공통 방법

각 Prompt가 끝날 때마다 사용자가 혼자 모든 파일을 읽고 판단하려 하면 너무 어렵다. 따라서 v9부터는 Codex를 작성자뿐 아니라 검증 보조자로도 쓴다. 단, Codex가 "정상"이라고 말하는 것만으로 통과 처리하지 않는다. 실제 명령 실행 결과와 파일 존재가 최종 기준이다.

각 단계가 끝나면 다음 순서로 검증한다.

1. 사용자가 실제 명령을 실행한다.
2. 파일 목록, git diff, PASS/FAIL 로그를 저장한다.
3. 그 로그를 Codex에 넣고 "이 단계 산출물이 다음 단계 실행 카드에 필요한 파일을 모두 만들었는지 검사"하게 한다.
4. Codex가 누락을 찾으면 같은 Prompt로 돌아가서 수정한다.
5. 사용자가 다시 명령을 실행해서 PASS를 확인한다.

Codex 검증용 공통 프롬프트는 다음을 복사해서 쓴다.

너는 구현자가 아니라 검증자다.

내가 방금 실행한 Prompt의 산출물이 다음 실행 카드에서 요구하는 파일과 명령을 만족하는지 검사해라.

검사할 것:

1. 생성되어야 할 파일이 실제 파일 목록에 있는가?
2. 실행 카드에서 사용하는 명령이 실제 생성된 파일명과 일치하는가?
3. README/docs/howto에 실행 위치, 명령, PASS 기준, 실패 시 로그 위치가 있는가?
4. git diff에 의도하지 않은 파일 삭제나 대규모 재작성은 없는가?
5. 다음 Prompt로 넘어가도 되는가, 아니면 현재 Prompt를 수정해야 하는가?

내가 제공할 입력:

- find 결과
- git diff --stat

- 관련 로그 파일
- 에러 전문

출력 형식:

- PASS 또는 BLOCKED
- 누락 파일 목록
- 경로 불일치 목록
- 다음에 Codex에게 줄 수정 프롬프트

주의:

실제로 실행하지 않은 테스트를 PASS라고 쓰지 마라.

파일이 없으면 추측하지 말고 누락이라고 말해라.

아래 각 단계의 "Codex 검증" 항목은 이 공통 프롬프트를 그 단계 파일명에 맞게 적용하라는 뜻이다.

15.1.1 Prompt 00 전후 - 프로젝트 폴더와 Python 환경

왜 하는가: 이후 모든 스크립트가 같은 폴더 구조와 같은 Python 패키지를 기준으로 동작해야 한다. 환경이 틀리면 뒤에서 GGUF를 못 읽거나 golden 생성기가 실패한다.

누가 하는가: 사용자.

언제 하는가: Prompt 00을 Codex에 넣기 전 프로젝트를 처음 만들 때, 그리고 Codex가 requirements/폴더 구조를 만든 직후.

어디서 하는가: PC의 터미널. 처음에는 PROJECT_ROOT를 만들 위치에서 실행한다.

무엇을 만드는가: smollm2-zybo 프로젝트 폴더, git 저장소, Python venv, 기본 폴더 구조.

어떻게 실행하는가:

```
mkdir -p /home/user22/Desktop/smollm2-zybo
cd /home/user22/Desktop/smollm2-zybo
git init
python3 -m venv pycharm/.venv
source pycharm/.venv/bin/activate
python -m pip install --upgrade pip
```

Codex가 Prompt 00을 수행한 뒤에는 다음을 실행한다.

```
cd /home/user22/Desktop/smollm2-zybo
find . -maxdepth 2 -type d | sort
git diff --stat
```

requirements가 생겼다면 설치한다.

```
cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate
pip install -r requirements.txt
python -c "import numpy, transformers, huggingface_hub; print('IMPORT_OK')"
```

무엇을 확인하는가: IMPORT_OK가 출력되고, pycharm, docs, fpga_layout, golden 같은 기본 폴더가 예상대로 생겼는지 본다.

실패하면: 패키지 설치 로그 전체를 Codex에 넣는다. 폴더가 엉뚱한 위치에 생겼으면 다음 단계로 넘어가지 말고 경로부터 고친다.

15.1.2 Prompt 01 전후 - Q8_0 GGUF 다운로드

왜 하는가: 이 프로젝트는 BF16 원본을 직접 양자화하지 않는다. 이미 Q8_0으로 양자화된 GGUF 파일을 받아서 시작한다.

누가 하는가: 사용자 또는 Codex가 만든 다운로드 스크립트를 실행하는 사람.

언제 하는가: Prompt 01 코드가 생성된 뒤.

어디서 하는가: PYCHARM_ROOT.

입력: Hugging Face repo 이름과 파일명. 기본은 SmoLLM2-135M-Instruct-Q8_0.gguf.

어떻게 실행하는가:

```
cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate
python pc_tools/download_gguf_model.py --help
python pc_tools/download_gguf_model.py
ls -lh ../quantized_model/original_gguf/
sha256sum ../quantized_model/original_gguf/*.gguf > ../quantized_model/original_gguf/SHA256SUMS.txt
cat ../quantized_model/original_gguf/SHA256SUMS.txt
```

무엇을 확인하는가: GGUF 파일 크기가 0이 아니고, 대략 100MB 이상이며, 같은 명령을 다시 실행해도 불필요하게 매번 새로 받지 않아야 한다.

실패하면: 네트워크 오류인지, repo/file 이름 오류인지, 저장 경로 오류인지 구분한다. ls -lh 결과와 에러 전문을 Codex에 준다.

15.1.3 Prompt 02 전후 - GGUF metadata와 tensor 목록 검사

왜 하는가: 모델 안에 실제 tensor 이름과 shape가 무엇인지 알아야 뒤에서 tensor map, layout 변환, golden 생성이 가능하다.

누가 하는가: 사용자.

언제 하는가: GGUF 파일 다운로드가 끝난 직후.

어디서 하는가: PYCHARM_ROOT.

입력: ../quantized_model/original_gguf/Smollm2-135M-Instruct-Q8_0.gguf.

어떻게 실행하는가:

```
cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate
python pc_tools/inspect_gguf.py \
  --gguf ../quantized_model/original_gguf/Smollm2-135M-Instruct-Q8_0.gguf \
  --out reports/gguf_inspect

head -100 reports/gguf_inspect/summary.txt
head -20 reports/gguf_inspect/tensors.csv
grep -Ei 'attn|ffn|token|output|norm' reports/gguf_inspect/tensors.csv | head -80
```

무엇을 확인하는가: summary에 layer 30, hidden 576, vocab 49152, KV head 3 같은 값이 보이는지 확인한다. tensors.csv에는 q/k/v/o projection, gate/up/down projection, embedding, norm, output/lm_head가 보여야 한다.

실패하면: GGUF 경로가 맞는지 먼저 본다. reports/gguf_inspect가 생성되지 않았으면 import 오류인지 parser 오류인지 로그를 Codex에 넣는다.

15.1.4 Prompt 03 전후 - Tokenizer smoke test

왜 하는가: 모델 weight만 있어서는 문장을 넣을 수 없다. 텍스트를 token id로 바꾸고 다시 문자열로 복원할 tokenizer가 동작해야 한다.

누가 하는가: 사용자.

언제 하는가: GGUF inspect가 끝난 뒤.

어디서 하는가: PYCHARM_ROOT.

입력: Hugging Face tokenizer repo, 테스트 문장.

어떻게 실행하는가:

```
cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate
python pc_tools/tokenizer_smoke_test.py
cat reports/tokenizer_smoke_test.txt 2>/dev/null || true
cat docs/tokenizer_smoke_test.txt 2>/dev/null || true
```

chat template도 직접 확인한다.

```
python - <<'PYCODE'
from transformers import AutoTokenizer
repo = 'HuggingFaceTB/SmolLM2-135M-Instruct'
tok = AutoTokenizer.from_pretrained(repo)
messages = [{'role': 'user', 'content': 'Hello, how are you?'}]
print(tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True))
ids = tok('Hello', return_tensors=None)['input_ids']
print(ids)
print(tok.decode(ids))
PYCODE
```

무엇을 확인하는가: token id가 출력되고, decode 결과가 원문과 크게 어긋나지 않아야 한다. chat template에 user/assistant 형식이 들어가는지 본다.

실패하면: 인터넷/캐시 문제인지, transformers 버전 문제인지, repo 이름 문제인지 로그로 구분한다.

15.1.5 Prompt 04 전후 - GGUF tensor name mapping

왜 하는가: GGUF 내부 tensor 이름은 코드가 임의로 추측하면 안 된다. 각 layer의 q/k/v/o, gate/up/down, norm, lm_head가 어느 tensor인지 명시해야 한다.

누가 하는가: 사용자. Codex는 mapping 생성기를 만들 수 있지만, 최종 확인은 사람이 한다.

언제 하는가: GGUF inspect 결과가 나온 뒤.

어디서 하는가: PYCHARM_ROOT.

입력: reports/gguf_inspect/tensors.csv, GGUF 파일.

어떻게 실행하는가: 먼저 필요한 파일이 있는지 확인한다. validate_tensor_map.py가 없다면 Prompt 04 산출물 누락이다.

```
cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate
ls pc_tools/create_tensor_map.py pc_tools/validate_tensor_map.py
python -m json.tool ../fpga_layout/tensor_map.json > /dev/null
python pc_tools/validate_tensor_map.py \
  --gguf ../quantized_model/original_gguf/SmolLM2-135M-Instruct-Q8_0.gguf \
  --map ../fpga_layout/tensor_map.json
```

Codex 검증: find pc_tools -maxdepth 1 -type f, git diff --stat, reports/tensor_map_validation.txt를 Codex에 넣고 다음 단계로 넘어가도 되는지 확인시킨다.

무엇을 직접 보는가: tensor_map.json에서 layer 0과 layer 29를 연다. q/k/v/o, gate/up/down, final norm, lm_head 또는 output weight가 빠지지 않았는지 확인한다.

통과 기준: unknown tensor가 없고, 필수 tensor가 모두 1:1로 매핑되며, shape mismatch가 없어야 한다.

실패하면: fuzzy matching으로 억지 매핑하지 말고, 누락 tensor 이름과 tensors.csv 일부를 Codex에 넣어 mapping rule을 고친다.

15.1.6 Prompt 05 전후 - Q8_0 decoder reference

왜 하는가: GGUF Q8_0 weight는 단순 int8 배열이 아니라 block scale + int8 배열이다. 이 구조를 잘못 읽으면 이후 FPGA layout과 golden이 전부 틀린다.

누가 하는가: 사용자.

언제 하는가: tensor map이 통과한 뒤.

어디서 하는가: PYCHARM_ROOT.

입력: GGUF 파일, 실제 tensor 이름 하나. 예: blk.0.attn_q.weight.

어떻게 실행하는가:

```
cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate
python pc_reference/q8_0_decode_ref.py --self-test
python pc_reference/q8_0_decode_ref.py \
  --gguf ../quantized_model/original_gguf/SmolLM2-135M-Instruct-Q8_0.gguf \
  --tensor blk.0.attn_q.weight \
  --rows 2 \
  --cols 32
```

무엇을 확인하는가: scale endian, signed int8 해석, 32-column block 경계 처리가 맞는지 본다. 가능하면 GGUF 라이브러리 또는 llama.cpp 쪽 dequant 결과와 비교한다.

실패하면: 이 단계에서 멈춘다. decoder가 틀리면 뒤의 모든 PASS는 의미가 없다.

15.1.7 Prompt 06 전후 - FPGA lane layout 변환

왜 하는가: FPGA는 GGUF 파일을 직접 읽지 않는다. PC가 GGUF tensor를 꺼내서 FPGA가 순서대로 먹기 쉬운 weight_i8 + scale_q layout으로 바꿔야 한다.

누가 하는가: 사용자. 팀원에게 넘기기 전에는 이 단계 산출물이 계약서 역할을 한다.

언제 하는가: Q8_0 decoder reference가 통과한 뒤.

어디서 하는가: PYCHARM_ROOT.

입력: GGUF 파일, tensor map, v7 fixed-scale 정책. 기본 정책은 block size 32, scale_shift 20, scale_q fixed-point, lane 1/16이다.

어떻게 실행하는가:

```
cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate
python pc_tools/convert_q8_0_to_fpga_layout.py --help
python pc_tools/convert_q8_0_to_fpga_layout.py --lanes 1 --scale-shift 20
python pc_tools/convert_q8_0_to_fpga_layout.py --lanes 16 --scale-shift 20
find ../fpga_layout -maxdepth 3 -type f -ls
python -m json.tool ../fpga_layout/q8_0_lane16/manifest.json > /dev/null
```

무엇을 확인하는가: manifest.json에 lane 수, block size, scale shift, scale_q bit width, row padding, tensor shape가 들어가야 한다. weight와 scale_q가 같은 row/block 순서로 저장되는지도 확인한다.

통과 기준: 변환 후 round-trip 또는 reference 비교가 통과해야 한다. float scale과 fixed-point scale_q의 오차 report가 있어야 한다.

실패하면: lane order, block order, 상대경로/절대경로 문제를 먼저 본다. manifest가 없다면 Prompt 07로 넘어가지 않는다.

15.1.8 Prompt 07 전후 - Golden vector 생성

왜 하는가: golden vector는 FPGA와 C reference가 맞는지 확인하는 정답지다. 이게 없으면 RTL에서 숫자가 나와도 맞는지 틀린지 판단할 수 없다.

누가 하는가: 사용자.

언제 하는가: FPGA layout 변환이 통과한 뒤.

어디서 하는가: PYCHARM_ROOT.

입력: FPGA layout manifest, weight_i8/scale_q 파일, 입력 activation seed.

어떻게 실행하는가:

```
cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate
ls generate_golden_from_gguf.py
python generate_golden_from_gguf.py
find ../golden -type f -ls
```

```

xxd -l 64 ../golden/fake_gemv/input_i16.bin
xxd -l 64 ../golden/fake_gemv/output_scaled_ref_i32.bin
xxd -l 64 ../golden/fake_gemv/output_block_acc_ref_i32.bin
python -m json.tool ../golden/fake_gemv/manifest.json > /dev/null

```

파일명이 프로젝트에서 다르다면 `find ../golden/fake_gemv -maxdepth 1 -type f -print`로 실제 이름을 먼저 확인한다.

무엇을 확인하는가: 최소 fake_gemv, layer0_q_proj, layer0_gate_proj 같은 작은 case가 있어야 한다. mode=0용 scaled output과 mode=1용 block_acc debug output이 둘 다 있어야 한다.

통과 기준: 재실행해도 seed가 같으면 결과가 같고, 각 bin 파일 크기가 manifest의 shape와 일치해야 한다.

Codex 검증: `find ../golden -maxdepth 2 -type f, reports/golden_generation_report.txt`, 에러 로그를 Codex에 넣고, Prompt 08에서 사용할 파일이 모두 있는지 확인시킨다.

실패하면: ImportError는 파일 간 상수 이름 불일치일 가능성이 높다. 에러 전문과 관련 import 줄을 Codex에 넣고 같은 단계에서 수정한다.

15.1.9 Prompt 08 전후 - C reference GEMV

왜 하는가: Verilog를 만들기 전에, C로 작성한 기준 GEMV가 Python golden과 같은 결과를 내는지 확인한다. C reference가 맞아야 Verilog testbench의 기준도 믿을 수 있다.

누가 하는가: 사용자.

언제 하는가: Prompt 08을 Codex가 작성한 뒤, RTL 작업을 시작하기 전.

어디서 하는가: PC의 PYCHARM_ROOT. 이 명령은 Zybo 보드가 아니라 PC에서 먼저 실행한다.

입력: Prompt 07에서 생성된 ../golden/fake_gemv, ../golden/layer0_q_proj 같은 golden case와 runtime_c 소스.

무엇을 하는가: C reference를 빌드하고, C 결과가 Python reference와 같은지 비교한다. 여기서 확인하는 것은 모델 전체 추론이 아니라 **GEMV 한 개가 정확히 계산되는지**다.

어떻게 실행하는가:

```

cd /home/user22/Desktop/smollm2-zybo/pycharm
source .venv/bin/activate

```

1. 컴파일 도구가 있는지 확인한다.

```

gcc --version
cmake --version

```

2. CMake 입력 파일이 있는지 확인한다. 없으면 Prompt 08 미완료다.

```
ls runtime_c/CMakeLists.txt runtime_c/gemv_q8_0_ref.c runtime_c/gemv_q8_0_ref.h runtime_c/gemv_q8_0_test.c
```

3. C reference를 빌드한다.

```

cmake -S runtime_c -B build/runtime_c
cmake --build build/runtime_c

```

4. 가장 작은 fake_gemv부터 실행한다.

```

./build/runtime_c/gemv_q8_0_test --case ../golden/fake_gemv --mode scaled
./build/runtime_c/gemv_q8_0_test --case ../golden/fake_gemv --mode block-acc

```

5. fake가 통과하면 실제 tensor 일부도 실행한다.

```

./build/runtime_c/gemv_q8_0_test --case ../golden/layer0_q_proj --mode scaled
./build/runtime_c/gemv_q8_0_test --case ../golden/layer0_q_proj --mode block-acc

```

무엇을 확인하는가: 출력에 PASS, mismatch 0, max error 같은 문구가 있는지 본다. scaled mode는 fixed-point scale_q 적용 결과를 확인하고, block-acc mode는 scale 적용 전 raw block accumulator를 확인한다.

sanitizer는 언제 하는가: 위 기본 테스트가 통과한 뒤 메모리 오류가 없는지 추가로 확인할 때 한다.

```

cd /home/user22/Desktop/smollm2-zybo/pycharm
cmake -S runtime_c -B build/asan -DCMAKE_C_FLAGS="-fsanitize=address,undefined -g"
cmake --build build/asan
./build/asan/gemv_q8_0_test --case ../golden/fake_gemv --mode scaled

```

Codex 검증: 빌드 로그, `find runtime_c -maxdepth 1 -type f`, `find build/runtime_c -maxdepth 2 -type f`, 테스트 출력 전문을 Codex에 넣고, Prompt 09가 요구하는 golden/C reference가 준비됐는지 확인시킨다.

통과 기준: Python reference와 C `block_acc_i32`가 일치하고, Python reference와 C `scaled_output_i32`가 일치해야 한다. float reference와 fixed-point 결과의 오차 report가 출력되어야 한다. sanitizer 실행 시 메모리 오류가 없어야 한다.

실패하면: 다음 단계인 Verilog로 넘어가지 않는다. --case 경로 오류인지, 파일명 불일치인지, scaled만 틀리는지, block-acc도 틀리는지 나눠서 Codex에 로그를 넣는다. block-acc가 맞고 scaled만 틀리면 scale_q/shift 문제다. block-acc부터 틀리면 weight layout/input/endian 문제다.

15.1.10 Prompt 09 전후 - Verilog GEMV core

왜 하는가: 이제 C reference로 검증된 GEMV 계산을 FPGA RTL로 옮긴다. 이 단계는 아직 보드 실행이 아니라 시뮬레이션 검증이다.

누가 하는가: RTL 담당자 또는 Codex를 쓰는 팀원. 사용자는 결과 로그를 검수한다.

언제 하는가: Prompt 08 C reference가 통과하고 docs/interface_contract.md가 나온 뒤.

어디서 하는가: VIVADO_PC.

입력: docs/interface_contract.md, golden/fake_gemv, golden/layer0_q_proj, fpga_layout manifest.

먼저 무엇을 확인하는가: Prompt 09가 실행 카드에 필요한 파일을 만들었는지 확인한다. 아래 파일 중 하나라도 없으면 아직 시뮬레이션을 실행할 단계가 아니다.

```
cd /home/user22/Desktop/smollm2-zybo
ls vivado_ip/rtl/gemv_q8_0_stream_core.v
ls vivado_ip/tb/tb_gemv_q8_0_stream_core.v
ls scripts/run_gemv_sim.tcl
ls docs/rtl_sim_howto.md
```

없으면 무엇을 하는가: scripts/run_gemv_sim.tcl이 없으면 Prompt 09 산출물 누락이다. Vivado를 실행하지 말고 Codex에게 scripts/run_gemv_sim.tcl과 docs/rtl_sim_howto.md를 생성하게 한다.

어떻게 실행하는가: 위 파일이 모두 있을 때만 실행한다.

```
cd /home/user22/Desktop/smollm2-zybo
vivado -version
vivado -mode batch -source scripts/run_gemv_sim.tcl
cat logs/gemv_sim_result.txt 2>/dev/null || true
```

무엇을 확인하는가: Verilog는 float를 쓰면 안 된다. real, shortreal, floating-point IP 없이 fixed-point scale_q를 사용해야 한다. 로그나 파형에서 start, busy, tvalid, tready, block_acc, scaled_block, row_acc, done을 확인한다.

Codex 검증: `find vivado_ip -name '*.v'`, `find scripts -name '*gemv*sim*.tcl'`, logs/gemv_sim_result.txt, Vivado 에러 전문을 Codex에 넣고 누락 파일과 FAIL 원인을 분석시킨다.

통과 기준: mode=1 block_acc debug output이 golden과 bit-exact로 일치하고, mode=0 scaled output_i32도 golden과 bit-exact로 일치해야 한다. 중간 stall/backpressure test에서도 결과가 같아야 한다.

실패하면: scaled만 틀리면 scale_q alignment/shift를 의심한다. block_acc부터 틀리면 weight_i8 packing, signed extension, column count, endian을 먼저 본다.

15.1.11 Prompt 10 전후 - AXI-Stream weight/scale protocol

왜 하는가: DMA가 FPGA로 보내는 stream의 byte 순서를 고정해야 RTL, Python converter, C reference가 같은 데이터를 같은 의미로 해석한다.

누가 하는가: RTL 담당자와 사용자.

언제 하는가: GEMV core의 내부 계산 구조가 정해진 뒤, DMA wrapper를 만들기 전.

어디서 하는가: 문서는 PC에서 작성하고, testbench는 VIVADO_PC에서 실행한다.

입력: manifest의 packet format, lane 수, scale_q bit width, block size.

먼저 무엇을 확인하는가: Prompt 10 산출물이 있는지 확인한다.

```
cd /home/user22/Desktop/smollm2-zybo
ls docs/axi_stream_q8_0_packet_protocol.md
ls runtime_c/weight_stream_pack.h runtime_c/weight_stream_pack.c
ls vivado_ip/tb/weight_stream_protocol_tb.v
ls scripts/run_weight_stream_protocol_tb.tcl
```

어떻게 실행하는가: test TCL이 생성되어 있으면 실행한다.

```
vivado -mode batch -source scripts/run_weight_stream_protocol_tb.tcl
```

Codex 검증: protocol 문서와 runtime_c/weight_stream_pack.*, testbench, 실행 로그를 Codex에 넣고 packing 규칙이 서로 일치하는지 검사시킨다.

무엇을 확인하는가: 다음 질문에 문서만 보고 답할 수 있어야 한다.

TDATA 폭은 몇 bit인가?

한 packet은 scale_q header와 weight payload 중 무엇을 포함하는가?

scale_q는 lane별로 몇 개 들어가는가?

한 Q8 block은 몇 column인가?

TKEEP은 언제 1인가?

TLAST는 row, block, tensor 중 어디의 끝에서 올라가는가?

padding lane은 계산에서 어떻게 무시되는가?

통과 기준: Python converter, C reference, Verilog testbench가 같은 packet packing을 사용해야 한다. protocol violation test가 있어야 한다.

실패하면: packet format을 감으로 고치지 말고 manifest와 interface_contract부터 수정한다.

15.1.12 Prompt 11 전후 - PS-PL ABI와 register map

왜 하는가: Linux C 앱이 FPGA register에 값을 쓸 때, Verilog가 같은 주소를 같은 의미로 해석해야 한다. 이 약속이 ABI다.

누가 하는가: 사용자와 RTL/Linux 담당자.

언제 하는가: AXI-Lite wrapper와 Linux 앱을 만들기 전.

어디서 하는가: PC에서 문서/헤더를 확인하고, 나중에 보드 Linux에서 register read로 검증한다.

입력: C header, Verilog localparam, 문서 register table, testbench.

어떻게 확인하는가:

```
cd /home/user22/Desktop/smollm2-zybo
grep -R "ABI_VERSION\|SCALE_SHIFT\|MODE\|CONTROL\|STATUS" -n runtime_c rtl docs | head -80
```

통과 기준: register offset 중복이 없어야 한다. MODE, SCALE_SHIFT, ABI_VERSION이 있어야 한다. start/done/error clear 방식이 명확해야 한다. mode=0과 mode=1의 output word count도 문서화되어야 한다.

실패하면: C header와 Verilog localparam 중 하나를 기준으로 억지로 맞추지 말고, docs/interface_contract.md를 먼저 고친 뒤 양쪽을 다시 생성한다.

15.1.13 Prompt 12 전후 - 1-lane smoke test

왜 하는가: 16-lane은 파형이 복잡하다. 가장 단순한 1-lane에서 MAC 횟수, block 경계, scale 적용이 맞는지 먼저 확인한다.

누가 하는가: RTL 담당자.

언제 하는가: GEMV core와 testbench가 만들어진 뒤.

어디서 하는가: VIVADO_PC.

입력: 1-lane layout과 1-lane golden.

어떻게 실행하는가: 먼저 Prompt 12가 스크립트를 만들었는지 확인한다. 없으면 Prompt 12 미완료다.

```
cd /home/user22/Desktop/smollm2-zybo
ls scripts/run_gemv_sim_1lane.tcl
vivado -mode batch -source scripts/run_gemv_sim_1lane.tcl
cat logs/gemv_sim_1lane_result.txt 2>/dev/null || true
```

Codex 검증: 1-lane testbench, TCL, 로그를 Codex에 넣고 16-lane 확장으로 넘어가도 되는지 확인시킨다.

무엇을 확인하는가: 파형에서 start, busy, row_idx, block_idx, col_idx, scale_q, block_acc, scaled_block, row_acc, done을 본다.

통과 기준: mode=1 block_acc와 mode=0 scaled output이 모두 golden과 맞아야 한다. 두 번 연속 실행해도 이전 accumulator가 남으면 안 된다.

실패하면: 16-lane 확장 금지. 1-lane에서 끝까지 잡는다.

15.1.14 Prompt 13 전후 - 16-lane 확장

왜 하는가: 1-lane으로 정확도를 확인한 뒤 병렬 lane을 늘려 성능을 올린다.

누가 하는가: RTL 담당자.

언제 하는가: 1-lane smoke test가 통과한 뒤.

어디서 하는가: VIVADO_PC.

어떻게 진행하는가: 바로 16-lane으로 가지 말고 다음 순서로 늘린다.

1 lane -> 2 lane -> 4 lane -> 8 lane -> 16 lane

Prompt 13이 만든 regression TCL이 있는지 먼저 확인한다.

```
cd /home/user22/Desktop/smollm2-zybo
ls scripts/run_gemv_sim_multilane.tcl
vivado -mode batch -source scripts/run_gemv_sim_multilane.tcl
```

각 lane 수에서 같은 의미의 golden을 통과해야 한다.

Codex 검증: logs/gemv_sim_lane*.txt와 compare script 결과를 Codex에 넣고 첫 실패 lane을 찾게 한다.

통과 기준: lane 수가 달라도 최종 scaled output이 같아야 한다. partial row group, padding lane, scale_q lane 순서가 모두 맞아야 한다.

실패하면: 틀린 첫 lane 수에서 멈추고, lane packing과 scale_q 순서를 먼저 본다.

15.1.15 Prompt 14 전후 - AXI-Lite wrapper

왜 하는가: Linux에서 FPGA GEMV IP를 시작/상태확인/설정하려면 AXI-Lite register wrapper가 필요하다.

누가 하는가: RTL 담당자.

언제 하는가: core 단독 시뮬레이션이 통과한 뒤.

어디서 하는가: VIVADO_PC.

입력: register map, GEMV core.

무엇을 테스트하는가: register write/read, start write, status read, 잘못된 주소 access, reset.

어떻게 실행하는가: Prompt 14가 만든 test TCL이 있으면 실행한다.

```
cd /home/user22/Desktop/smollm2-zybo
ls scripts/run_axi_lite_regs_tb.tcl
vivado -mode batch -source scripts/run_axi_lite_regs_tb.tcl
```

Codex 검증: register map 문서, C header, Verilog wrapper, test 로그를 Codex에 넣고 offset 불일치를 검사시킨다.

통과 기준: write strobe가 반영되고, read/write handshake가 deadlock 없이 끝나며, control register가 core 신호와 연결되어야 한다.

실패하면: GEMV core를 고치기 전에 AXI-Lite wrapper 단독 testbench에서 bus handshake부터 잡는다.

15.1.16 Prompt 15 전후 - DMA/FIFO wrapper

왜 하는가: 실제 보드에서는 weight/scale stream을 CPU가 매번 레지스터로 넣지 않는다. DDR에 둔 데이터를 DMA가 AXI-Stream으로 GEMV IP에 공급해야 한다.

누가 하는가: RTL/Vivado 담당자.

언제 하는가: AXI-Lite wrapper가 통과한 뒤.

어디서 하는가: VIVADO_PC.

무엇을 테스트하는가: 정상 stream, 중간 stall, TLAST 조기 도착, TLAST 누락, reset 중 stream.

어떻게 실행하는가: Prompt 15가 만든 test TCL이 있으면 실행한다.

```
cd /home/user22/Desktop/smollm2-zybo
ls scripts/run_dma_fifo_wrapper_tb.tcl
vivado -mode batch -source scripts/run_dma_fifo_wrapper_tb.tcl
```

Codex 검증: stream test 로그를 Codex에 넣고 TVALID/TREADY/TLAST 처리 누락이 있는지 확인시킨다.

통과 기준: backpressure가 동작하고 FIFO overflow/underflow가 없어야 한다. error status가 software에서 읽힐 수 있어야 한다.

실패하면: DMA까지 붙이지 말고 wrapper 단독 testbench에서 stream 문제를 잡는다.

15.1.17 Prompt 16 전후 - Vivado block design

왜 하는가: Zynq PS, AXI-Lite, AXI DMA, FIFO, custom GEMV IP를 실제 FPGA design으로 묶어 bitstream을 만들기 위해서다.

누가 하는가: Vivado 담당자.

언제 하는가: RTL simulation이 통과한 뒤.

어디서 하는가: VIVADO_PC.

입력: custom IP, Zybo Z7-20 board file, block design Tcl.

어떻게 실행하는가: 먼저 Prompt 16이 실행 카드에서 쓰는 wrapper TCL을 만들었는지 확인한다. 없으면 Prompt 16 미완료다.

```
cd /home/user22/Desktop/smollm2-zybo
ls scripts/create_block_design.tcl
ls vivado_ip/scripts/create_bd_gemv_q8_0.tcl
vivado -mode batch -source scripts/create_block_design.tcl
```

Codex 검증: Vivado log/journal, block design TCL, Address Editor 결과를 Codex에 넣고 누락 IP/주소 충돌을 검사시킨다.

GUI에서는 Validate Design, Generate Output Products, Create HDL Wrapper, Synthesis, Implementation, Generate Bitstream 순서로 확인한다.

통과 기준: validation error 0, bitstream 생성 성공, XSA export 성공, utilization/timing report 저장.

실패하면: critical warning을 무시하지 말고, clock/reset/address/DMA 연결 문제를 먼저 본다.

15.1.18 Prompt 17 전후 - Device tree

왜 하는가: Linux가 custom GEMV register와 DMA를 인식하려면 bitstream과 맞는 device tree가 필요하다.

누가 하는가: Linux/Vivado 담당자.

언제 하는가: bitstream/XSA가 나온 뒤.

어디서 하는가: Linux build PC 또는 PetaLinux 환경.

입력: 최신 XSA, register base address, DMA node 정보.

어떻게 확인하는가: 빌드 후 DTB를 decompile해서 node를 확인한다.

```
dtc -I dtb -O dts system.dtb > system_decoded.dts
grep -n "gemv\|dma\|reserved-memory\|uio" system_decoded.dts
```

통과 기준: GEMV register address가 Vivado Address Editor와 일치해야 한다. UIO 또는 devmem 접근 계획이 문서화되어야 한다.

실패하면: 보드에서 앱을 실행하지 말고 device tree와 address부터 맞춘다.

15.1.19 Prompt 18 전후 - Linux register access

왜 하는가: DMA나 모델 실행 전에, Linux에서 FPGA register가 실제로 읽히는지 확인해야 한다.

누가 하는가: 보드 Linux 담당자.

언제 하는가: 새 bitstream과 device tree로 Zybo가 부팅된 뒤.

어디서 하는가: BOARD_LINUX.

입력: GEMV base address, register map, control app.

어떻게 실행하는가:

```
uname -a
ls -l /dev/uio* 2>/dev/null || true
ls -l /dev/mem 2>/dev/null || true
make -C linux_app gemv_control
./linux_app/gemv_control --read-version
./linux_app/gemv_control --dump-registers
```

Codex 검증: 보드의 uname, /dev/uio*, /dev/mem, gemv_control 출력 로그를 Codex에 넣고 주소/device-tree/bitstream mismatch 가능성을 분류시킨다.

통과 기준: ABI_VERSION 또는 version register가 예상값으로 읽혀야 한다. start를 누르기 전 status가 정상이어야 한다.

실패하면: 주소, device tree, bitstream mismatch를 먼저 의심한다. DMA 테스트로 넘어가지 않는다.

15.1.20 Prompt 19 전후 - DMA buffer/cache coherency

왜 하는가: Zynq PS와 PL은 DDR을 공유하지만 cache 때문에 CPU가 쓴 데이터와 DMA가 보는 데이터가 다를 수 있다. 이를 미리 잡아야 한다.

누가 하는가: 보드 Linux 담당자.

언제 하는가: register access가 통과한 뒤.

어디서 하는가: BOARD_LINUX.

입력: DMA buffer 할당 방식. 예: reserved-memory, udmabuf, dma-proxy 중 하나.

무엇을 확인하는가: known pattern을 DDR buffer에 쓰고 DMA/loopback 뒤에도 같은 값인지 확인한다. flush/invalidate가 필요한지 기록한다.

통과 기준: 여러 buffer 크기에서 pattern mismatch가 없어야 한다. stale data가 나오면 cache 처리부터 고친다.

실패하면: GEMV 문제가 아니라 DMA/cache 문제일 수 있으므로 GEMV core를 건드리지 않는다.

15.1.21 Prompt 20 전후 - Linux DMA weight stream app

왜 하는가: 이제 실제 Linux 앱이 DDR의 weight/scale packet을 DMA로 GEMV IP에 보내고 output을 받아야 한다.

누가 하는가: 보드 Linux 담당자와 사용자.

언제 하는가: DMA buffer/cache 테스트가 통과한 뒤.

어디서 하는가: BOARD_LINUX.

입력: fake_gemv weight/scale packet, input_i16, golden output, GEMV register map.

실행 순서:

1. input buffer에 input_i16을 넣는다.
2. output buffer를 0xCD 같은 패턴으로 초기화한다.
3. GEMV register에 rows, cols, mode, scale_shift를 쓴다.
4. DMA source와 length를 설정한다.
5. GEMV start를 건다.
6. DMA start를 건다.
7. done 또는 timeout을 기다린다.
8. output cache invalidate 후 읽는다.
9. golden과 비교한다.

어떻게 실행하는가: Prompt 20이 만든 Makefile과 binary가 있는지 확인한 뒤 실행한다.

```
cd /home/user22/Desktop/smollm2-zybo
make -C linux_app gemv_dma_smoke
./linux_app/gemv_dma_smoke --case golden/fake_gemv --mode scaled
```

Codex 검증: DMA 앱 출력, dmesg, GEMV status register dump를 Codex에 넣고 timeout/DMA length/cache/output mismatch 중 어느 문제인지 분류시킨다.

통과 기준: fake_gemv부터 bit-exact로 맞아야 한다. 그 다음 layer0_q_proj 일부로 확장한다.

실패하면: timeout인지, DMA length 문제인지, output mismatch인지 나눠서 로그를 남긴다.

15.1.22 Prompt 21 전후 - CPU-only minimal runtime

왜 하는가: FPGA 없이도 SmolLM2 한 토큰 forward가 돌아가야 FPGA backend를 붙였을 때 비교 기준이 생긴다.

누가 하는가: 사용자.

언제 하는가: PC reference와 보드 파일 로딩 경로가 어느 정도 안정된 뒤.

어디서 하는가: 먼저 PC, 이후 BOARD_LINUX.

입력: GGUF, tokenizer, tensor map, Q8_0 decoder.

통과 기준: 한 토큰 logits가 생성되고, token id가 vocab 범위 안에 있어야 한다. 같은 입력에 같은 결과가 나와야 한다.

실패하면: FPGA와 무관한 runtime 문제다. tokenizer, tensor name, shape, layer loop부터 본다.

15.1.23 Prompt 22 전후 - GEMV backend 추상화

왜 하는가: 모델 runtime 코드가 CPU GEMV와 FPGA GEMV를 같은 함수 인터페이스로 호출해야 나중에 교체가 쉽다.

누가 하는가: 사용자 또는 runtime 담당자.

언제 하는가: CPU-only runtime과 FPGA GEMV 단독 테스트가 각각 통과한 뒤.

어디서 하는가: PC와 BOARD_LINUX.

통과 기준: backend 옵션만 바꿔 CPU reference와 FPGA 결과를 비교할 수 있어야 한다. FPGA 실패 시 CPU fallback이 가능해야 한다.

실패하면: 모델 전체를 고치지 말고 GEMV 호출 interface만 분리해서 고친다.

15.1.24 Prompt 23 전후 - CPU 부가 연산 검증

왜 하는가: Transformer는 GEMV만으로 끝나지 않는다. RMSNorm, RoPE, attention, softmax, SiLU/SwiGLU가 CPU에서 맞게 돌아가야 한다.

누가 하는가: runtime 담당자.

언제 하는가: CPU-only forward를 만들 때.

어디서 하는가: PC 우선, 이후 BOARD_LINUX.

무엇을 확인하는가: RMSNorm output, RoPE 전후 q/k, attention score, softmax sum, SwiGLU output.

통과 기준: NaN/Inf가 없어야 하고, Python reference와 오차가 허용 범위 내여야 한다.

실패하면: GEMV가 아니라 부가 연산 문제일 수 있으니 연산별 unit test로 쪼갬다.

15.1.25 Prompt 24 전후 - KV cache

왜 하는가: 이전 토큰들의 K/V를 저장해야 다음 토큰을 생성할 때 과거 문맥을 다시 볼 수 있다.

누가 하는가: runtime 담당자.

언제 하는가: 한 토큰 forward가 된 뒤, 여러 토큰 생성으로 넘어가기 전.

어디서 하는가: Linux C runtime 메모리. 초기에는 CPU/DDR에서 관리한다. FPGA가 직접 관리하지 않는다.

초기 설정: max context는 128 또는 256으로 시작한다. 구조는 8192까지 확장 가능하게 만들되, 처음부터 최대 크기를 무조건 할당하지 않는다.

통과 기준: layer별 K/V가 섞이지 않고, position이 0,1,2 순서대로 증가하며, 대화 reset 시 cache가 초기화되어야 한다.

실패하면: context overflow, layer/head index, KV head mapping을 먼저 본다.

15.1.26 Prompt 25 전후 - Linux console chat

왜 하는가: HDMI와 USB keyboard가 이미 동작한다면 UART serial chat을 1차 UI로 만들 필요가 없다. stdin/stdout 콘솔 앱이면 HDMI, UART, SSH에서 모두 실행할 수 있다.

누가 하는가: 보드 Linux 담당자와 runtime 담당자.

언제 하는가: CPU-only 또는 FPGA backend가 최소 한 토큰 생성할 수 있을 때.

어디서 하는가: BOARD_LINUX의 HDMI console 또는 SSH.

어떻게 실행하는가:

```
./smollm2_chat --model /mnt/sd/model.gguf --backend cpu --max-new-tokens 32
./smollm2_chat --model /mnt/sd/model.gguf --backend fpga --max-new-tokens 32
```

통과 기준: 키보드 입력을 받고 token을 하나씩 출력해야 한다. FPGA 오류가 나면 CPU fallback 메시지가 보여야 한다.

실패하면: UI 문제인지, model load 문제인지, backend 문제인지 분리한다.

15.1.27 Prompt 26 전후 - HDMI/USB console 확인과 UI polish

왜 하는가: 이미 Linux console이 되면 별도 framebuffer/HID 직접 구현은 필수가 아니다. 발표용으로 보기 좋게 다듬는 단계다.

누가 하는가: 보드 Linux 담당자.

언제 하는가: console chat이 동작한 뒤.

어디서 하는가: BOARD_LINUX.

확인 명령:

```
ls /dev/fb* 2>/dev/null || true
ls /dev/input/ 2>/dev/null || true
cat /proc/bus/input/devices 2>/dev/null || true
```

통과 기준: HDMI console에서 같은 chat binary가 실행되고 USB keyboard 입력이 된다.

실패하면: chat 앱을 고치기 전에 Linux console/keyboard 설정 문제인지 본다.

15.1.28 Prompt 27 전후 - Vivado report parser

왜 하는가: 자원 사용량과 timing을 수동으로 매번 읽으면 실수한다. report를 파싱해 변화 추이를 봐야 한다.

누가 하는가: Vivado 담당자.

어디서 하는가: VIVADO_PC.

입력: utilization.rpt, timing_summary.rpt, power.rpt.

통과 기준: LUT/FF/BRAM/DSP, WNS/TNS가 Vivado GUI와 일치해야 한다.

15.1.29 Prompt 28 전후 - 성능 측정 logger

왜 하는가: FPGA가 실제로 어디서 빠르고 어디서 느린지 알아야 다음 최적화 방향을 잡을 수 있다.

누가 하는가: 사용자.

어디서 하는가: PC와 BOARD_LINUX.

측정 항목: model load, GEMV 시간, DMA 시간, CPU 부가 연산 시간, first token latency, tokens/s, 메모리 사용량.

통과 기준: CPU-only와 FPGA backend가 같은 prompt로 비교되고, CSV/JSON 로그가 남아야 한다.

15.1.30 Prompt 29 전후 - fallback script

왜 하는가: 발표 중 FPGA가 timeout 나도 전체 데모가 죽지 않게 CPU backend나 작은 GEMV demo로 빠지는 길이 필요하다.

누가 하는가: 사용자.

통과 기준: 일부러 FPGA timeout을 만들었을 때 CPU fallback으로 넘어가고, 로그에 원인이 남아야 한다.

15.1.31 Prompt 30 전후 - 최종 demo checklist

왜 하는가: 발표 직전에 매번 손으로 기억해서 실행하면 빠뜨린다. 같은 SD카드/전원/bitstream/model로 재현되는 절차가 필요하다.

누가 하는가: 전원.

체크 항목: 부팅, HDMI/keyboard, model path, bitstream version, chat 실행 명령, fallback 명령, 예상 출력.

통과 기준: 인터넷 없이 처음부터 끝까지 데모가 2회 이상 재현되어야 한다.

15.1.32 Prompt 31 전후 - 전체 regression runner

왜 하는가: 뒤에서 고친 코드가 앞 단계의 GGUF decode, layout, golden, C reference, RTL simulation을 깨지 않았는지 한 번에 확인한다.

누가 하는가: 사용자.

어디서 하는가: PC와 필요 시 BOARD_LINUX.

통과 기준: GGUF 검사, tensor map, Q8 decode, layout round-trip, golden generation, C GEMV, RTL simulation, register test, DMA fake GEMV, chat smoke가 자동으로 PASS/FAIL을 낸다.

15.1.33 Prompt 32 전후 - 직접 양자화 fallback

왜 하는가: Q8_0 GGUF 경로가 깨졌을 때만 쓰는 비상 경로다.

언제 하는가: 기본 GGUF-first 루트가 실패했을 때만.

주의: 8~9일 MVP에서는 기본 작업이 아니다. 실행하면 일정이 늘어난다.

15.1.34 Prompt 33 전후 - 360M Q4_K_M 확장

왜 하는가: 135M이 끝까지 동작한 뒤 품질을 올리기 위한 확장이다.

언제 하는가: 135M Q8_0 chat, FPGA backend, DMA, 성능 측정이 모두 끝난 뒤.

통과 기준: 기존 135M regression이 깨지지 않아야 한다. 360M이 보드 메모리에 올라가고 최소 한 토큰을 생성해야 한다.

15.1.35 Prompt 34 전후 - 문서 업데이트 자동화

왜 하는가: 실제 파일명, 실행 명령, register map이 문서와 어긋나면 팀원이 따라 할 수 없다.

누가 하는가: 사용자.

어디서 하는가: PC.

통과 기준: 문서의 명령을 복사해 실행했을 때 실제 프로젝트에서 동작해야 한다. 자동 업데이트가 기존 설명을 삭제하지 않았는지 diff를 확인한다.

15.1.36 Prompt 35 전후 - 최종 통합 검토

왜 하는가: 더 이상 새 기능을 넣지 않고, 처음부터 끝까지 재현 가능한 상태인지 확인하는 단계다.

누가 하는가: 전원.

무엇을 하는가: 새 clone 또는 깨끗한 환경에서 PC 도구, bitstream, Linux boot, chat 실행, fallback, 성능 로그까지 다시 수행한다.

통과 기준: 문서만 보고 같은 결과를 재현할 수 있어야 한다.

15.1.37 중요한 저장점

각 지점이 통과하면 git tag를 남긴다. tag는 “돌아갈 수 있는 안전지점”이다.

```
git tag pc-model-tools-passed
git tag c-reference-passed
git tag rtl-1lane-passed
git tag rtl-multilane-passed
git tag vivado-bitstream-passed
git tag linux-register-access-passed
git tag dma-gemv-passed
git tag cpu-runtime-passed
git tag console-chat-passed
git tag final-demo
```

실패한 상태에서는 tag를 찍지 않는다. tag는 “Codex가 성공했다고 말한 지점”이 아니라 “사용자가 직접 실행해서 PASS를 확인한 지점”이다.

16. 프로젝트 디렉터리 구조

권장 구조는 v9에서 실제 프롬프트와 실행 카드가 사용하는 경로에 맞춰 정리한다. 오래된 hw/, sw/, models/ 중심 구조보다 아래 구조를 우선한다.

```
smollm2-zybo/
├── docs/
│   ├── project_guide.pdf
│   ├── interface_contract.md
│   ├── c_reference_howto.md
│   ├── rtl_sim_howto.md
│   ├── axi_stream_q8_0_packet_protocol.md
│   ├── gemv_register_map.md
│   └── vivado_block_design_steps.md
├── quantized_model/
│   └── original_gguf/
│       └── SmollM2-135M-Instruct-Q8_0.gguf
├── fpga_layout/
│   ├── tensor_map.json
│   └── q8_0_lane16/
│       ├── manifest.json
│       ├── *weight*.bin
│       └── *scale_q*.bin
├── golden/
│   ├── fake_gemv/
│   ├── layer0_q_proj/
│   └── layer0_gate_proj/
├── pycharm/
│   ├── pc_tools/
│   │   ├── download_gguf_model.py
│   │   ├── inspect_gguf.py
│   │   ├── tokenizer_smoke_test.py
│   │   ├── create_tensor_map.py
│   │   ├── validate_tensor_map.py
│   │   └── convert_q8_0_to_fpga_layout.py
│   ├── pc_reference/
│   │   ├── q8_0_decode_ref.py
│   │   └── golden_compare.py
│   └── generate_golden_from_gguf.py
├── runtime_c/
│   ├── CMakeLists.txt
│   ├── gemv_q8_0_ref.c
│   ├── gemv_q8_0_ref.h
│   ├── gemv_q8_0_test.c
│   ├── weight_stream_pack.c
│   ├── weight_stream_pack.h
│   └── gemv_regs.h
├── vivado_ip/
│   ├── rtl/
│   │   ├── gemv_q8_0_stream_core.v
│   │   ├── gemv_axi_lite_regs.v
│   │   └── gemv_dma_stream_top.v
│   ├── tb/
│   │   ├── tb_gemv_q8_0_stream_core.v
│   │   ├── tb_gemv_q8_0_1lane.v
│   │   ├── tb_gemv_q8_0_multilane.v
│   │   └── weight_stream_protocol_tb.v
│   └── scripts/
│       └── create_bd_gemv_q8_0.tcl
├── scripts/
│   ├── run_gemv_sim.tcl
│   ├── run_gemv_sim_1lane.tcl
│   └── run_gemv_sim_multilane.tcl
```

```

|   |—— run_weight_stream_protocol_tb.tcl
|   |—— run_axi_lite_regs_tb.tcl
|   |—— run_dma_fifo_wrapper_tb.tcl
|   |—— create_block_design.tcl
|—— linux_app/
|   |—— Makefile
|   |—— gemv_control.c
|   |—— gemv_dma_smoke.c
|   |—— dma_buffer.c
|   |—— dma_buffer.h
|—— reports/
|   |—— gguf_inspect/
|   |—— tensor_map_validation.txt
|   |—— golden_generation_report.txt
|   |—— vivado_utilization.csv
|   |—— runtime_perf.csv
|—— logs/
|   |—— gemv_sim_result.txt
|   |—— gemv_sim_1lane_result.txt
|   |—— linux_env.txt
|   |—— dmesg_boot.txt

```

규칙은 단순하다.

실행 카드에서 실행하는 파일은 위 구조 안에 있어야 한다.

없으면 해당 Prompt가 미완료다.

파일 위치를 바꾸고 싶으면 실행 카드와 Prompt 산출물도 함께 바꾼다.

17. 수치 검증 전략

FPGA 프로젝트에서 제일 무서운 것은 “출력은 나오는데 맞는지 모르는 상태”다. v7에서는 Q8_0 scale을 FPGA fixed-point로 적용하므로, raw MAC 검증과 scaled output 검증을 분리한다.

반드시 네 단계 reference를 둔다.

1. Python float Q8_0 reference
2. Python fixed-point scale_q reference
3. C/C++ fixed-point reference
4. PL GEMV result

각 단계마다 같은 test vector를 써야 한다.

17.1 작은 test vector

```

input_size = 8 또는 32
output_size = 4
q8_block_size = 32
lanes = 1부터 시작

```

작은 벡터는 손계산이 가능해야 한다. scale_q도 사람이 확인할 수 있게 단순값을 포함한다.

17.2 실제 크기 test vector

```

135M hidden GEMV:
input_size = 576
output_size = 576
blocks_per_row = 18

```

```

135M MLP up/gate:
input_size = 576
output_size = 1536
blocks_per_row = 18

```

```
135M LM head:
input_size = 576
output_size = 49152
blocks_per_row = 18
```

17.3 성공 기준

INT8 양자화에서는 float 원본과 완전 일치할 필요는 없다. 하지만 다음은 일치해야 한다.

- Python fixed-point vs C fixed-point block_acc debug: bit-exact
- Python fixed-point vs C fixed-point scaled output_i32: bit-exact
- C reference block_acc debug vs PL mode=1 block_acc debug: bit-exact
- C reference scaled output_i32 vs PL mode=0 scaled output_i32: bit-exact
- Python float Q8_0 reference vs fixed-point scaled output: 허용 오차 내

mode=0만 맞고 mode=1을 보지 않으면 scale/weight alignment 오류를 놓칠 수 있다. 따라서 RTL이 틀리면 먼저 mode=1 block_acc debug부터 본다.

18. 속도 감각

매 토큰마다 대략적인 MAC 수를 계산하면 다음 정도다.

18.1 135M 근사

```
hidden = 576
intermediate = 1536
layers = 30
vocab = 49152
```

per layer 주요 GEMV:

```
q_proj: 576 x 576
k_proj: 576 x 192
v_proj: 576 x 192
o_proj: 576 x 576
MLP gate/up/down: 3 x 576 x 1536
```

대략 100M+ MAC/layer 전체 + LM head 28M MAC
총 대략 130M MAC/token 근처

18.2 360M 근사

```
hidden = 960
intermediate = 2560
layers = 32
vocab = 49152
```

총 대략 350M+ MAC/token 근처

실제 값은 구현 방식, tied embedding, cache, quant format에 따라 변한다. 그래도 감각적으로 360M은 135M보다 약 2.5~3배 무겁다.

18.3 lane 수별 감각

MAC lanes	100 MHz 이상적 처리량	135M 예상	360M 예상
1	100 MMAC/s	느림	매우 느림
4	400 MMAC/s	가능	느림
8	800 MMAC/s	현실적 목표	가능하지만 느림
16	1.6 GMAC/s	좋은	괜찮음, 구현 난이도 증가

DDR bandwidth, DMA overhead, CPU scheduling 때문에 실제 속도는 이론보다 느리다. 처음 성능 목표는 “빠름”이 아니라 **정상 동작**이다.

19. 자주 터지는 문제와 예방책

문제	원인	예방책
FPGA 결과가 Python과 다름 DMA는 도는데 데이터가 이상함 Linux에서 메모리 부족 Vivado timing fail AXI handshake deadlock Codex가 큰 코드 망침 SD 카드 읽기 느림	scale, sign, endian, row-major 오류 cache coherency full GUI, 360M Q8, 큰 buffer lanes 과다, routing 복잡 tready/tvalid 규칙 위반 한 번에 너무 많이 시킴 모델을 매 토큰 SD에서 읽음	작은 vector부터 bit-exact 검증 coherent buffer, dma-proxy/u-dma-buf 사용 headless, 135M Q8, 360M Q4 1->4->8 lane 순서로 확장 testbench에서 stall 조건 넣기 작은 모듈+testbench 단위로 요청 시작 시 DDR에 로드 또는 mmap/pread chunk c

20. 최종 UI 계획

20.1 1차: Linux console chat, HDMI/USB keyboard 우선

팀원이 준비한 Linux에서 framebuffer와 HID가 이미 동작한다면 최종 UI의 1차 목표는 다음과 같다.

```
HDMI monitor + USB keyboard
-> Linux console 또는 tty
-> ./smollm2_chat
```

앱은 stdin/stdout 기반으로 만든다. 그러면 HDMI console, SSH, UART console에서 같은 binary를 그대로 실행할 수 있다.

장점:

- 별도 framebuffer UI를 만들 필요 없음
- USB HID 이벤트를 직접 파싱하지 않아도 됨
- Linux가 이미 제공하는 console 입력/출력을 그대로 사용
- 데모 시 모니터와 키보드만 꽂으면 사용 가능

20.2 2차: SSH 또는 network console

Ethernet 설정이 되면 SSH로 접속해서 같은 앱을 실행한다. 개발 중에는 SSH가 로그 복사와 파일 전송에 더 편할 수 있다.

20.3 UART serial은 boot/debug fallback

UART serial은 더 이상 기본 채팅 UI가 아니라 비상 디버그용이다.

UART serial로 할 일:

```
boot log 확인
HDMI/USB가 안 될 때 복구
dmesg 확인
네트워크 실패 시 console 접근
```

따라서 v6 기준 UI 우선순위는 다음과 같다.

1. HDMI + USB keyboard + Linux console chat
2. SSH console chat
3. UART serial debug console
4. 직접 framebuffer/HID UI, optional

처음부터 직접 framebuffer나 /dev/input/eventX 기반 UI를 만들면 시간이 샌다. Linux console이 살아 있으면 그걸 쓰는 것이 가장 빠르고 안정적이다.

21. 8~9일 초단기 애자일 일정

이 장은 기존 주차 단위 일정표를 대체한다. 이 프로젝트는 8~9일짜리로 본다. 매일 끝날 때 “돌아가는 것”을 남기는 것이 핵심이다.

21.1 전체 원칙

매일의 산출물:

- 실행 가능한 코드
- 검증 로그
- 실패 시 되돌아갈 수 있는 백업

금지:

- 2일 이상 결과 없는 대형 리팩터링
- 검증 없는 IP 확장
- 모델/보드/OS를 동시에 바꾸기

허용:

- 느린 구현
- CPU fallback
- 일부 Linear만 PL 가속
- serial-only UI

21.2 Day 1 - Q8_0 GGUF 다운로드와 모델 파일 검사

목표:

SmolLM2-135M-Instruct Q8_0 GGUF를 PC에 다운로드
GGUF metadata와 tensor 목록 확인
tokenizer 동작 확인

작업:

1. PyCharm/PC Python 환경 구성
2. Q8_0 GGUF 다운로드 스크립트 실행
3. GGUF 파일 크기, tensor 목록, quant type 확인
4. 원본 Hugging Face tokenizer 로드
5. "Hello, how are you?" encode/decode smoke test
6. tensor_map.json 초안 생성

완료 조건:

SmolLM2-135M-Instruct-Q8_0.gguf가 quantized_model/original_gguf/에 존재한다.
reports/gguf_inspect/tensors.csv와 tensor_map.json이 생성된다.
tokenizer smoke test가 통과한다.

21.3 Day 2 - Q8_0 decoder, FPGA layout, golden vector

목표:

Q8_0 block decoder reference를 만든다.
GGUF tensor를 FPGA lane16 layout으로 변환한다.
C/Verilog가 쓸 golden vector를 생성한다.

작업:

1. Q8_0 block decode reference 작성
2. layer0_q_proj 일부 slice decode 검증
3. lane_count=16 기준 weight 재배열 함수 작성
4. layout 변환 -> 역변환 테스트 작성
5. fake_gemv, layer0_q_proj, layer0_gate_proj golden 생성
6. manifest.json 작성

완료 조건:

작은 행렬에서 layout 변환 -> 역변환이 bit-exact로 맞는다.
golden/fake_gemv와 golden/layer0_q_proj가 생성된다.
C reference와 Verilog testbench가 같은 raw binary를 읽을 수 있다.

21.4 Day 3 - Zybo Linux 부팅, HDMI/USB console, SD 파일 읽기

목표:

Zybo Z7-20에서 Linux serial console 부팅
SD 카드의 모델/테스트 파일 읽기

작업:

1. Vivado에서 Zynq PS only 또는 최소 PL 포함 XSA 생성
2. PetaLinux 프로젝트 생성
3. BOOT.BIN/image.ub 생성
4. SD 카드 파티션 구성
5. UART 115200으로 부팅 로그 확인
6. /home/root/models 아래 파일 읽기 테스트

완료 조건:

serial console에 로그인 가능하다.

C 프로그램 또는 shell에서 SD 카드의 test_vector.bin을 읽을 수 있다.

21.5 Day 4 - CPU-only C runtime 최소화

목표:

Zybo Linux에서 작은 Transformer 조각 또는 최소 GEMV C reference 실행

작업:

1. PC Python에서 만든 test vector를 SD에 복사
2. Zynq ARM C 코드로 int8 GEMV reference 작성
3. RMSNorm/RoPE/Softmax는 일단 CPU C 함수로 작성
4. 작은 vector/matrix 결과를 Python expected와 비교
5. 실행 시간 대략 측정

완료 조건:

Zybo ARM에서 int8 GEMV C reference가 Python expected와 일치한다.

21.6 Day 5 - Vivado AXI-Lite + 작은 Q8_0 fixed-scale GEMV IP

목표:

PL에 작은 Q8_0 fixed-scale GEMV IP를 만들고 AXI-Lite로 start/done/mode/scale_shift 제어

작업:

1. custom AXI-Lite IP 생성
2. register map 작성: start, done, mode, scale_shift, input_size, output_size, debug
3. GEMV core는 처음에 1-lane, 작은 fake_gemv만 처리
4. mode=1 block_acc debug testbench 작성
5. mode=0 scaled output testbench 작성
6. Block Design에 Zynq PS + custom IP 연결
7. bitstream 생성

완료 조건:

testbench에서 작은 GEMV의 block_acc와 scaled output이 모두 맞는다.

Vivado Address Editor에서 custom IP 주소가 할당된다.

21.7 Day 6 - Linux에서 GEMV IP 호출

목표:

Linux user-space에서 /dev/mem 또는 UIO로 GEMV IP를 호출

작업:

1. 새 bitstream 포함 BOOT.BIN 또는 fpga manager 방식 준비
2. Linux에서 AXI-Lite base address mmap
3. MODE=1로 block_acc debug 먼저 실행
4. MODE=0으로 scaled output 실행
5. 작은 input/weight/scale_q/output buffer 연결
6. 결과를 C reference와 비교

완료 조건:

Linux C 프로그램에서 PL GEMV를 실행하고 block_acc와 scaled output을 모두 읽는다.

21.8 Day 7 - AXI DMA 또는 FIFO 기반 Q8_0 packet stream 1차 연결

목표:

DDR의 Q8_0 weight/scale_q packet을 GEMV IP로 공급하는 경로를 만든다.

작업:

1. AXI DMA MM2S 추가
2. AXI4-Stream Data FIFO 추가
3. GEMV IP에 AXI-Stream slave 입력 추가
4. output은 일단 BRAM 또는 AXI-Lite debug read로 단순화
5. ILA로 TVALID/TREADY/TLAST, scale_q, block_idx 확인
6. mode=1과 mode=0을 모두 golden과 비교

완료 조건:

DMA에서 나온 Q8_0 packet stream이 GEMV IP까지 들어간다.

작은 행렬 1개가 DMA 경유로 block_acc와 scaled output 모두 맞는다.

DMA가 막히면 fallback:

AXI DMA는 포기하고 Day 6 구조로 발표 가능한 demo를 만든다.

21.9 Day 8 - SmolLM2 runtime에 PL GEMV 붙이기

목표:

SmolLM2 runtime의 Linear 하나 이상을 PL GEMV로 교체

Linux console chat demo 확보

작업:

1. q_proj 또는 작은 Linear 하나를 PL GEMV로 교체
2. CPU-only 결과와 PL 결과 비교
3. 문제가 없으면 MLP up/down 또는 LM head 일부로 확대
4. Linux console 또는 SSH에서 prompt 입력 -> token 출력 확인. UART는 fallback으로만 사용

완료 조건:

serial console에서 모델이 실제 token을 생성한다.

최소 하나 이상의 Linear가 PL GEMV를 사용한다.

21.10 Day 9 - 안정화와 발표용 확장

Day 9가 있으면 새 기능보다 안정화를 우선한다.

우선순위:

1. demo script 고정
2. 실패 시 CPU fallback 경로 확보
3. 로그 저장
4. 결과 재현성 확인
5. ILA 제거 또는 debug build/final build 분리
6. 속도 측정 표 작성
7. 여유가 있으면 HDMI/USB 또는 16 lane 확장

21.11 8일 프로젝트에서 포기해야 할 것

아래는 시간이 남을 때만 한다.

360M Q4/Q8 실기동

HDMI + USB keyboard UI

AXI master 직접 구현

RMSNorm/RoPE/Softmax PL 구현

완전한 top-k stream hardware

32 lane 이상 고성능화

8일 안에 가져가야 하는 최종 결과물은 다음 중 하나다.

A안, 최고:

135M Q8 Linux console chat + 여러 Linear PL GEMV 가속

B안, 현실:

135M Q8 Linux console chat + 일부 Linear PL GEMV 가속

C안, 방어:

CPU-only Linux console chat + PL GEMV 단독 demo + 통합 계획

22. 번외: AXI master를 직접 만들고 싶다면

본편에서는 하지 않는다.

AXI master를 직접 만들면 IP가 DDR에서 weight를 직접 읽을 수 있어 구조가 깔끔해진다. 하지만 burst read, outstanding transaction, alignment, HP port, cache coherency, address generator까지 신경써야 한다.

번외 목표:

GEMV IP 내부 AXI4 master read engine

- input_addr
- weight_addr
- output_addr
- burst read
- local FIFO
- MAC engine
- burst write

본 프로젝트에서는 AXI DMA가 이 역할을 대신한다.

23. Linux 설치 절차: Zybo Z7-20 기준

이 장은 "Linux를 쓴다면 실제로 뭘 해야 하는가"를 정리한다. 결론부터 말하면, 이 프로젝트에서 말하는 Linux는 Ubuntu Desktop 같은 범용 PC 리눅스가 아니라 Zynq용 PetaLinux 또는 가벼운 임베디드 Linux 이미지다.

23.1 권장 결론

빠른 완성 기준 추천은 다음과 같다.

1차 성공 목표:

- PetaLinux 부팅
- serial console 로그인
- SD 카드의 모델 파일 읽기
- /dev/mem 또는 UIO로 AXI-Lite register 접근
- AXI DMA는 나중에 붙임

2차 목표:

- DMA proxy/UIO/custom driver로 AXI DMA 사용
- GEMV IP에 weight stream 공급

3차 목표:

- USB keyboard + HDMI console/framebuffer UI

처음부터 HDMI/USB UI를 목표로 잡지 않는다. 반드시 serial console에서 먼저 모델이 한 토큰이라도 생성되는 것을 확인한다.

23.2 준비물

PC:

Vivado/Vitis/PetaLinux, 가능하면 같은 major version 사용
Ubuntu 계열 Linux host 또는 WSL2가 아닌 진짜 Linux 권장

보드:

Zybo Z7-20
microSD 16GB 이상, 현재 보유 64GB면 충분
USB-UART 케이블 또는 보드의 UART-over-USB
Ethernet, 선택

파일:

Vivado에서 export한 XSA
bitstream 포함 XSA 권장
PetaLinux project
BOOT.BIN
image.ub
boot.scr, 필요한 경우
rootfs 또는 WIC image
model.bin / tokenizer files

PetaLinux의 SD 부팅은 AMD 문서 기준으로 SD 카드에 필요한 이미지를 직접 복사하거나 WIC 이미지를 flash하는 방식으로 진행할 수 있다. [R11]

23.3 Vivado에서 XSA 만들기

Linux를 올리려면 먼저 하드웨어 설명 파일인 XSA가 필요하다.

Vivado에서:

1. Board: Zybo Z7-20 선택
2. Block Design 생성
3. ZYNQ7 Processing System 추가
4. Run Block Automation
5. DDR, FIXED_IO 연결 확인
6. FCLK_CLK0 활성화, 예: 100 MHz
7. M_AXI_GPO 활성화: AXI-Lite control용
8. S_AXI_HPO 활성화: 나중에 DMA/DDR 고속 이동용
9. AXI Interconnect 또는 SmartConnect 추가
10. custom GEMV control IP 연결, 없으면 AXI GPIO/BRAM만으로 시작
11. Validate Design
12. Generate Bitstream
13. File > Export > Export Hardware, Include bitstream 체크
14. .xsa 저장

초기 Linux bring-up만 할 때는 custom GEMV IP가 없어도 된다. 처음에는 Zynq PS만으로 Linux가 부팅되는지 확인하고, 그 다음 PL IP를 붙여도 된다.

23.4 PetaLinux 프로젝트 생성 흐름

명령 흐름은 버전에 따라 조금씩 달라질 수 있지만 큰 흐름은 다음과 같다.

```
# PetaLinux 환경 로드, 경로는 설치 위치에 맞게 수정
source /opt/petalinux/settings.sh
```

```
# 프로젝트 생성
petalinux-create -t project --template zynq -n zybo_smollm2_linux
cd zybo_smollm2_linux
```

```
# Vivado에서 export한 XSA 반영
petalinux-config --get-hw-description=/path/to/xsa_directory
```

```
# kernel/rootfs/device-tree 설정
petalinux-config
```

```
petalinux-config -c kernel
petalinux-config -c rootfs
```

빌드

```
petalinux-build
```

boot image 생성

```
petalinux-package boot --fsbl images/linux/zynq_fsbl.elf \
    --fpga images/linux/system.bit \
    --u-boot \
    --force
```

최신 PetaLinux 문서에서는 SD 카드 부팅용 image를 수동 복사하거나 WIC image로 flash할 수 있다고 설명한다. [R11] Device tree를 수정해야 할 때는 PetaLinux 프로젝트의 `project-spec/meta-user/recipes-bsp/device-tree/files/` 아래 사용자 device-tree 파일을 수정하는 흐름이 공식 문서에 나온다. [R12]

23.5 SD 카드 구성

쉬운 방식은 WIC image를 쓰는 것이다.

방법 A, WIC 사용:

- generated .wic 또는 .wic.gz를 SD 카드에 기록
- 보드 boot mode를 SD로 설정
- 전원 인가

방법 B, 수동 복사:

- SD partition 1: FAT32 boot

 - BOOT.BIN

 - image.ub

 - boot.scr, 필요한 경우

- SD partition 2: ext4 rootfs

 - root filesystem

 - /home/root/models/model.bin

 - /home/root/models/tokenizer.json

64GB SD를 쓴다면 권장 파티션은 다음과 같다.

partition 1, FAT32, 512MB~1GB:
boot files

partition 2, ext4, 나머지 대부분:

- Linux rootfs

- model files

- logs

- test vectors

23.6 첫 부팅 체크

UART serial terminal 설정은 보통 다음과 같이 시작한다.

baudrate: 115200

8N1

flow control: none

부팅 후 확인할 것:

```
uname -a
```

```
cat /proc/cpuinfo
```

```
free -h
```

```
lsblk
```

```
mount
```

```
dmesg | less
```

모델 파일 확인:

```
mkdir -p /home/root/models
ls -lh /home/root/models
sha256sum /home/root/models/*
```

23.7 PL register 접근 방법

초기에는 driver를 만들지 말고, 제일 단순하게 간다.

1단계:

/dev/mem으로 AXI-Lite register 접근

2단계:

UIO로 register 접근

3단계:

DMA proxy 또는 custom kernel driver

초기 C 프로그램은 /dev/mem으로 AXI-Lite base address를 mmap()해서 start/done register를 읽고 쓸 수 있다. 완성도가 올라가면 UIO 또는 DMA proxy 방식으로 바꾼다. Xilinx Wiki의 DMA Proxy 구조는 Linux DMA Engine, character device, mmap, ioctl을 이용해 user space에서 DMA buffer를 다루는 예시를 제공한다. [R8]

23.8 Linux가 난이도를 올리나, 내리나

정답은 단계에 따라 다르다.

항목	Linux	Bare-metal
SD 파일 읽기	쉬움. 파일시스템 사용	FatFs 설정 필요
HDMI/USB keyboard	Linux가 훨씬 유리	직접 드라이버 지옥
UART serial fallback	쉬움	쉬움
DMA 고성능 제어	중간. driver/UIO 필요	중간. 캐시 flush 직접 관리
디버깅	ssh, gdb, printf, 파일 로그 가능	단순하지만 도구가 제한됨
메모리 오버헤드	있음	작음
최종 UI 확장성	좋음	낮음

이 프로젝트는 SD 카드 모델 파일, Linux console/SSH/USB/HDMI UI, 멀티스레드, 로그 파일이 모두 필요해질 가능성이 크다. 그래서 최종 형태는 Linux가 유리하다. 단, Linux 자체가 목적이 되면 시간이 새므로 **Linux console + 파일 읽기 + AXI register 접근**까지만 먼저 한다.

23.9 1GB DDR이 Linux 때문에 부족할까

135M Q8 기준으로는 부족할 가능성이 낮다.

대략 예산:

Linux kernel/rootfs/userspace: 수십~수백 MB

135M Q8 weight: 약 135~170 MB

activation/KV/cache/buffer: 수십~100MB대

여유: 충분한 편

360M Q4도 가능권이다.

360M Q4 weight: 약 200~260 MB 근처, 포맷 의존

Linux + runtime + buffer 포함해도 1GB 안에 넣을 수 있는 편

360M Q8은 가능은 하지만 여유가 줄어든다.

360M Q8 weight: 약 360~430 MB 근처

Linux, DMA buffer, KV cache, 임시 logits까지 같이 올라가면 관리가 필요

따라서 모델 선택은 다음처럼 한다.

빠른 완성:

135M Q8

품질 욕심 + 구현 여유:

360M Q4

24. Transformer 용어 사전: QKV, Embedding, Attention, RMSNorm

이 장은 DNN/CNN을 조금 아는 상태에서 SmolLM2 문서를 읽을 수 있게 하는 최소 용어 사전이다.

24.1 Token

문자열을 모델이 직접 읽는 게 아니라, tokenizer가 문자열을 숫자 ID 배열로 바꾼다.

"hello world"
↓
[토큰ID, 토큰ID, ...]

SmolLM2의 vocab size는 49152이므로, 다음 토큰 후보도 49152개다. [R2][R3]

24.2 Embedding

Embedding은 token ID를 벡터로 바꾸는 lookup table이다.

token_id
↓
embedding_table[token_id]
↓
hidden vector

135M에서는 hidden vector 길이가 576이고, 360M에서는 960이다. [R2][R3]

CNN식으로 비유하면, embedding은 "토큰을 feature map의 채널 벡터로 바꾸는 첫 층"에 가깝다.

24.3 Positional Encoding과 RoPE

Transformer는 순서를 자동으로 알지 못한다. 그래서 위치 정보를 넣어야 한다.

SmolLM2 같은 LLaMA 계열은 보통 RoPE, Rotary Position Embedding을 사용한다. RoPE는 Q/K 벡터 일부를 sin/cos로 회전시켜 위치 정보를 섞는다.

짝수/홀수 차원 쌍을 잡고
[x0, x1]을 위치 pos에 따라 회전

CPU 구현은 sin/cos table을 미리 만들면 어렵지 않다. PL 구현은 LUT와 fixed-point 회전이 필요하므로 초반에는 CPU에 맡긴다.

24.4 Q, K, V

Attention에서 각 토큰 벡터 x는 세 종류의 벡터로 바뀐다.

Q = Query
K = Key
V = Value

비유하면 다음과 같다.

Q: 내가 지금 찾고 싶은 것
K: 과거 토큰들이 가진 검색 태그
V: 실제로 가져올 내용

현재 토큰의 Q와 과거 토큰들의 K를 dot product한다.

$score[t] = Q \cdot K_cache[t]$

score가 크면 현재 토큰이 그 과거 토큰을 많이 참고한다는 뜻이다.

24.5 Attention

Attention은 과거 토큰들의 V를 가중합하는 연산이다.

```
scores = Q · K_cache  
weights = softmax(scores)  
out =  $\sum weights[t] \times V\_cache[t]$ 
```

CNN의 3x3 convolution은 항상 주변 3x3만 본다. Attention은 현재 토큰이 과거 토큰 전체 중 무엇을 볼지 입력에 따라 매번 바꾼다.

24.6 Multi-Head Attention

한 가지 관점만 쓰지 않고 여러 head로 나눠 attention을 한다.

135M:

```
attention heads = 9  
KV heads = 3
```

360M:

```
attention heads = 15  
KV heads = 5
```

KV head가 attention head보다 적은 것은 GQA, Grouped Query Attention 구조다. 여러 Q head가 일부 K/V head를 공유해서 KV cache 크기와 연산량을 줄인다.

24.7 KV Cache

KV cache는 과거 토큰의 K와 V를 저장해두는 메모리다. 이전 토큰의 K/V를 매번 다시 계산하지 않기 위해 사용한다.

새 token 처리:

```
q = 현재 token에서 새로 계산  
k, v = 현재 token에서 계산 후 cache에 저장
```

attention 계산:

```
현재 q와 과거 k_cache를 비교  
나온 가중치로 과거 v_cache를 섞음
```

이 프로젝트의 v1에서는 KV cache를 FPGA가 아니라 Linux C runtime이 DDR/RAM에 관리한다. FPGA는 GEMV 계산만 한다.

MVP는 context 128 또는 256으로 시작하고, 구조만 8192까지 확장 가능하게 만든다. 처음부터 8192를 모두 float32로 할당하면 135M에서도 수백 MB가 들 수 있어 디버깅에는 불리하다.

24.8 RMSNorm

RMSNorm은 벡터의 크기를 정규화하는 연산이다.

```
rms =  $\sqrt{\text{mean}(x[i]^2) + \text{eps}}$   
y[i] = x[i] / rms * weight[i]
```

BatchNorm처럼 batch 전체를 보는 게 아니라, 현재 token의 hidden vector 내부를 정규화한다. CPU에서 C로 구현하기는 쉽지만, PL에서는 sqrt/rsqrt가 들어가므로 귀찮다. 초반에는 CPU에 맡긴다.

24.9 MLP, SwiGLU, SiLU

Transformer block 안의 MLP는 token 벡터를 채널 방향으로 가공하는 부분이다.

LLaMA 계열은 보통 단순 ReLU MLP가 아니라 gate가 있는 구조를 쓴다.

```
up = W_up x  
gate = W_gate x  
hidden = SiLU(gate) * up  
out = W_down hidden
```

SiLU는 대략 다음 함수다.

```
SiLU(x) =  $x / (1 + \exp(-x))$ 
```

CPU에서는 expf() 또는 LUT로 처리 가능하다. PL에서는 LUT/근사기가 필요하므로 초반에는 CPU에 맡긴다.

24.10 LM Head와 logits

Transformer block을 모두 통과하면 마지막 hidden vector가 나온다. 이 벡터를 vocab 크기 방향으로 Linear 변환한다.

135M:

$x[576] \rightarrow \text{logits}[49152]$

360M:

$x[960] \rightarrow \text{logits}[49152]$

logit은 각 토큰 후보의 점수다. sampling은 이 점수 중 다음 토큰을 고르는 과정이다.

24.11 왜 GEMV가 핵심인가

위 용어가 많아 보여도, 전체 계산량 대부분은 Linear/GEMV다.

Q projection

K projection

V projection

O projection

MLP gate/up/down

LM head

이 모든 것이 결국 다음 형태다.

$$y[\text{row}] = \sum x[\text{col}] \times W[\text{row}][\text{col}]$$

그래서 PL에는 Transformer 전체가 아니라 GEMV IP를 만든다.

24.12 GEMV

GEMV는 General Matrix-Vector multiplication이다. 행렬 W와 벡터 x를 곱해 출력 벡터 y를 만드는 연산이다.

$$y = W x$$

SmolLM2에서 q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj, lm_head는 모두 이 형태로 볼 수 있다. 따라서 FPGA가 맡을 가장 중요한 작업은 GEMV다.

24.13 GGUF

GGUF는 LLM 모델 파일 포맷이다. Q8_0 GGUF 파일에는 8비트로 양자화된 weight tensor와 metadata가 들어 있다. 하지만 FPGA가 GGUF를 직접 파싱하지는 않는다. PC에서 GGUF를 읽고, 필요한 tensor를 FPGA가 쉽게 읽을 수 있는 lane layout binary로 변환한 뒤 Zybo에 넣는다.

GGUF 파일 -> PC 변환기 -> FPGA용 .bin + manifest.json -> SD/DDR -> DMA -> GEMV IP

25. 리소스 사용률 예상과 성능 확장 루트

Zybo Z7-20의 PL 자원은 처음 GEMV IP에는 꽤 남을 가능성이 크다. 하지만 자원이 남는다고 성능이 선형으로 늘지는 않는다. 어느 순간부터 병목은 DSP가 아니라 DDR 대역폭과 데이터 공급 구조가 된다.

25.1 예상 사용률

정확한 값은 Vivado synthesis/implementation report를 봐야 한다. 설계 전 감각은 다음 정도로 잡는다.

설계	LUT	DSP	BRAM	평가
AXI-Lite + 4~8 lane GEMV	10~25%	4~8%	10~25%	smoke test용, 널널
AXI DMA + 8~16 lane GEMV	25~55%	8~20%	25~55%	본편 권장
32 lane + double buffer	50~85%	15~40%	45~80%	성능형, 타이밍 주의
64 lane 이상	높음	DSP는 남아도 배선/DDR 병목	높음	Z7-20에서는 비추천

DSP는 남아도 weight를 매 클럭 충분히 공급하지 못하면 MAC lane이 논다. 따라서 lane 수 확장보다 DMA/FIFO/버퍼링으로 굼기지 않는 구조가 먼저다.

25.2 성능 확장 우선순위

권장 확장 순서:

1. 8 lane -> 16 lane -> 32 lane GEMV
2. AXI DMA + AXI4-Stream Data FIFO로 weight stream 안정화
3. ping-pong buffer로 계산과 데이터 이동 겹치기
4. Q4 weight decoder로 DDR traffic 감소
5. LM head top-k stream 처리
6. attention QK/V cache 일부 PL 가속
7. RMSNorm/RoPE/SwiGLU PL 이관, 후순위

초반에는 RMSNorm/RoPE/SwiGLU를 PL로 옮기지 않는다. 구현 난이도 대비 초반 성능 향상이 작고, CPU가 충분히 처리할 수 있다.

25.3 135M과 360M 확장 판단

135M Q8:

첫 완성 목표
구현 단순
DDR 여유 좋음

360M Q4:

2차 목표
품질 향상 기대
Q4 unpack/scale 처리 필요

360M Q8:

DDR에는 들어가도 weight traffic이 큼
프로젝트 기간이 짧으면 비추천

360M으로 가고 싶다면 먼저 135M Q8에서 다음 조건을 확인한다.

1. Linux에서 모델 파일 로드 성공
2. CPU-only reference와 bit-level 또는 tolerance 기반 일치
3. GEMV IP로 q_proj/k_proj/v_proj 중 하나 교체 성공
4. 모든 Linear를 PL 호출로 대체 성공
5. Linux console chat이 안정적으로 동작
6. Vivado report에서 LUT/BRAM/DSP 여유 확인
7. 토큰당 시간이 감당 가능

이 조건을 만족한 뒤 360M Q4로 넘어간다.

26. 버퍼, DMA, Vivado IP 활용 전략

이 프로젝트는 직접 RTL을 많이 만드는 방향이 아니다. 미리 만들어진 AMD/Vivado IP를 최대한 활용하고, 직접 만드는 것은 GEMV core와 얇은 제어 FSM 정도로 제한한다.

26.1 ping-pong buffer와 ring buffer

둘 다 가능하지만 본편에서는 ping-pong 또는 FIFO를 먼저 쓴다.

ping-pong buffer:

buffer A를 계산하는 동안 buffer B를 채움
다음 tile에서 A/B swap
단순하고 디버깅 쉬움

ring buffer:

read pointer/write pointer/full/empty 관리
더 유연하지만 디버깅 난이도 상승

링버퍼를 직접 만들지 말고, AXI4-Stream Data FIFO나 FIFO Generator를 쓰면 사실상 검증된 ring buffer 역할을 한다.

26.2 데이터 종류별 권장 버퍼

데이터	권장 구조	이유
input vector	BRAM buffer	여러 output row 계산에 재사용
Q8_0 weight/scale_q packet	AXI DMA + AXI4-Stream FIFO	순차 stream 후 버림
output vector, mode=0	처음엔 BRAM, 나중엔 S2MM DMA	row당 int32라 상대적으로 작음
debug output, mode=1	BRAM 또는 S2MM DMA	block_acc가 커서 필요할 때만 사용
KV cache	DDR, 일부 hot tile만 BRAM	크기와 context에 따라 변동
sin/cos/RoPE table	CPU 메모리 또는 작은 BRAM	초반은 CPU 처리

26.3 최대한 활용할 Vivado IP

IP	용도	비고
Zynq7 Processing System	ARM PS, DDR, SD, UART	필수
Processor System Reset	reset 정리	필수
AXI Interconnect/SmartConnect	AXI 연결	필수
AXI GPIO	초기 smoke test	LED/register 테스트용
AXI BRAM Controller	PS가 BRAM 접근	input/output buffer 디버깅용
Block Memory Generator	BRAM 생성	input/output/tile buffer
AXI DMA	DDR <-> AXI-Stream 이동	본편 데이터 이동
AXI4-Stream Data FIFO	DMA와 GEMV 사이 완충	Q8_0 packet stream 안정화
FIFO Generator	raw FIFO 필요 시	직접 ring buffer 대신 사용
ILA	내부 신호 디버깅	valid/ready/scale_q/block_acc/done 확인

AXI DMA는 memory-mapped AXI와 AXI4-Stream 주변장치 사이의 고대역폭 DMA를 제공하는 IP다. [R13] AXI4-Stream Data FIFO는 AXI4-Stream 사이에 넣는 FIFO IP이고, Vivado IP customization을 통해 depth와 memory type을 조정할 수 있다. [R14]

26.4 v1 권장 Block Design

초기 본편 구조:

Zynq7 PS

M_AXI_GP0 -> AXI Interconnect -> GEMV AXI-Lite control
-> AXI BRAM Controller -> input/output BRAM

DDR -> AXI DMA MM2S -> AXI4-Stream Data FIFO -> GEMV Q8_0 packet stream

GEMV mode=0 output -> output BRAM

GEMV mode=1 debug output -> output BRAM 또는 별도 debug BRAM

처음에는 S2MM DMA까지 넣지 않아도 된다. output은 BRAM에 쓰고 PS가 읽는다. mode=0 output은 row당 int32라 작고, mode=1 block_acc debug는 크므로 필요한 test에서만 사용한다.

26.5 v2 권장 Block Design

속도가 필요해지면 output도 DMA로 보낸다.

DDR -> AXI DMA MM2S -> AXIS FIFO -> GEMV Q8_0 packet stream

GEMV output stream -> AXIS FIFO -> AXI DMA S2MM -> DDR

다만 output vector는 weight에 비해 작으므로, v1에서는 BRAM 출력으로 충분하다. 먼저 weight/scale_q packet stream이 안정적으로 도는지가 중요하다.

26.6 weight layout은 PC에서 미리 바꿔라

multi-lane GEMV에서 성능을 내려면 weight 파일 배치를 FPGA lane에 맞춰야 한다. v7에서는 weight와 scale_q를 함께 맞춰야 한다.

일반 row-major:

```
row0: col0 col1 col2 ...
row1: col0 col1 col2 ...
```

16 lane 권장 packet layout:

```
row group 0~15, block 0:
  scale_q lane0..lane15
  col0의 16개 weight
  col1의 16개 weight
  ...
  col31의 16개 weight
```

```
row group 0~15, block 1:
  scale_q lane0..lane15
  col32..col63 weights
```

이렇게 저장하면 DMA stream에서 들어오는 scale과 16개 weight를 그대로 16개 lane에 먹일 수 있다. 이 재배열은 PyCharm/PC의 GGUF 변환 단계에서 처리한다.

26.7 valid/ready 규칙

AXI-Stream은 valid/ready handshake다. GEMV core는 다음 규칙을 지킨다.

S_AXIS_TVALID && S_AXIS_TREADY:
이 클럭에 packet word를 수신

TVALID=1, TREADY=0:
source는 같은 data를 유지

TVALID=0:
data는 의미 없음

이 규칙을 깨면 DMA/FIFO와 연결했을 때 랜덤하게 데이터가 밀리거나 깨진다. 특히 scale_q header와 weight payload 중간에서 stall이 걸려도 FSM이 packet 상태를 유지해야 한다.

26.8 ILA로 반드시 볼 신호

Vivado ILA를 넣으면 디버깅 난이도가 크게 내려간다.

control:
start
busy
done
mode
matrix_id
row_idx
block_idx

AXI stream:
s_axis_tvalid
s_axis_tready
s_axis_tdata
s_axis_tlast
packet_state

scale/MAC:
scale_q[0]
weight_i8[0]
block_acc[0]
scaled_block[0]
row_acc[0]
out_valid
error_code

26.9 v9 파일 의존성 전수조사 요약

v9에서는 실행 카드에 등장하는 파일이 어느 Prompt에서 생성되는지 다시 연결했다. 원칙은 다음과 같다.

실행 카드에서 `ls 파일`을 먼저 한다.

파일이 없으면 실행하지 않고, 해당 Prompt 산출물 누락으로 보고 Codex에게 생성 요청한다.

핵심 의존성은 다음과 같다.

실행 카드	실행/확인 파일	생성해야 하는 Prompt
15.1.5	pc_tools/validate_tensor_map.py	Prompt 04
15.1.8	generate_golden_from_gguf.py	Prompt 07
15.1.9	runtime_c/CMakeLists.txt, gemv_q8_0_test	Prompt 08
15.1.10	scripts/run_gemv_sim.tcl	Prompt 09
15.1.11	scripts/run_weight_stream_protocol_tb.tcl	Prompt 10
15.1.13	scripts/run_gemv_sim_1lane.tcl	Prompt 12
15.1.14	scripts/run_gemv_sim_multilane.tcl	Prompt 13
15.1.15	scripts/run_axi_lite_regs_tb.tcl	Prompt 14
15.1.16	scripts/run_dma_fifo_wrapper_tb.tcl	Prompt 15
15.1.17	scripts/create_block_design.tcl	Prompt 16
15.1.19	linux_app/gemv_control	Prompt 18
15.1.21	linux_app/gemv_dma_smoke	Prompt 20

전체 표는 별도 CSV로도 내보낸다.

27. 참고자료

[R1] Hugging Face, HuggingFaceTB/SmolLM2-135M-Instruct model card. SmolLM2 model family size and model summary. <https://huggingface.co/HuggingFaceTB/SmolLM2-135M-Instruct>

[R2] Hugging Face, HuggingFaceTB/SmolLM2-135M config.json. hidden_size 576, intermediate_size 1536, layers 30, heads 9, KV heads 3, vocab 49152, bfloat16. <https://huggingface.co/HuggingFaceTB/SmolLM2-135M/blob/main/config.json>

[R3] Hugging Face, HuggingFaceTB/SmolLM2-360M-Instruct config.json. hidden_size 960, intermediate_size 2560, layers 32, heads 15, KV heads 5, vocab 49152, bfloat16. <https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct/blob/main/config.json>

[R4] Hugging Face Docs, GGUF format. GGUF is optimized for loading/saving models and includes tensors plus metadata; PyTorch models can be converted to GGUF. <https://huggingface.co/docs/hub/en/gguf>

[R5] Digilent / Zybo Z7 Reference Manual. Zybo Z7 includes 1 GB DDR3L with 32-bit bus at 1066 MHz. <https://www.mouser.com/pdfdocs/zybo-z7-rm.pdf>

[R6] AMD, Zynq 7000 SoC Technical Reference Manual UG585. Zynq-7000 includes ARM Cortex-A9 processing system and programmable logic. <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM>

[R7] AMD, PetaLinux Tools Documentation UG1144, Preparing the SD Card. SD card has FAT32 boot partition and ext4 root filesystem partition. <https://docs.amd.com/r/en-US/ug1144-petalinux-tools-reference-guide/Preparing-the-SD-Card>

[R8] Xilinx Wiki, Linux DMA From User Space / DMA Proxy. Describes DMA Proxy design using Linux DMA Engine, character devices, mmap, and ioctl. <https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842418/Linux+DMA+From+User+Space>

[R9] AMD/Xilinx Wiki, AXI DMA Standalone Driver. AXI DMA driver supports simple DMA mode and scatter/gather mode. <https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842100/AXI+DMA+Standalone+Driver>

[R10] Xilinx Embedded Design Tutorials, Linux for Zynq-7000. Covers building/debugging Linux applications for Zynq-7000. <https://xilinx.github.io/Embedded-Design-Tutorials/docs/2021.1/build/html/docs/Introduction/Zynq7000-EDT/4-linux-for-zynq.html>

[R11] AMD, PetaLinux Tools Documentation UG1144, Booting a PetaLinux Image on Hardware with SD Card. Explains manual SD image copy and WIC image flashing flows. <https://docs.amd.com/r/en-US/ug1144-petalinux-tools-reference-guide/Booting-a-PetaLinux-Image-on-Hardware-with-SD-Card>

- [R12] AMD, PetaLinux Tools Documentation UG1144, Device tree customization through project-specific meta-user files. <https://docs.amd.com/r/en-US/ug1144-petalinux-tools-reference-guide/Customizing-the-Device-Tree>
- [R13] AMD, AXI DMA Product Guide PG021. AXI DMA provides high-bandwidth direct memory access between AXI4 memory-mapped and AXI4-Stream interfaces. https://docs.amd.com/r/en-US/pg021_axi_dma
- [R14] AMD, AXI4-Stream Infrastructure IP Suite PG085, AXI4-Stream Data FIFO. Describes AXI4-Stream Data FIFO and stream buffering use. <https://docs.amd.com/r/en-US/pg085-axi4stream-infrastructure/AXI4-Stream-Data-FIFO>
- [R15] Hugging Face, `lmstudio-community/SmolLM2-135M-Instruct-GGUF`, Q8_0 GGUF file page. Shows `SmolLM2-135M-Instruct-Q8_0.gguf` and file size around 145 MB. https://huggingface.co/lmstudio-community/SmolLM2-135M-Instruct-GGUF/blob/main/SmolLM2-135M-Instruct-Q8_0.gguf
- [R16] Hugging Face, `bartowski/SmolLM2-135M-Instruct-GGUF`, quantization table. Lists Q8_0 as a high quality/max available quant around 0.14 GB. <https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF>
- [R17] Hugging Face, `HuggingFaceTB/SmolLM2-360M-Instruct-GGUF`. Provides GGUF variants and usage options including Q8_0. <https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct-GGUF>
- [R18] `llama.cpp` / ggml quantization discussion and headers. Q8_0 is block-based quantization; a common Q8_0 block contains a fp16 delta/scale and 32 int8 quants. <https://github.com/ggml-org/llama.cpp/discussions/4068>